

The Fabius Function Ecosystem in Lean

A Complete, Current Walkthrough of the ProveIt Development



Core Fabius–Rvachev theory, exact arithmetic, probability,
Thue–Morse and q-series infrastructure, Fourier products,
Lambert coordinates, inverse theory, Volterra calculus, and computability

Prepared from the Lean 4 development in

`Analysis/FabiusFunction`

Pinned repository snapshot: 3 September 2026

`83428e0939114407e02b329621563f835d3e7c20`

Lean 4.32.0; mathlib v4.32.0; 740 direct public-facade imports

This document is a human-readable map of the kernel-checked source. Exact hypotheses and proof terms remain authoritative in Lean.

Abstract

The Fabius-function development in Vladimir Reshetnikov’s `ProveIt` repository is no longer a small formalization of two classical papers. At the snapshot pinned on the title page, the public facade `Analysis/FabiusFunction/Lean/FabiusFunction.lean` contains exactly 740 direct `import FabiusFunction.*` commands. The classical bounded Fabius function, the compactly supported Rvachev up-function, and the signed global extension remain the mathematical center, but the source tree now also carries large reusable libraries for Bell–Stirling algebra, formal power-series inversion, geometric interpolation and extrapolation, weighted uniform random series, Thue–Morse arithmetic and harmonic analysis, generalized sinc products, Stieltjes transforms, orthogonal-polynomial reconstruction, inverse functions, normalized and fractional Volterra operators, computable analysis, Lambert W , and a substantial q -series corpus.

This volume replaces the earlier snapshot-based walkthrough with a current, proof-oriented map. It preserves the detailed explanation of the classical construction, exact dyadic arithmetic, Fourier theory, paper APIs, negative Laplace analysis, saddle asymptotics, and Legendre approximation, while adding the module families that entered the public surface afterward. It also corrects the build instructions: with the present Lake toolchain neither `LAKE_JOBS=1` nor `-j 1` supplies the advertised worker bound, so the safe repository protocol is one module target per invocation after its dependencies have been compiled.

Completeness is handled at two levels. The narrative gives the dependency logic and theorem interfaces of every major family. Appendix A then records every direct public import, in exact facade order, so no newly exposed leaf is silently omitted. Historical source-module and declaration counts printed in date-stamped README blocks are not promoted to current facts; the live facade itself is used as the exact inventory that can be reproduced without interpretation.

Reader’s map

For the classical Fabius proof spine, read Chapters 3, 6, 8, 10, and 15. For the post-August expansion, read Chapters 2, 21, 22, 23, 24, 25, 26, and 27. Lean developers should also read Chapters 1, 4, 29, and 30. Appendix A is the exhaustive public-import ledger; Appendix D is the annotated classical proof spine; Appendix H explains the snapshot and counting method.

Scope, evidence, and conventions

The repository is active. All statements about the current tree in this walkthrough are pinned to commit 83428e0939114407e02b329621563f835d3e7c20, the head of the inspected main branch on 3 September 2026. A later branch tip may contain more modules or stronger theorems. The primary evidence is the Lean source itself, especially the import facade, its long module documentation comment, the core files `Basic.lean`, `Arithmetic.lean`, and `Existence.lean`, the paper aggregators, and the named leaves discussed below. The project README, `docs/PAPER_COVERAGE.md`, `docs/ASYMPTOTIC_COMPLETION_AUDIT.md`, and `AGENTS.md` are used for navigation and explicit status boundaries, not as substitutes for theorem bodies.

The phrase *direct public import* means one literal line of the form

```
import FabiusFunction.ModuleName
```

in `Lean/FabiusFunction.lean`. It does not mean that the listed file has no dependencies, and it is not the same as the total number of Lean source files in the transitive closure. The exact count 740 is therefore both useful and narrow: it describes the facade at the pinned commit. This document does not state a new total declaration count, because the README’s figure of 8,909 is explicitly dated 1 September 2026 and the facade acquired further imports after that checkpoint.

Mathematically, three functions must remain distinct:

- the bounded cumulative-distribution form $F: \mathbb{R} \rightarrow [0, 1]$, equal to 0 on the left of the unit interval and 1 on the right;
- the compactly supported even Rvachev function up, obtained by folding the bounded form and supported on $[-1, 1]$; and
- the signed global extension, assembled from Thue–Morse-signed translates of up, which satisfies the clean global dilation equation but is not globally bounded between zero and one.

Several historical normalization errors amount to applying a theorem about one of these objects to another. The Lean APIs intentionally keep them separate.

Lean identifiers are set in monospaced type, for example `Fabius.IsFabius`, `Fabius.fabiusDyadicValue`, `Fabius.fabiusInv`, and `AutonomousODE.iteratedDeriv_eq_comp`. File names are relative to `Analysis/FabiusFunction/Lean/FabiusFunction/` unless another base path is shown. Equations are written in conventional notation even when the formal statement uses subtypes, coercions, filters, interval integrals, or bundled continuous linear maps. A module name in the facade proves only that the module is publicly imported; mathematical claims in this volume are restricted to interfaces documented by the source.

Contents

Abstract	i
Scope, evidence, and conventions	ii
1 Building and entering the development	1
1.1 Workspace location and pinned toolchain	1
1.2 The current serialization rule	1
1.3 Scoped validation instead of a facade rebuild	2
1.4 A practical reading workflow	2
2 Current snapshot and the anatomy of the facade	4
2.1 The exact census used by this walkthrough	4
2.2 What the current public comment says is proved	5
2.2.1 Sharp global shape and the inverse	5
2.2.2 Derivative distributions	5
2.2.3 Small-argument asymptotics	5
2.2.4 Computable analysis	6
2.2.5 Generalized products, transfer operators, and q-series	6
2.2.6 Measure-level closure of the probability model	6
2.3 Why the old walkthrough became incomplete	6
3 The mathematical objects and their Lean interfaces	8
3.1 The bounded function	8
3.1.1 Why specify before constructing?	8
3.2 The Rvachev up-function	9
3.3 The signed global extension	9
3.4 Auxiliary analytic objects	9

4	The dependency architecture	11
4.1	A layered view	11
4.2	Why existence is not the first analytic file	12
4.3	Two independent uniqueness arguments	12
4.4	Paper aggregators versus theorem-producing modules	12
5	The exact-arithmetic foundation	13
5.1	Why the development begins over $\mathbb{N}$ and $\mathbb{Q}$	13
5.2	Binary weight and the Thue–Morse sign	13
5.3	Product normalizations	14
5.4	Moment and half-moment recurrences	14
5.5	Closed dyadic formulas	15
5.6	The inverse-power table	15
5.7	Horner evaluation	15
5.8	Recursive removal of the highest bit	16
5.9	Clamping, reflection, and the global evaluator	16
5.10	Recognizing dyadic rationals	17
5.11	Arithmetic API design lessons	17
6	Constructing the canonical bounded function	18
6.1	The ambient complete metric space	18
6.2	Extending an interval function	18
6.3	The nonlinear-looking transform that is actually affine	19
6.4	The contraction estimate	19
6.5	From an integral fixed point to the differential law	20
6.6	Bootstrapping smoothness	20
6.7	Uniqueness by dyadic density	20
7	Differential identities and the signed global extension	21
7.1	The role of <code>Differential.lean</code>	21
7.2	Domain bookkeeping	21
7.3	Scale and translation lemmas	22
7.4	Local finiteness of the translate sum	22
7.5	Agreement and global identities	22
7.6	Why use a <code>tsum</code> for a locally finite expression?	22

8	The exact-analytic dyadic bridge	23
8.1	The purpose of <code>DyadicAnalytic.lean</code>	23
8.2	Finite Taylor expansion on a dyadic scale	23
8.3	Inverse-power values	24
8.4	From closed polynomial to Horner program	24
8.5	Correctness of highest-bit reduction	24
8.6	Representation independence	25
8.7	Dyadic density and uniqueness	25
8.8	The global dyadic theorem	25
9	Moments, generating functions, and exact denominators	27
9.1	Analytic moments of the up-function	27
9.1.1	Integration over compact support	27
9.2	Identifying analytic and exact moments	28
9.3	Generating functions	28
9.4	Removable singularities	28
9.5	Normalized half moments	29
9.6	Normalized even moments	29
9.7	Parity through Lucas' theorem	29
9.8	Reduced numerators, denominators, and two-adic valuation	30
9.9	Denominator bounds	30
9.10	Bernoulli recurrences and source-facing arithmetic	30
10	Fourier analysis and Rvachev's original characterization	31
10.1	Why the up-function is the Fourier object	31
10.2	Fourier conventions	31
10.3	Transforming the derivative equation	31
10.4	Iteration to a finite product	32
10.5	The infinite sinc product	32
10.6	Fourier injectivity and uniqueness	33
10.7	Moment series from the Fourier product	33
10.8	Poisson summation	33
11	Finite convolutions, probability, and approximation	34
11.1	The probabilistic representation	34

11.2	Finite convolution measures	34
11.3	Weak convergence	34
11.4	Step approximants	35
11.5	The endpoint half-weight correction	35
11.6	Coefficient unimodality	35
11.7	From distribution functions to the bounded Fabius function	36
11.8	Exact finite Taylor formulas and nonanalyticity	36
11.9	Corrections recorded by the original-paper aggregator	36
12	The two paper formalizations as navigable APIs	37
12.1	The first paper: <code>Paper05442.lean</code>	37
12.2	The arithmetic paper: <code>Paper06487.lean</code>	37
12.3	Statements that remain questions or conjectures	38
12.4	Why a separate coverage document matters	38
13	Thue–Morse and q-binomial identities	39
13.1	Why q-binomial algebra appears	39
13.2	The finite q-Pochhammer infrastructure	39
13.3	Why prove a specialized q-binomial theorem?	40
13.4	Raw, scalar, and polished formulas	40
13.5	The unrestricted finite identity	40
13.6	Translation invariance as a polynomial theorem	41
13.7	Taylor expansion in the q parameter	41
13.8	The global binary-reduction series	41
13.9	Parity-power series	41
14	Discrete limits and complex shifts	43
14.1	The shape of the limit theorem	43
14.2	Uniform spline approximants	43
14.3	Complex-shift control	43
14.4	The Toeplitz averaging mechanism	44
14.5	Integration and identification of the limit	44
14.6	Proof flow of the discrete-limit family	44
15	The logarithmic scale near the flat endpoint	46

15.1	Why ordinary Taylor asymptotics fail	46
15.2	The exact logarithmic delay identity	46
15.3	Positivity near zero	47
15.4	The quadratic logarithmic law	47
15.5	A quantitative coarse bound	47
15.6	Filter formulations	47
16	Negative Laplace analysis and the periodic correction	49
16.1	The negative moment generating function	49
16.2	The kernel and its bounds	49
16.3	The exact dilation equation	50
16.4	How the periodic function is constructed	50
16.5	Continuity and regularity	50
16.6	Mellin–Bose analysis	51
16.7	Finite parts and Bromwich preparation	51
16.8	Why the periodic term cannot be dropped	51
16.9	Identification with the generating function	51
17	Bromwich inversion and the sharp saddle point	53
17.1	From a transform to the endpoint density	53
17.2	The Lambert phase	53
17.3	Exact reduction at the saddle	54
17.4	Central and tail decomposition	54
17.5	The Gaussian reference weight	54
17.6	Cumulants and derivative bounds	54
17.7	Minor arcs and tail decay	55
17.8	The first correction	55
17.9	The corrected sharp formula	55
17.10	Failure of an uncorrected elementary formula	55
18	All-order asymptotic expansions	57
18.1	Why the exact phase is retained	57
18.2	Formal logarithm of the Laplace product	57
18.3	Higher small-argument expansion	57
18.4	Gaussian polynomial contractions	58

18.5	Coefficient recurrences	58
18.6	Uniform central remainders	58
18.7	All-order tails	58
18.8	Assembling the normalized mass expansion	59
18.9	From mass to logarithm	59
18.10	Transfer to $F(x)$	59
18.11	First-order and Wikipedia-facing corollaries	59
18.12	The proof-audit role of the asymptotic aggregator	60
19	Legendre expansions and polynomial approximation	61
19.1	Why Legendre polynomials are a natural basis	61
19.2	Building the polynomial basis	61
19.3	Sturm–Liouville orthogonality	61
19.4	Norms and pointwise bounds	62
19.5	Vanishing of odd coefficients	62
19.6	Exact coefficient evaluation	62
19.7	Generic convergence infrastructure	63
19.8	Identifying the uniform limit	63
19.9	Translated series for the global Fabius function	64
19.10	Least-squares optimality	64
19.11	Gaunt coefficients and finite Rvachev Gram sums	64
19.12	Computational significance	65
20	The k-fold Thue–Morse draft audit	66
20.1	Purpose of the audit	66
20.2	Exact finite identities	66
20.3	Polygonal interpolation and corrected approximation	66
20.4	Formal counterexamples	67
20.5	Repairing the lower Lambert branch	67
20.6	Decay comparisons	67
21	Formal combinatorics and power-series infrastructure	68
21.1	Why a Fabius project owns so much combinatorial algebra	68
21.2	Triangular transforms as the organizing abstraction	68
21.3	Bell polynomials and cumulants	69

21.4	Endpoint-transfer polynomials and finite expectations	69
21.5	Formal logarithms, exponentials, and inversion	70
21.6	Symmetric functions and geometric alphabets	70
21.7	What to import	71
22	Weighted-uniform laws, transforms, and geometric filters	72
22.1	The generic random-series layer	72
22.2	Affine independent-copy laws and the continuous-CDF bridge	72
22.3	CDFs, quantiles, and Cauchy/Stieltjes transforms	73
22.4	Finite filters on arbitrary and geometric nodes	73
22.5	Gram matrices and orthogonal reconstruction	74
22.6	Activation Taylor jets, square-summability, and effective dimension	74
23	The expanded Thue–Morse formalization	76
23.1	From one sign sequence to a digital-analysis library	76
23.2	The all-rings product identity and Prouhet cancellation	76
23.3	Valuations and Pascal multiplicities	77
23.4	Automatic and algebraic structure	77
23.5	Fourier, aliasing, and cyclic characters	77
23.6	Centered sine and sinh products	77
23.7	Infinite products, Mellin transforms, and natural boundaries	78
23.8	Newman, Gelfond, Riesz, and Hankel directions	78
23.9	Bridges back to Fabius and q-series	78
24	Generalized Rvachev products, Fourier decay, and synthesis	79
24.1	Beyond the classical dyadic sinc product	79
24.2	Zeros, analytic order, and identifiability	79
24.3	Lobe geometry and local factorization	80
24.4	Fourier decay as a dynamical problem	80
24.5	Transfer operators and certified spectral bounds	80
24.6	Moment matrices, orthogonal polynomials, and exact synthesis	80
24.7	Quarter-cell and inverse-germ technology	81
25	Inverse theory, Volterra calculus, non-elementarity, and computability	82
25.1	The totalized inverse as an order-theoretic object	82

25.2	Endpoint steepness and the exact smoothness locus	82
25.3	Sharp interval-mass and inverse-gap rigidity	82
25.4	Moduli and effective inversion	83
25.5	Normalized Volterra operators	83
25.6	Fractional order	84
25.7	Layer-cake and inverse-pair integration	84
25.8	Analyticity and non-elementarity	84
26	The current q-series corpus	85
26.1	Why q-series are in the public Fabius facade	85
26.2	Finite q-binomial algebra	85
26.3	Infinite q-Pochhammer products	85
26.4	q-gamma and Jackson calculus	86
26.5	Basic hypergeometric transformations	86
26.6	Theta products and sums of squares	86
26.7	Partitions, Bailey chains, and Rogers–Ramanujan	86
26.8	Finite geometry and cyclic sieving	86
26.9	Return bridges to Fabius and Thue–Morse	87
27	Lambert coordinates and phase-locked extrapolation	88
27.1	Two real branches with explicit domains	88
27.2	A generic autonomous-ODE engine	88
27.3	All derivatives of both Lambert branches	88
27.4	Lambert coordinates in the Fabius saddle problem	89
27.5	Phase-locked Richardson and Bell extraction	89
27.6	Power-exponential inverse calculus	89
28	Proof status, documentation boundaries, and maintenance	90
28.1	Kernel-checked, executable, conditional, and frontier claims	90
28.2	Repository invariants	90
28.3	How to update this walkthrough again	91
28.4	What this walkthrough does not claim	91
29	Recurring proof-engineering patterns	92
29.1	Strong induction as the default arithmetic engine	92

29.2	Finite sums and index transport	92
29.3	Casts as explicit interfaces	93
29.4	Pointwise derivatives instead of raw <code>deriv</code>	93
29.5	Piecewise functions and boundary glue	93
29.6	Local finiteness before infinite-sum algebra	94
29.7	Integrability from support and bounds	94
29.8	Interchanging sums, limits, derivatives, and integrals	94
29.9	Big-O, little-o, and eventual domains	95
29.10	Polynomial identities	95
29.11	Choosing automation deliberately	95
29.12	Public wrappers and internal strength	96
29.13	No placeholders and explicit corrections	96
30	How to read and extend the code	97
30.1	Four recommended reading paths	97
30.1.1	Path A: existence and uniqueness	97
30.1.2	Path B: exact evaluation	97
30.1.3	Path C: the historical Rvachev theory	98
30.1.4	Path D: sharp asymptotics	98
30.2	Adding a new exact identity	98
30.3	Adding a new moment theorem	99
30.4	Adding an asymptotic correction	99
30.5	Testing a suspected source error	99
30.6	Maintaining dependency hygiene	100
30.7	Naming and theorem granularity	100
30.8	A minimal downstream example	100
30.9	What to trust when prose and code differ	101
31	Concluding perspective	102
A	Exact public-import census at the pinned commit	103
A.1	Reproduction command	114
B	Detailed dependency map	115
B.1	Core spine	115

B.2	Arithmetic spine	115
B.3	Original-paper spine	115
B.4	Asymptotic spine	116
C	Paper theorem map	117
C.1	Arias de Reyna, arXiv:1702.05442	117
C.2	Arias de Reyna, arXiv:1702.06487	117
C.3	How to use the map	118
D	Annotated classical-core module guide	119
E	Corrections and critical distinctions	126
E.1	What these corrections teach	128
F	Lean and mathematical glossary	129
G	Example exploration snippets	131
G.1	Inspect the public objects	131
G.2	Evaluate exact dyadic data	131
G.3	Work generically under the specification	132
G.4	Find source-facing arithmetic theorems	132
G.5	Inspect asymptotic interfaces	132
G.6	Search tactics	132
H	Snapshot and reproducibility audit	134
H.1	Pinned sources	134
H.2	Counts that are exact, historical, or intentionally omitted	134
H.3	Trust boundary	135
H.4	Document validation	135
I	Bibliographic and repository notes	136
	Document metadata	138

Building and entering the development

1.1 Workspace location and pinned toolchain

The Fabius library is registered by the repository-root `lakefile.toml` as

```
[[lean_lib]]
name = "FabiusFunction"
srcDir = "Analysis/FabiusFunction/Lean"
```

The root `lean-toolchain` selects `leanprover/lean4:v4.32.0`, and the root Lake manifest requests `mathlib v4.32.0`. The public facade is `Analysis/FabiusFunction/Lean/FabiusFunction.lean`; subject modules are under `Analysis/FabiusFunction/Lean/FabiusFunction/`.

A minimal exploration file can begin with the facade:

```
import FabiusFunction

open Fabius

#check IsFabius
#check fabius_spec
#check rvachevUp
#check extendedFabius
#check fabiusDyadicValue
#check fabiusInv
```

This is convenient for browsing. A new proof should import the narrowest leaf that owns the declarations it uses. Narrow imports expose accidental coupling, shorten rebuilds, and preserve the directed dependency architecture.

1.2 The current serialization rule

The present Lake version invalidates a prominent piece of older advice. On the repository's measured setup, `LAKE_JOBS=1` is ignored; Lake also rejects both `-j` and `--jobs`. Consequently, setting an environment variable does not turn a broad invocation into a one-worker build. The only reliable repository protocol is operational rather than flag-based:

1. run exactly one Lake command at a time;

2. give that command exactly one module target;
3. make sure the target's dependencies are already compiled; and
4. do not start a second driver or background loop while the first is alive.

A module target needs a leading plus sign:

```
lake exe cache get
lake build +FabiusFunction.Basic
lake build +FabiusFunction.Arithmetic
lake build +FabiusFunction.Existence
```

By contrast, `lakebuildFabiusFunction` names the broad library target. It answers the expensive corpus-wide question and may expose enough independent stale work for Lake to fan out. It is not the first validation command to run on a memory-constrained machine.

Formalization-sensitive point

Do not use `LEAN_NUM_THREADS=0` as a memory-control substitute. In this toolchain it serializes elaboration inside each Lean process and was measured by the project to slow a focused build by roughly a factor of thirty. If one Lean process itself is too large, the repository guidance is a small positive value, not zero. Errors such as a failed read of a Mathlib `.ole` an followed by `std::bad_alloc` can be memory-starvation symptoms rather than proof failures.

1.3 Scoped validation instead of a facade rebuild

For a change set, the mathematically relevant validation target is the reverse import closure of the edited modules. The repository provides scripts that compute that closure and drive one target at a time:

```
python3 Analysis/FabiusFunction/scripts/affected_modules.py \
  <base-commit> --stats --out affected.txt
sh Analysis/FabiusFunction/scripts/validate_affected.sh affected.txt
```

The first script closes the changed paths under dependents in the `importFabiusFunction.*` graph and topologically orders them. The second runs the serialized builds. This is stronger than compiling only the edited leaf and substantially cheaper than recompiling hundreds of unaffected files. Moving a declaration into a root module such as `Arithmetic.lean`, `Basic.lean`, or `Differential.lean` is correspondingly expensive, because almost the entire corpus can become stale.

1.4 A practical reading workflow

A useful editor workflow is:

1. start at a source-facing theorem in a paper or subject aggregator;
2. inspect the complete type before reading the proof;
3. follow the first substantial helper with “go to definition”;

4. decide whether the proof is algebraic, exact-arithmetic, analytic, probabilistic, or a transport between those layers;
5. stop descending when the mathematical engine becomes generic; and
6. return to the wrapper to see how domains, casts, and notation are discharged.

A short aggregator proof is often the final cast or specialization of a much stronger reusable theorem. Searching by mathematical transition—for example “moment to cumulant”, “measure to CDF”, “local product to analytic order”, or “exact dyadic value to real value”—is usually more effective than searching only by a paper’s theorem number.

Current snapshot and the anatomy of the facade

2.1 The exact census used by this walkthrough

At the pinned commit, `Lean/FabiusFunction.lean` has 1,021 lines. Lines 1–740 are 740 distinct direct imports; `set_option autoImplicit false` then appears, followed by a long documentation comment describing the public mathematical surface. Appendix A reproduces all 740 imports in their exact order.

The facade order is not a strict topological sort and should not be read as a pedagogical sequence. It behaves more like a release manifest: mature leaves, aggregators, bridges, and recently completed frontier modules are all imported explicitly. The following contiguous strata are an audit convenience, not Lean namespaces.

Range	Count	Dominant role
1–7	7	Cross-cutting Parseval, Pascal, periodic-orbit, half-integer-energy, and elementary Lambert bounds.
8–95	88	Symmetric functions, Bell/Stirling/Eulerian/Bernoulli algebra, basis changes, generating functions, and formal inversion.
96–127	32	Finite q-binomial algebra, geometric Lagrange/Richardson filters, residual moments, and Lambert-phase extraction.
128–144	17	Principal and lower Lambert branches, branch geometry, all derivatives, and power-exponential inverse calculus.
145–199	55	Weighted-uniform laws, the paper facades, polynomial functionals, Gram/Jacobi data, cumulants, Appell systems, activation functions, and implicit series.
200–307	108	Thue–Morse and digital algebra: Prouhet identities, valuations, automata, Fourier/Mahler/Mellin products, natural boundaries, Hankel theory, Newman and Gelfond estimates.
308–390	83	Generalized Rvachev products, spline plateaux, Stieltjes and Cauchy transforms, geometric conditioning, moment matrices, orthogonal polynomials, comb exactness, and synthesis.
391–488	98	Lacunary Fourier decay, cocycles, transfer operators, canonical products, zero divisors, lobe geometry, analytic orders, spectral bounds, and the centered sinh-product bridge.

Range	Count	Dominant role
489–559	71	Measure refinement, negative Laplace analysis, Legendre energy, global shape, inverse theory, Volterra and fractional calculus, non-elementarity, and computability.
560–595	36	Closed saddle coefficients, Euler-log and sinc-zeta calculus, certified Perron bounds, measure laws, weak spline convergence, and normalization bridges.
596–740	145	Infinite q-Pochhammer theory, q-gamma and basic hypergeometric functions, theta identities, partitions, Bailey chains, Rogers–Ramanujan and Andrews–Gordon theory, plus final Fabius/Thue–Morse q-bridges.

2.2 What the current public comment says is proved

The documentation comment attached to the facade is unusually substantive. It records the following proved interfaces, with important boundaries.

2.2.1 Sharp global shape and the inverse

The bounded function has the global derivative formula

$$F'(x) = 2 \operatorname{up}(2x - 1),$$

the optimal Lipschitz constant two, strict monotonicity on $[0, 1]$, a bijection of the unit interval, convexity on the left half and concavity on the right, exact support and strict unimodality for up, effective and sharp flatness, sharp uniform derivative bounds, and the exact real-analytic locus: F is analytic precisely outside $[0, 1]$. The order isomorphism is inverted by a totalized `fabiusInv`; this inverse is continuous and monotone on all reals, smooth away from the two clamping values, and steeper than every algebraic root at the endpoints. The interior derivative of the inverse tends to $+\infty$ at both endpoints.

2.2.2 Derivative distributions

The normalized iterated derivatives of the up-function have exact dyadic cell profiles controlled by Thue–Morse signs. The API includes Banach-valued test functions, equality of Borel pushforwards on $[-1, 1]$, signed natural-power moments, and exact absolute p -moments for all nonnegative real powers. The facade comment is equally explicit about what is not packaged: no general L^p -norm ladder, inverse-level-set theorem, or spectral theorem is inferred from those measure identities.

2.2.3 Small-argument asymptotics

The periodic saddle jets and saddle exponent coefficients, formerly represented recursively, now have closed finite forms. Shifted signed Stirling-number coefficients solve the jet recurrence, and the coefficient of λ^{-2} is explicitly computed and inserted. Thus the public surface includes a three-term logarithmic expansion

$$\log F(x) = M(x) + \frac{A_1(\lambda)}{\lambda} + \frac{A_2(\lambda)}{\lambda^2} + O(\lambda^{-3}),$$

where M is the exact Lambert-phase main term and the corrections retain the periodic data.

2.2.4 Computable analysis

The canonical bounded function is a computable real function through a primitive-recursive centered-spline evaluator and an explicit modulus. The signed centered splines converge uniformly on all reals, and the finite binary reduction has a certified uniform error. A generic tolerant-bisection engine then turns dyadic approximation and an effective inverse modulus into a computable inverse without deciding exact equality. The specialization `fabiusInv_isComputableRealFunction` gives the totalized Fabius inverse a full computable-real interface. This is a correctness certificate, not a bit-complexity theorem.

2.2.5 Generalized products, transfer operators, and q-series

The generalized Rvachev product is identifiable from its dyadic analytic-order divisor; natural-linear identities of exponent sequences become exact product identities. The Fourier-decay program includes shell envelopes, lacunary means, a positive transfer operator, Collatz–Wielandt bounds, and the certified interval

$$0.66126798 \leq \rho_1 \leq 0.66134921$$

for the relevant Perron root. The q-series surface includes finite and infinite q-Pochhammer products, Gaussian coefficients, q-gamma, Jackson integration, theta products, basic hypergeometric transformations, cyclotomic factorization, q-Catalan objects, partitions, Bailey chains, Rogers–Ramanujan, and Andrews–Gordon identities. The analytic hypotheses—strict contraction, nonvanishing denominators, positivity, or domain restrictions—remain visible rather than being hidden in notation.

2.2.6 Measure-level closure of the probability model

The up-measure satisfies a convolution/refinement equation and its iterates. It is identified with the affine image of the infinite weighted-uniform random series, and finite centered spline laws converge weakly to that law. The characteristic-function bridge and the uniform CDF error connect the probability, Fourier, approximation, and computability viewpoints without asserting an unproved density formula at a generic parameter.

2.3 Why the old walkthrough became incomplete

The earlier guide was organized around a classical spine of roughly one hundred named modules. That spine remains valuable, but the current facade has added several largely reusable sub-libraries around it. In particular:

- Bell, Stirling, Eulerian, Bernoulli, Riordan, Appell, and symmetric-function modules now support formal series and saddle coefficients at a general algebraic level;
- the Thue–Morse development now extends from the dyadic evaluator to digital products, valuations, automata, Mahler equations, Mellin transforms, natural-boundary results, Hankel determinants, and sharp exponential-sum estimates;
- generalized Rvachev products now carry zero multiplicities, entire-function structure, lobe geometry, identifiability, and transfer-operator decay machinery;

- inverse theory, normalized and fractional Volterra calculus, non-elementarity, and computable analysis are public modules rather than prospective topics;
- the q-series layer is a substantial formal corpus in its own right; and
- on 3 September, the Lambert layer gained a generic autonomous-ODE engine and formulas for all derivatives of both real branches.

An updated walkthrough must therefore distinguish the *Fabius core* from the *Fabius-generated infrastructure*. Both are in the same public library, but not every q-series theorem depends essentially on the Fabius function.

Architectural point

The public facade is a union, not a single chain. The correct mental model has a central Fabius–Rvachev trunk with several reusable engines branching out: exact combinatorial algebra, probability laws, digital/Thue–Morse analysis, Fourier products, orthogonal polynomials, inverse and Volterra calculus, Lambert coordinates, and q-series. Bridges return from those engines to the core through named specialization modules.

The mathematical objects and their Lean interfaces

3.1 The bounded function

The foundational module `Basic.lean` introduces the closed unit interval as a subtype and uses it as the codomain of a bounded map. Schematically, the first layer has the shape

```
abbrev UnitInterval := Set.Icc (0 : Real) 1
abbrev BoundedFabiun := Real -> UnitInterval
```

The subtype codomain records the range bound in the type itself. The projection to $\mathbb{R}$, which the code exposes through a convenience definition such as `fabiunReal`, is used for calculus and integration. This division of labor avoids reproving $0 \leq F(x) \leq 1$ every time a measure-theoretic bound is required, while still giving the differentiable real-valued function expected by `mathlib`'s analysis APIs.

The abstract predicate `IsFabiun` packages the characteristic properties of a candidate. Its fields express, in essence:

$$\begin{array}{ll} F(x) = 0 & (x \leq 0), \\ F(x) = 1 & (1 \leq x), \\ F(x) + F(1 - x) = 1 & (0 \leq x \leq 1), \\ F'(x) = 2F(2x) & (0 \leq x \leq \frac{1}{2}), \end{array}$$

together with global smoothness. The precise formal structure uses pointwise derivative predicates and interval membership. Later lemmas extend the symmetry law from the core interval to a statement valid for all real x , with tail cases handled separately.

3.1.1 Why specify before constructing?

Most analytic theorems are stated for a variable F with hypothesis `hF : IsFabiunF`. This is an important architectural choice. It allows derivative identities, Taylor formulas, moment recurrences, and dyadic uniqueness to be proved without unfolding the fixed-point construction. The existence module can then instantiate this generic API, while uniqueness is proved once at the level of the specification.

This organization resembles an interface-and-implementation pattern:

- `Basic.lean` supplies the interface;
- `Differential.lean`, `DyadicAnalytic.lean`, and related modules prove generic consequences of the interface;
- `Existence.lean` supplies the implementation; and
- the canonical term and its theorem `fabius_spec` let later files forget the implementation again.

3.2 The Rvachev up-function

The compactly supported function `up` is the symmetric “twin” of the bounded cumulative form. The code defines `rvachevUp` by folding the bounded function around the center of the unit interval. Up to the chosen normalization and translation, the relation is the familiar one

$$\text{up}(x) = F(x + 1) \quad (-1 \leq x \leq 0), \quad \text{up}(x) = 1 - F(x) \quad (0 \leq x \leq 1),$$

with zero outside the support. The exact Lean definition is arranged to make symmetry and support proofs tractable and to match the Fourier normalization used in the original paper.

The up-function is the more natural object for Fourier analysis because it is even, smooth, compactly supported, and normalized to have integral one. The bounded form is the more natural object for dyadic arithmetic because its derivative equation and boundary values generate finite Taylor recurrences.

3.3 The signed global extension

The bounded function cannot satisfy an unrestricted translation law: it is constant on both tails. The repository therefore defines `extendedFabius`, a signed global extension assembled as a locally finite sum of translated copies of `up`, weighted by the Thue–Morse sign. In informal notation it has the form

$$\tilde{F}(x) = \sum_{k \in \mathbb{Z}} \varepsilon(k) \text{up}(x - k),$$

where only finitely many summands are nonzero at any given x . Lean nevertheless treats the expression through a `tsum`, so the library must prove local finiteness, summability, and the reduction of the infinite sum to a finite set before elementary rearrangements are legal.

On the unit interval the extension agrees with the bounded function. Away from that interval it carries the signs and period-two behavior needed for global dyadic formulas. This distinction is used pervasively in `GlobalExtension.lean`, `GlobalDyadic.lean`, and the `q-binomial` modules.

Formalization-sensitive point

When an informal statement writes $F(x)$ for all real x , inspect the theorem’s target. It may refer to the bounded map, the real projection of that map, the compactly supported up-function, or the signed extension. Several corrected statements in the repository amount precisely to choosing the right one.

3.4 Auxiliary analytic objects

`Basic.lean` also reserves names for the analytic transforms that later modules identify:

- a complex sinc function with a removable value at zero;
- the Fourier transform and infinite sinc product associated with up ;
- moment and half-moment generating series;
- real and complex versions of $(\exp z - 1)/z$, again with the singularity removed; and
- integral definitions of moments over the support.

Defining these objects early stabilizes names and normalizations. Proofs of convergence, equality, and differentiability are deferred to modules with the necessary imports. This keeps `Basic.lean` conceptually small even though it determines the vocabulary of the whole project.

The dependency architecture

4.1 A layered view

The following diagram suppresses many helper modules but captures the principal flow. Arrows point from prerequisites to consumers.

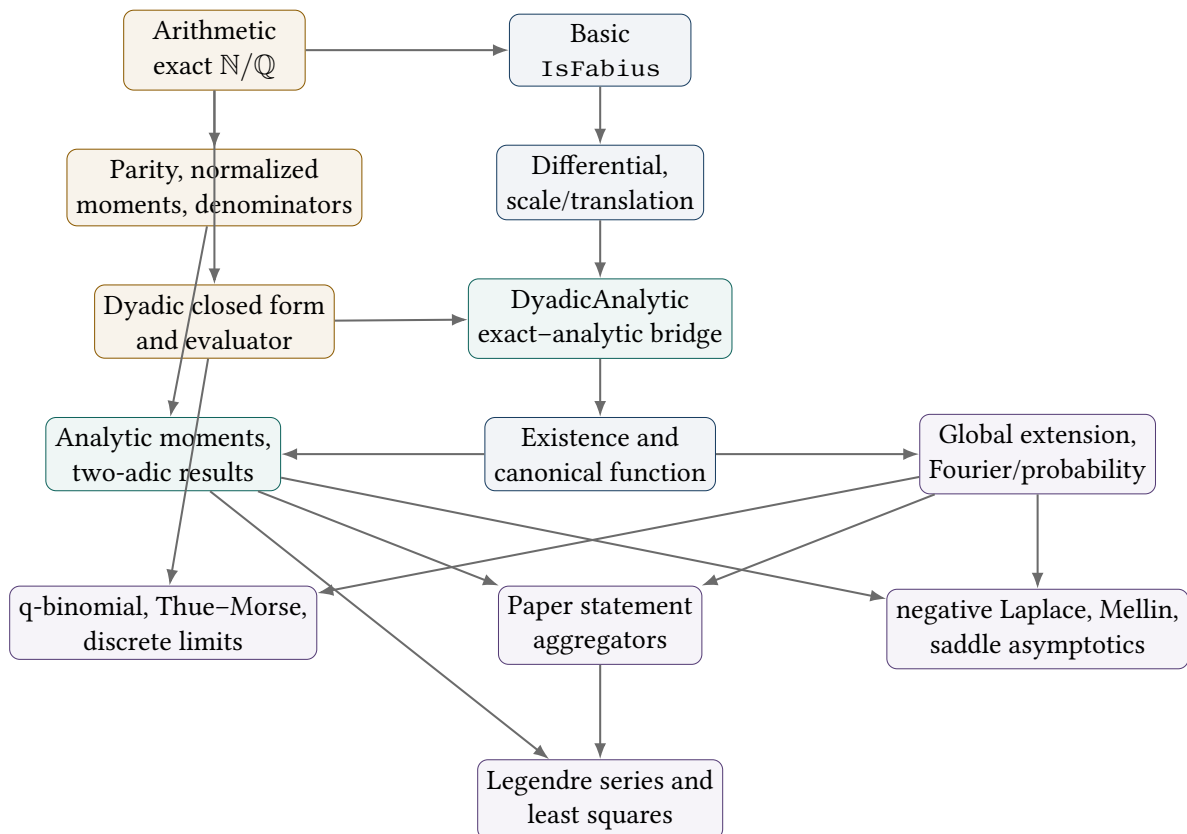


Figure 4.1: Conceptual dependency graph. The source contains finer-grained helper modules between nearly every displayed pair.

4.2 Why existence is not the first analytic file

A newcomer might expect `Existence.lean` to precede every theorem about the function. Instead, the code first proves a great deal about an arbitrary `IsFabius` object. This is not circular. The direction is:

1. assume a candidate satisfies the abstract laws;
2. derive its dyadic values and prove that any two candidates agree on a dense set;
3. construct one candidate by a contraction mapping; and
4. invoke the already-proved generic uniqueness theorem.

This order isolates the hard analytic construction from the algebraic uniqueness mechanism. It also means that the dyadic evaluator is not merely a computational afterthought: it participates in the proof that the canonical object is unique.

4.3 Two independent uniqueness arguments

The repository contains two conceptually different uniqueness proofs.

The first is adapted to the bounded specification. The derivative and symmetry equations determine exact values at every dyadic rational. Dyadics are dense, and the candidates are continuous, so agreement on dyadics implies agreement everywhere.

The second is adapted to Rvachev’s original compactly supported characterization. The derivative/refinement law is transformed into a Fourier-domain sinc equation. Iteration forces the entire Fourier transform to equal an infinite sinc product. Fourier injectivity on an appropriate smooth rapidly decreasing class then forces the original functions to coincide.

Keeping both arguments is valuable. The dyadic proof is computational and feeds exact arithmetic; the Fourier proof mirrors the historical characterization and explains the infinite product.

4.4 Paper aggregators versus theorem-producing modules

Files named `Paper05442.lean`, `Paper06487.lean`, and `PaperFabiusAsymptotic.lean` are indexes and audit records. They import theorem-producing modules and expose a source-facing collection of results. For example, `PaperStatements.lean` gives declarations whose numbering and prose correspond to the arithmetic paper, but the actual proof of a denominator theorem may be distributed across `NormalizedMoments.lean`, `Parity.lean`, `TwoAdic.lean`, and `DenominatorBound.lean`.

This separation has three advantages:

- source numbering does not dictate internal dependency boundaries;
- corrected hypotheses can be documented at the aggregator without contaminating low-level lemmas; and
- later developments can reuse strong internal theorems rather than wrappers tailored to a paper’s notation.

The exact-arithmetic foundation

5.1 Why the development begins over $\mathbb{N}$ and $\mathbb{Q}$

The module `Arithmetic.lean` is deliberately independent of the canonical analytic function. It defines the integer and rational sequences that later turn out to be moments, dyadic values, normalizing factors, and denominator bounds. This has a profound practical advantage: exact identities can be proved using induction, finite sums, divisibility, and polynomial normalization without carrying measurability, differentiability, or convergence hypotheses.

The exact side is not just a transcription of closed formulas. It supplies executable definitions. One can evaluate a dyadic rational by normalization and recursion, and Lean’s kernel can reduce the resulting rational expression. The same definitions then serve as specifications for correctness theorems.

5.2 Binary weight and the Thue–Morse sign

The smallest combinatorial ingredients are the binary digit sum and its sign:

$$s_2(n) = \text{number of ones in the binary expansion of } n, \quad \varepsilon(n) = (-1)^{s_2(n)}.$$

In the code these appear under names such as `binaryWeight` and `thueMorseSign`. Elementary identities under doubling and successor are proved early because they drive both the dyadic evaluator and the q-binomial development:

$$s_2(2n) = s_2(n), \quad s_2(2n + 1) = s_2(n) + 1,$$

so that $\varepsilon(2n) = \varepsilon(n)$ and $\varepsilon(2n + 1) = -\varepsilon(n)$.

Lean’s natural-number division and remainder operations require explicit control of positivity. Proofs that are a one-line observation on binary expansions often become a short cluster of lemmas about `Nat.div`, `Nat.mod`, powers of two, and bit constructors. Establishing this API once prevents the higher-level files from repeatedly descending to bit arithmetic.

5.3 Product normalizations

Several finite products recur in moment denominators. The arithmetic module names them rather than expanding them at every use. Principal examples include variants of

$$\prod_{j=1}^n (2^j - 1), \quad \prod_{j=1}^n (2^{2j} - 1), \quad (2n - 1)!!.$$

The source uses names such as `mersenneProduct`, `evenMersenneProduct`, and `oddDoubleFactorial`. Their basic positivity, nonvanishing, splitting, and divisibility properties are the denominator-management toolkit for later proofs.

A formal proof benefits from keeping two versions of a formula:

- a mathematically transparent quotient in $\mathbb{Q}$; and
- a division-free equality in $\mathbb{N}$, obtained after multiplying by a common product.

The quotient is convenient for stating a moment recurrence. The division-free equality is the robust object for induction, parity, and divisibility, because it avoids repeated side goals asserting that denominators are nonzero.

5.4 Moment and half-moment recurrences

The exact sequences `moment` and `halfMoment` are defined recursively as rational numbers. They model, respectively, moments of the centered up-function and half-interval moments of the bounded function. The definitions mirror the recurrences obtained analytically by integration by parts and scaling.

Alongside them, `momentNumerator` and `halfMomentNumerator` are natural-number recurrences with the common denominators cleared. Their role is easy to underestimate. They provide:

1. a finite, executable representation of exact values;
2. a target for strong-induction proofs of closed forms;
3. a clean setting for parity arguments; and
4. a bridge to reduced rational numerators and denominators.

A recurring proof pattern is to start from the rational recurrence, multiply by the anticipated global normalizing product, distribute that product through a finite sum, and use factorial or Mersenne product splitting to identify every term with an earlier natural numerator. Once the normalizations are designed correctly, the final goal is a polynomial identity in natural numbers and powers of two.

Lean pattern

For exact rational identities, the development usually proves all denominator factors positive or nonzero near the beginning of the proof, rewrites divisions into a common denominator, and invokes `field_simp` only after the relevant nonzero facts are available. The remaining equality is discharged by a combination of finite-sum rewrites, `ring_nf`, and induction hypotheses. Calling `field_simp` too early tends to produce enormous and poorly shaped goals.

5.5 Closed dyadic formulas

The exact values at inverse powers of two are represented by a sequence such as `halfMomentFabiussValue`. A more general closed expression, exposed through a name such as `fabiussDyadic`, gives the value at $a/2^n$ as a finite sum involving the inverse-power table and powers determined by the local Taylor formula.

The formalization maintains a distinction between:

- a closed formula suited to mathematical theorems;
- a Horner form suited to evaluation;
- a unit-interval evaluator that clamps or reflects inputs; and
- a global evaluator that incorporates Thue–Morse signs and period-two reduction.

This separation makes correctness modular. One first proves that the Horner program computes the finite polynomial, then that recursive bit reduction computes the closed formula, then that the closed formula equals the analytic function.

5.6 The inverse-power table

The function `fabiussInversePowTwoTable` computes a prefix of exact values

$$F(2^{-1}), F(2^{-2}), \dots, F(2^{-n}).$$

A table-based implementation avoids recomputing the same recurrence at every recursive step. Correctness is expressed through prefix stability: extending a table preserves all previously computed entries. This is essential because the evaluator may request values in an order dictated by the bits of the numerator rather than by a single monotone loop.

Formal prefix stability is typically a theorem about list lookup or finite-indexed entries. It combines:

- length calculations;
- bounds for `Fin` indices or `List.get?`;
- compatibility of a recurrence with appending a new term; and
- extensional equality of the old prefix.

Such lemmas look implementation-specific, but they prevent computational code from leaking into the mathematical correctness theorem.

5.7 Horner evaluation

The local Taylor polynomial is evaluated by `fabiussTaylorHorner`. Horner’s rule is not only computationally efficient; it is proof-friendly. The correctness theorem is an induction over the coefficient list, and the induction invariant is a suffix polynomial. In contrast, a direct power-sum implementation forces repeated reasoning about exponents, list indices, and truncated ranges.

The coefficients are rational and involve powers of two coming from iterated differentiation. The proof therefore includes cast-normalization lemmas ensuring that a natural exponent or binomial coefficient

has the same interpretation in $\mathbb{N}$, $\mathbb{Q}$, and $\mathbb{R}$. The code frequently uses explicit type annotations at precisely these interfaces; their apparent verbosity documents where coercion inference would otherwise be ambiguous.

5.8 Recursive removal of the highest bit

At the center of the evaluator is a structurally terminating function such as `fabiusDyadicUnitAux`. Given a dyadic numerator a and exponent n , it removes the highest set bit of a . Write

$$a = 2^b + r, \quad 0 \leq r < 2^b.$$

A Taylor identity expresses the value at $a/2^n$ using the value and inverse-power data associated with the smaller remainder r . Since $r < a$ whenever $a > 0$, the recursive call decreases in a well-founded order.

This is the computational counterpart of traversing the ones in the binary expansion of a . The number of recursive Taylor reductions is therefore controlled by $s_2(a)$, not by the magnitude of a itself. Together with precomputation of the inverse-power table, the design yields the approximate complexity described in the repository documentation: a quadratic table phase in the exponent and a bit-weight-dependent evaluation phase.

Lean does not accept “the highest bit was removed” as a termination argument. The source proves a chain of inequalities connecting the bit logarithm, the largest power of two not exceeding a , and the remainder. This chain is packaged so the recursive definition can use a measure. The correctness proof later reuses the very same decomposition, an example of an implementation lemma doing double duty as a mathematical induction lemma.

5.9 Clamping, reflection, and the global evaluator

The unit-interval wrapper `fabiusDyadicUnit` normalizes a dyadic representative and applies the endpoint and symmetry laws. The rational-valued `fabiusDyadicValue` accepts an integer numerator and power-of-two denominator, managing negative values, inputs beyond one, and equivalent unreduced representations.

For the signed extension, `extendedFabiusDyadicValue` first reduces the integer part modulo the relevant period and then applies a Thue–Morse sign. The exact theorem is stronger than a formula restricted to reduced representatives: multiplying numerator and denominator by a common power of two leaves the output unchanged. This refinement invariance is proved explicitly rather than delegated to quotient types, because the evaluator’s input is computational data, not an abstract element of a dyadic localization.

Proof idea

There are three distinct notions of normalization:

1. arithmetic normalization of a fraction by canceling powers of two;
2. geometric normalization into $[0, 1]$ using tails and reflection; and
3. global normalization modulo translation, with a Thue–Morse sign.

The code proves each normalization preserves the intended mathematical value before composing them.

5.10 Recognizing dyadic rationals

The definitions `IsDyadicRational` and `dyadicExponent?` support a user-facing exact evaluator for rational inputs. A rational is dyadic when its reduced denominator is a power of two. The decision procedure must connect:

- the internal normalized numerator and positive denominator of $\mathbb{Q}$;
- a Boolean or optional power-of-two test; and
- a proposition asserting the existence of an integer numerator and natural exponent.

The proof that recognition is sound is separate from the proof that the returned exponent and numerator evaluate correctly. This makes failure meaningful: a non-dyadic rational is rejected because no representation with a power-of-two denominator exists, not merely because a search limit was reached.

5.11 Arithmetic API design lessons

The exact core illustrates several design principles that recur throughout the repository.

First, executable functions and theorem-friendly formulas need not be identical definitions. They are connected by named correctness lemmas. Second, normalization data should be made explicit, especially when a mathematical quotient hides representation choices. Third, the strongest useful invariant is often not the final equality but a stability statement about prefixes, refinements, or common denominators. Finally, natural-number recurrences are frequently the right interface for arithmetic theorems even when the motivating quantities are rational.

Constructing the canonical bounded function

6.1 The ambient complete metric space

The construction in `Existence.lean` uses Banach’s fixed-point theorem. Let

$$I = [0, 1], \quad C = C(I, \mathbb{R})$$

with the uniform metric. Since the codomain is complete and I is compact, the continuous-map space is complete. The code then cuts out a closed subset of C consisting of functions satisfying two elementary conditions:

$$\begin{aligned} 0 &\leq f(x) \leq 1, \\ f(x) + f(1 - x) &= 1. \end{aligned}$$

The source calls the reflection operator and the admissibility predicate by local names and proves that the admissible set is closed. A linear function, essentially $x \mapsto x$, witnesses nonemptiness.

Using a closed subspace rather than the entire continuous-function space is decisive. Symmetry is not recovered after the fixed point; it is an invariant of the contraction. This makes the integral normalization automatic and improves the contraction constant from the naive value one to one half.

6.2 Extending an interval function

An element of $C(I, \mathbb{R})$ accepts a subtype argument. To integrate it over an ordinary real interval, the code defines an extension by projecting an arbitrary real number to I . This projection clamps values below zero to zero and values above one to one. Continuity follows from continuity of the metric projection to a closed convex interval.

For an admissible f , define the cumulative integral

$$A_f(y) = \int_0^y \widehat{f}(t) \, dt,$$

where $\widehat{f}$ is the clamped extension. Symmetry implies

$$\int_0^1 f(t) \, dt = \frac{1}{2}.$$

The Lean proof reflects the integral by the substitution $t \mapsto 1 - t$, adds the two copies, and uses the pointwise symmetry equation. The subtype coercions and interval maps are handled in helper lemmas so the final normalization proof resembles the paper calculation.

6.3 The nonlinear-looking transform that is actually affine

The fixed-point transform is defined piecewise at the midpoint:

$$(Tf)(x) = \begin{cases} A_f(2x), & 0 \leq x \leq \frac{1}{2}, \\ 1 - A_f(2 - 2x), & \frac{1}{2} \leq x \leq 1. \end{cases}$$

The two branches agree at $x = 1/2$ because $A_f(1) = 1/2$. Each branch is continuous, and the gluing lemma supplies continuity of Tf on the whole interval. Bounds follow because f is bounded between zero and one; symmetry follows by exchanging the two branches.

The transform is affine on the admissible set. The constants in the right branch cancel when two transformed functions are subtracted. This is why the contraction estimate can be proved through integrals of $f - g$.

6.4 The contraction estimate

Let $h = f - g$ for two admissible functions. Symmetry gives

$$\int_0^1 h(t) \, dt = 0.$$

For $0 \leq y \leq 1$, the prefix integral may be estimated in two ways:

$$\left| \int_0^y h \right| \leq y \|h\|_\infty, \quad \left| \int_0^y h \right| = \left| \int_y^1 h \right| \leq (1 - y) \|h\|_\infty.$$

Therefore

$$\left| \int_0^y h \right| \leq \min(y, 1 - y) \|h\|_\infty \leq \frac{1}{2} \|h\|_\infty.$$

This is the key insight of the construction. A crude estimate would give constant one and no contraction. The zero-total-integral identity lets the proof select the shorter of the prefix and complementary suffix.

Lean pattern

The formal estimate is assembled from interval-integral norm bounds rather than from an informal “length times supremum” principle. The code proves integrability of the extended difference, obtains a pointwise uniform bound from the distance in the continuous-map space, applies the interval integral inequality on both subintervals, and rewrites the total integral as zero. A final case split on $y \leq 1/2$ selects the useful bound.

Banach’s theorem now yields a unique fixed point in the admissible subspace. The source extracts its underlying continuous map and extends it to a bounded real function with constant tails.

6.5 From an integral fixed point to the differential law

A fixed point satisfies an integral equation. On the left half,

$$F(x) = \int_0^{2x} F(t) \, dt.$$

The fundamental theorem of calculus and the chain rule give

$$F'(x) = 2F(2x).$$

On the right half, differentiation of the reflected branch yields the corresponding equation through symmetry. The proof handles the midpoint and endpoints separately because standard derivative rules apply most directly on open neighborhoods, while the desired structure requests derivatives at boundary points as well.

The source uses `HasDerivAt` statements whenever possible. Such statements compose cleanly through affine maps and integral primitives; a theorem about `deriv` alone would require differentiability side conditions each time it is rewritten. Endpoint derivatives are obtained by showing that the formulas from adjacent pieces agree, using the tail values and symmetry.

6.6 Bootstrapping smoothness

The fixed-point theorem initially gives only continuity. The integral equation gives one derivative. The derivative is a scaled composition of the original continuous function, so it is continuous. Iterating this observation upgrades the function to every finite differentiability order.

The formal proof is organized around reusable scale-and-derivative lemmas rather than a single enormous induction on `ContDiff`. At stage $n+1$, the derivative formula expresses the new derivative in terms of a scaled version of the n -th derivative. Boundary compatibility follows from vanishing on the tails and symmetry. The final theorem packages all finite stages as C^∞ .

6.7 Uniqueness by dyadic density

The contraction theorem already gives uniqueness inside the admissible continuous space. The exported specification, however, is phrased for real functions satisfying `IsFabius`; it is useful to prove uniqueness at this abstract level. The later dyadic analysis shows that every such function has the same exact value at every dyadic point. Since dyadics are dense in $\mathbb{R}$ and both candidates are continuous, the functions coincide.

The proof has the standard topological shape: approximate an arbitrary x by a sequence of dyadics, rewrite equality at each sequence term, and pass to the limit by continuity. The nontrivial work lies earlier, in proving exact dyadic equality without assuming the fixed-point implementation.

Architectural point

The fixed-point construction proves that the interface is inhabited. The dyadic theory proves that the interface is subsingleton. This yields a canonical object whose public properties are independent of the construction details.

Differential identities and the signed global extension

7.1 The role of `Differential.lean`

`Differential.lean` is the calculus engine for an arbitrary `ISFabi` candidate. Starting from the first derivative law on the left half and the globalized symmetry law, it derives corresponding formulas on other intervals and iterates the derivative relation.

The characteristic coefficient after n differentiations is a triangular power of two. In a typical region where all scaled arguments remain in the appropriate branch, one obtains

$$F^{(n)}(x) = 2^{1+2+\dots+n} F(2^n x) = 2^{n(n+1)/2} F(2^n x).$$

The exact theorem includes domain conditions ensuring that each intermediate scaled point lies in the branch where the derivative law is valid. Reflection introduces signs and translated arguments on the other half.

These iterated formulas explain three later phenomena at once:

- finite Taylor formulas at dyadic points;
- flatness at the support boundary; and
- extremely rapid decay near zero.

7.2 Domain bookkeeping

An informal derivation often differentiates the relation $F'(x) = 2F(2x)$ without restating its domain. Lean forces the domain to be propagated. If a theorem is iterated n times, the proof must establish that $2^j x$ belongs to the relevant interval for every $j < n$. The source packages this as inequalities involving powers of two and dyadic cells.

This bookkeeping is not bureaucratic. One of the asymptotic draft corrections later in the repository arises because a transformed delay-differential identity is valid only beyond a threshold. The formal derivative theorem makes the threshold impossible to omit.

7.3 Scale and translation lemmas

`ScaleTranslation.lean` collects chain-rule identities for functions of the form

$$x \mapsto c F(ax + b), \quad x \mapsto c \operatorname{up}(ax + b).$$

These are used by approximation arguments, global extension formulas, Fourier transforms, and splines. The value of a dedicated module is that every later proof can invoke a theorem with all factors of a , signs, and differentiability hypotheses already correct.

A common Lean mistake is to rewrite an iterated derivative of a composition as though the affine map contributed only one factor of a . The library's lemmas state the full a^n factor. Specialized powers-of-two corollaries then combine this with the triangular coefficient from the Fabius differential law.

7.4 Local finiteness of the translate sum

`GlobalExtension.lean` proves that the signed translate series defining $\tilde{F}$ is harmless. Because up has compact support, only finitely many integers k can satisfy $x - k \in [-1, 1]$ for fixed x . More strongly, on a compact neighborhood of x , a single finite set of translates contains every nonzero summand.

That stronger local statement is what permits termwise continuity and differentiation. The proof turns support inequalities into integer bounds, places all possible indices in a finite interval of $\mathbb{Z}$, and proves every index outside that set contributes zero. The `tsum` can then be rewritten as a finite sum using a summability-by-finite-support lemma.

7.5 Agreement and global identities

Once local finiteness is available, the signed extension is shown to agree with the bounded function on $[0, 1]$. The support of up reduces the sum to one or two adjacent translates, and the defining fold relation simplifies the result.

Translation by an integer changes the Thue–Morse coefficient in a controlled way. Translation by two gives a period-two relation; translation by one introduces a sign or complementary term depending on the normalization. These identities are the setting in which unrestricted versions of the dyadic recurrence are true. `GlobalDyadic.lean` combines them with exact values to remove assumptions that a numerator be reduced or lie between zero and its denominator.

7.6 Why use a `tsum` for a locally finite expression?

A hand proof would define the sum over the two possible integers adjacent to x . The repository instead uses a conceptually uniform sum over all integers. This pays off in translation arguments: shifting x corresponds to reindexing the same sum, and standard infinite-sum equivalences can be reused. The initial cost is a local-finiteness API; the later algebra becomes much cleaner.

The exact–analytic dyadic bridge

8.1 The purpose of `DyadicAnalytic.lean`

The exact evaluator is useful only after it is tied to the analytic function. `DyadicAnalytic.lean` is the central bridge. It proves, for every `IsFabius` candidate, the same finite recurrences that were defined independently in `Arithmetic.lean`; then it identifies the two by induction.

The proof has four main stages:

1. derive a finite Taylor identity on a dyadic cell;
2. specialize it to inverse powers of two and obtain an endpoint recurrence;
3. prove that the analytic inverse-power sequence equals the exact rational table; and
4. lift this equality to arbitrary dyadic numerators through the highest-bit recursion.

The surrounding modules `DyadicClosedForm.lean` and `DyadicCorrectness.lean` isolate the algebraic and algorithmic portions of this chain.

8.2 Finite Taylor expansion on a dyadic scale

Fix a sufficiently small dyadic base point. The iterated derivative formula expresses every derivative up to a chosen order through a scaled value of F . At the order where the scaled argument leaves the active interval and enters the zero tail, higher derivatives vanish. Taylor’s theorem therefore terminates exactly rather than producing an infinite series with a remainder.

A representative identity has the shape

$$F\left(\frac{a+r}{2^n}\right) = \sum_{k=0}^K \frac{2^{k(k+1)/2}}{k!} F\left(\frac{a}{2^{n-k}}\right) \left(\frac{r}{2^n}\right)^k,$$

with the base point, displacement, and upper index chosen to match a dyadic cell. The source uses a carefully normalized variant so the coefficients coincide with the exact Horner evaluator.

The proof does not invoke analyticity. It uses a finite-order Taylor theorem with a remainder, then proves the next derivative vanishes on the relevant segment. This is crucial: the Fabius function is smooth but nowhere analytic in the appropriate sense, so an argument based on convergence of an infinite Taylor series would be false.

Formalization-sensitive point

Finite Taylor identities are local consequences of compact support and the differential equation. They do not imply that the function equals its infinite Taylor series in a neighborhood. The formalization later turns the same exact truncation phenomenon into a proof of nonanalyticity.

8.3 Inverse-power values

For a candidate F , define informally

$$A_n(F) = F(2^{-n}).$$

The dyadic Taylor identity at a suitable endpoint gives a recurrence for $A_{n+1}(F)$ in terms of earlier $A_j(F)$. Boundary compatibility and symmetry supply the initial value $A_1(F) = 1/2$.

Independently, `Arithmetic.lean` defines a rational sequence satisfying the same recurrence. Strong induction proves

$$A_n(F) = A_n^{\text{exact}}$$

for every n . The induction step is predominantly algebraic, but the statement lives in $\mathbb{R}$; the proof must cast exact rational sums and products into reals. The source localizes these casts in lemmas asserting that the rational recurrence is preserved by the canonical embedding $\mathbb{Q} \hookrightarrow \mathbb{R}$.

8.4 From closed polynomial to Horner program

Once the analytic value is a finite polynomial in exact inverse-power values, `DyadicAnalytic.lean` invokes the algebraic theorem that the closed polynomial equals `fabiusTaylorHorner`. This is a clean interface point:

- analysis proves the value is the mathematical polynomial;
- arithmetic proves the polynomial has exact rational coefficients; and
- algorithmics proves Horner evaluation computes that polynomial.

No proof about derivatives needs to know how a coefficient list is stored, and no proof about lists needs to know the fundamental theorem of calculus.

8.5 Correctness of highest-bit reduction

Suppose the numerator decomposes as $a = 2^b + r$. The analytic Taylor theorem proves exactly the recurrence used by the recursive evaluator. The recursive call has numerator $r < a$, so strong induction on a applies. At the end, the theorem states that evaluating the exact program and embedding its rational output into $\mathbb{R}$ gives $F(a/2^n)$.

The proof handles endpoint and out-of-range cases before entering the recursion. This matches the program structure and avoids forcing the induction hypothesis to cover impossible branch conditions. Each branch ends with a theorem about the corresponding normalization operation: left-tail clamping, right-tail clamping, reflection, highest-bit Taylor reduction, or refinement.

8.6 Representation independence

The same dyadic rational has infinitely many representations:

$$\frac{a}{2^n} = \frac{2a}{2^{n+1}} = \frac{4a}{2^{n+2}} = \dots$$

The evaluator’s mathematical specification therefore includes refinement invariance. `DyadicCorrectness.lean` proves that one common factor of two can be introduced or removed without changing the result; induction yields invariance under any common power.

There are two possible proof routes. One can prove both representations equal the analytic function and conclude by transitivity, or prove refinement directly from the exact recursion. The repository contains strong exact correctness results so that the executable layer remains trustworthy even when used independently of analysis. Analytic equality then gives a second conceptual explanation.

8.7 Dyadic density and uniqueness

Let F and G satisfy `IsFabius`. Correctness gives

$$F(q) = G(q)$$

for every dyadic rational q . To extend this to an arbitrary real x , choose dyadic approximants such as

$$q_n = 2^{-n} \lfloor 2^n x \rfloor.$$

Then $q_n \rightarrow x$. Continuity gives

$$F(x) = \lim_n F(q_n) = \lim_n G(q_n) = G(x).$$

In Lean, density can be applied through a generic theorem saying continuous maps equal on a dense set are equal, or through an explicit approximating sequence. The repository’s dyadic infrastructure supplies the density statement in the exact form required by the function-extensionality proof. Tail values allow the argument to focus on $[0, 1]$ when convenient.

8.8 The global dyadic theorem

`GlobalDyadic.lean` transfers exact evaluation from the bounded interval to `extendedFabius`. The result accepts unrestricted integer numerators and nonreduced dyadic representations. It combines:

1. arithmetic decomposition into an integer translate and a unit-interval remainder;
2. the translation law for the signed extension;
3. exact unit-interval correctness; and
4. representation invariance.

This stronger formulation is the one required by the q -binomial identities, where summation indices naturally produce numerators outside $[0, 2^n]$.

Architectural point

The dyadic bridge is the repository’s main “commuting square.” One path starts with an analytic function and derives a finite Taylor recurrence; the other starts with exact rational recurrences and an evaluator. The correctness theorem says both paths produce the same real number.

Moments, generating functions, and exact denominators

9.1 Analytic moments of the up-function

`AnalyticMoments.lean` develops integral interpretations of the exact moment sequences. Because `up` is smooth and compactly supported, every polynomial moment exists:

$$M_n = \int_{\mathbb{R}} x^n \text{up}(x) \, dx.$$

Evenness immediately gives $M_{2k+1} = 0$. The total mass $M_0 = 1$ is proved from the bounded differential equation and the fundamental theorem of calculus, rather than assumed as part of the definition.

The main recurrence is obtained by integration by parts and a dyadic change of variables. The precise coefficients depend on the chosen centering of `up`, but the structure is a finite convolution of lower moments with powers of two and binomial coefficients. The same recurrence was used to define `moment` in the exact layer.

9.1.1 Integration over compact support

`Mathlib`'s integral on $\mathbb{R}$ is convenient for Fourier transforms, while interval integrals are convenient for integration by parts. The proof repeatedly converts between them using support lemmas. A standard sequence is:

1. prove the integrand vanishes outside a compact interval;
2. rewrite the whole-line integral as an integral over that interval;
3. apply an interval integration-by-parts theorem;
4. show endpoint terms vanish by support or flatness; and
5. transform the remaining integral by an affine substitution.

The support theorem is therefore not merely qualitative metadata; it is an active rewrite rule used throughout the analytic stack.

9.2 Identifying analytic and exact moments

After deriving the recurrence, the module proves by strong induction that

$$M_n = \text{moment}(n)$$

after embedding the rational value into $\mathbb{R}$. The displayed formula intentionally suppresses namespace and cast syntax; the Lean theorem states them explicitly.

The base case is total mass. In the induction step, the analytic recurrence is rewritten term by term using the induction hypotheses. A cast lemma turns the finite rational sum into the corresponding real sum. The exact recurrence then closes the goal.

The same method identifies half-interval moments with `halfMoment`. These moments are connected more directly to inverse-power Fabius values, so they become the analytic bridge between moment theory and the dyadic table.

9.3 Generating functions

The moment generating function is represented both as an integral and as a power series. Compact support gives uniform bounds

$$|M_n| \leq CR^n,$$

which are more than sufficient for convergence of

$$\sum_{n \geq 0} M_n \frac{z^n}{n!}$$

for every complex z . Dominated convergence justifies interchanging sum and integral, giving the usual identity with $\int e^{zx} \text{up}(x) \, dx$.

`MomentPowerSeries.lean` develops the power-series side in a reusable form. It links the exact inverse-power table, moments, and entire functions. The code uses `mathlib`'s `FormalMultilinearSeries` and `HasFPowerSeriesAt` machinery only where it pays off; many coefficient identities are easier as direct statements about `HasSum` and factorially weighted series.

9.4 Removable singularities

Expressions such as

$$\frac{e^z - 1}{z}$$

appear in half-moment generating functions. The source defines a total function with value one at zero, then proves continuity and analyticity across the removable singularity. This avoids carrying $z \neq 0$ through every downstream identity.

The proof combines the exponential power series with division by z term by term. In Lean, defining the value at zero first and proving a global power-series expansion is often cleaner than starting with a piecewise quotient and repeatedly rewriting the nonzero branch.

9.5 Normalized half moments

`NormalizedMoments.lean` proves a closed denominator normalization of the form

$$\text{halfMoment}(n) = \frac{H_n}{(n+1)! \prod_{j=1}^n (2^j - 1)},$$

where H_n is the division-free natural numerator defined in `Arithmetic.lean`. The theorem is not obtained by asking Lean to normalize a large rational expression automatically. It is proved by strong induction with a deliberately chosen common denominator.

At the induction step, every earlier half moment has a denominator whose factorial and Mersenne products are prefixes of the target denominator. The missing suffix factors are inserted explicitly. Product-splitting lemmas show that multiplying an earlier term by its suffix gives the target common denominator. The natural numerator recurrence was designed to be exactly the resulting cleared equation.

9.6 Normalized even moments

`NormalizedEvenMoments.lean` proves the analogous expression for even centered moments. A representative denominator consists of an odd double factorial and an even-indexed Mersenne product:

$$M_{2n} = \frac{E_n}{(2n+1)!! \prod_{j=1}^n (2^{2j} - 1)}.$$

The actual indexing in the source is chosen to align with its recursive sequence, but the structural point is the same. Odd moments vanish analytically, while even moments carry all arithmetic information.

The proof requires more combinatorial normalization than the half-moment case because binomial coefficients and parity restrictions appear in the recurrence. Factorial identities convert binomial coefficients into products compatible with the odd double factorial. The final division-free recurrence is again a natural-number equality.

9.7 Parity through Lucas' theorem

`Parity.lean` proves that the natural moment numerators are odd. The key combinatorial input is Lucas' theorem modulo two: a binomial coefficient $\binom{n}{k}$ is odd exactly when every one-bit of k occurs at a one-bit position of n . Consequently the number of odd entries in row n of Pascal's triangle is

$$2^{s_2(n)}.$$

The moment recurrence is reduced modulo two. Most terms vanish in pairs or carry explicit factors of two; the surviving binomial pattern is counted using Lucas' theorem. The source must bridge several representations of a finite range—natural inequalities, `Fin` types, and `Finset.range`—before applying `mathlib`'s Lucas results. Dedicated cardinality lemmas hide this bridge from the final parity induction.

9.8 Reduced numerators, denominators, and two-adic valuation

`TwoAdic.lean` turns the normalization and parity theorems into exact assertions about reduced rational numerators and denominators. If a displayed quotient has odd numerator and a denominator whose power of two is known, then no cancellation of two-adic factors is possible. This determines the two-adic valuation of the reduced denominator.

The argument differs for odd and even indices. Odd-index statements reduce to the normalized even moments. Even-index statements use the half-moment recurrence together with the Lucas count of odd binomial coefficients. The formal proof includes lemmas relating:

- the numerator and denominator fields of a normalized rational;
- divisibility by two in $\mathbb{N}$ and $\mathbb{Z}$;
- coprimality in reduced form; and
- a valuation statement for the dyadic value $F(2^{-n})$.

9.9 Denominator bounds

`DenominatorBound.lean` and `HalfMomentDenominator.lean` assemble divisibility results into explicit bounds. Rather than attempting to compute the reduced denominator directly, the proof supplies a common denominator known to clear every term in a recurrence. The reduced denominator then divides that common denominator by the universal property of rational normalization.

This method is robust under changes of indexing and does not require unique factorization calculations for the full denominator. Exact two-adic theorems can be layered on top where a sharper result is needed.

9.10 Bernoulli recurrences and source-facing arithmetic

`BernoulliRecurrences.lean` translates some moment identities into Bernoulli-number language used by the arithmetic paper. The formalization keeps Bernoulli manipulations separate from the basic moment recurrence: the latter is elementary and executable, while the former uses a larger special-function and finite-sum API.

`PaperStatements.lean` then exposes propositions and theorems with numbering matching the paper. Its proofs are often short compositions of the stronger internal results described above. This is a good place to learn what each source theorem says, but not always the place to learn why it is true.

Fourier analysis and Rvachev's original characterization

10.1 Why the up-function is the Fourier object

The bounded Fabius function has nonzero constant tails and is not integrable on the line. The Rvachev up-function is compactly supported, smooth, and even, so it belongs to every L^p space and, after viewing it as a smooth function with compact support, defines a Schwartz function. This is the natural object for the Fourier-transform theorems in `FourierAnalytic.lean` and `OriginalUniqueness.lean`.

The original characterization asks for a smooth compactly supported function satisfying a refinement or derivative equation and a normalization. The repository formalizes both existence and uniqueness. The uniqueness proof is especially instructive because it turns a differential equation in physical space into a multiplicative recurrence in frequency space.

10.2 Fourier conventions

Formal Fourier proofs are extremely sensitive to normalization. Mathlib's transform includes a fixed kernel convention; the source therefore writes explicit factors of π , 2π , and the imaginary unit rather than silently adopting a paper convention. The sinc factor is defined compatibly with that convention and extended continuously at zero.

A schematic transform identity is

$$\widehat{\text{up}}(z) = \text{sinc}(\pi z) \widehat{\text{up}}(z/2).$$

Depending on whether the source theorem uses angular or ordinary frequency, the argument of sinc is adjusted. The Lean theorem's constants should be taken as authoritative; the important structural feature is multiplication by one sinc factor and halving of frequency.

10.3 Transforming the derivative equation

The source first proves that a candidate satisfying the original assumptions is compactly supported and smooth enough to be represented as a Schwartz map. It then applies the Fourier derivative

theorem and the affine change-of-variable theorem. A derivative contributes a frequency factor; a dilation contributes both a reciprocal Jacobian and a rescaled frequency; translations contribute complex phases.

After simplification, the phases combine into a sine quotient, producing the sinc recurrence. The mass normalization gives

$$\widehat{\text{up}}(0) = \int_{\mathbb{R}} \text{up}(x) \, dx = 1.$$

This is the seed value that controls the limit after infinitely many refinements.

Lean pattern

The proof does not rewrite Fourier integrals by hand at every step. It first establishes membership in the function class required by `mathlib`, then invokes library theorems for derivative, translation, and dilation. Only the final algebra of complex exponentials is expanded. This concentrates the fragile measure-theoretic assumptions near the beginning of the proof.

10.4 Iteration to a finite product

Iterating the refinement identity n times gives

$$\widehat{\text{up}}(z) = \left(\prod_{j=0}^{n-1} \text{sinc}\left(\frac{\pi z}{2^j}\right) \right) \widehat{\text{up}}\left(\frac{z}{2^n}\right).$$

This is proved by induction. The Lean induction must reconcile two equivalent ways of writing a finite product: appending the last factor versus splitting off the first. A small library of `Finset.range` product identities keeps the main theorem readable.

As $n \rightarrow \infty$, $z/2^n \rightarrow 0$, so continuity of the Fourier transform gives

$$\widehat{\text{up}}(z/2^n) \rightarrow 1.$$

The finite products converge to an infinite product because $1 - \text{sinc}(w) = O(w^2)$ near zero and $\sum_j |z|^2/4^j$ converges.

10.5 The infinite sinc product

`FourierProduct.lean` develops the technical product independently. It proves convergence, continuity in the frequency variable, and compatibility between the finite products produced by iteration and the chosen definition of `rvachevFourierProduct`.

Infinite products in Lean can be represented through partial products and limits rather than through a single primitive operator. The source proves that the sequence is Cauchy using a summable bound on deviations from one. Special care is taken at zeros of a sinc factor: logarithmic product arguments are invalid there, whereas direct partial-product convergence remains sound.

The result is the celebrated formula

$$\widehat{\text{up}}(z) = \prod_{j \geq 0} \text{sinc}\left(\frac{\pi z}{2^j}\right)$$

with the indexing adjusted to the normalization used by the code.

10.6 Fourier injectivity and uniqueness

If two original candidates satisfy the same hypotheses, the iteration argument gives the same Fourier transform for both. The source then uses injectivity of the Fourier transform on Schwartz functions to conclude equality. The final transition is:

1. package each smooth compactly supported function as a Schwartz map;
2. prove their transforms are equal as functions;
3. apply Schwartz-space Fourier injectivity; and
4. project the equality back to ordinary functions.

The theorem also shows that the scaling constant in the original functional equation is forced, rather than being an arbitrary parameter. Evaluation at zero or comparison of total mass leaves only the canonical value, which in the repository’s normalization is two.

10.7 Moment series from the Fourier product

Expanding the transform at zero recovers moments. The logarithm of the sinc product yields cumulant-like sums, while direct multiplication gives a power series whose coefficients are rational combinations of powers of two. `FourierAnalytic.lean` relates this analytic expansion to `rvachevMomentSeries` and the exact moment sequence.

The proof again prefers an identity of entire functions followed by coefficient comparison. Establishing an entire power series permits differentiation at zero to extract moments without repeatedly justifying differentiation under the integral sign.

10.8 Poisson summation

The original paper’s Poisson-summation theorem is formalized in a dedicated module. Compact support and smoothness place the function in the Schwartz class, so `mathlib`’s Poisson summation framework applies. The main work is normalization: the theorem from the library and the paper use different Fourier conventions and summation coordinates. The proof rewrites the transform through the sinc product and then simplifies the sampled factors.

This is representative of source-facing modules: the analytic theorem already exists in a reusable library form, while the paper theorem is a carefully normalized corollary.

Proof idea

The Fourier uniqueness proof is a rigidity argument. The differential/refinement equation repeatedly pushes all unknown information into the value of the transform at a frequency tending to zero. Normalization fixes that limiting value, so no freedom remains.

Finite convolutions, probability, and approximation

11.1 The probabilistic representation

The infinite sinc product is also the characteristic function of an infinite sum of independent centered random variables whose scales decrease geometrically. A typical model is

$$X = \sum_{j \geq 1} 2^{-j} U_j,$$

where the law of each U_j is chosen so its characteristic function is the corresponding sinc factor. The exact formulation in `ProbabilityRepresentation.lean` matches the convolution construction used in the original paper.

The code first treats finite sums. Their laws are finite convolutions of simple compactly supported probability measures. Characteristic functions multiply, so the finite convolution transform is the finite sinc product. Tightness and almost-sure or distributional convergence then identify the limit law with the up-function density.

11.2 Finite convolution measures

A formal convolution proof must verify that every measure involved is finite, usually a probability measure. The source defines the elementary measures, proves their support and total mass, and then uses induction to show the convolution remains a probability measure. Associativity is applied only after the necessary sigma-finiteness or finiteness instances are in scope.

The support of the n -fold convolution is the Minkowski sum of the elementary supports. Geometric decay keeps these supports in a fixed compact interval. This uniform support bound supplies tightness essentially for free and gives uniform moment bounds.

11.3 Weak convergence

`WeakConvergence.lean` proves convergence against bounded continuous test functions. There are two complementary mechanisms:

- convergence of random partial sums, followed by bounded convergence; and
- convergence of characteristic functions plus identification of the unique limit.

The repository packages the result in the measure-theoretic language needed by later interval and distribution-function arguments.

The test-function formulation avoids trying to prove pointwise convergence of densities too early. Weak convergence is stable under convolution and directly reflects the probabilistic construction. To recover the Fabius function, the code later specializes to approximations of interval indicators.

11.4 Step approximants

`StepApproximationLimit.lean` studies the explicit step functions arising from finite convolution coefficients. The coefficient arrays satisfy symmetry and unimodality properties. These imply monotonicity of the approximants on each side of the center and provide pointwise upper and lower controls.

The desired limit is continuous, but indicator functions of intervals are not continuous test functions. The source uses an interval-average squeeze. Integrals over a small neighborhood converge by weak convergence; monotonicity bounds the point value of a step function between nearby interval averages. Shrinking the neighborhood and using continuity of the limit gives pointwise convergence.

11.5 The endpoint half-weight correction

Closed interval indicators double-count shared endpoints when adjacent steps are assembled. In ordinary integration this affects a measure-zero set, but a pointwise theorem about the approximating function sees the discrepancy. The formalization therefore uses a representative with half weight at step boundaries.

This correction is a good example of Lean revealing a genuine mathematical issue rather than a mere elaboration problem. The source paper's almost-everywhere intuition is adequate for an integral identity, but not for the stated pointwise approximation. Once the half-endpoint convention is adopted, neighboring intervals partition correctly and the convergence theorem has the intended endpoint values.

11.6 Coefficient unimodality

The finite convolution coefficients form rows with strong symmetry and unimodality. The proof proceeds through a recurrence: the next row is obtained by averaging shifted blocks of the previous row. Symmetry is immediate from the symmetric elementary measure; monotonicity requires comparing overlapping partial sums.

Lean handles the row as a finite function or list. The proof establishes index bounds before rewriting recurrence entries. Boundary entries, where one shifted block leaves the valid range, are split into separate cases. These technical cases explain why the coefficient lemmas occupy a substantial part of `StepApproximationLimit.lean`: the final analytic squeeze is comparatively short once monotonicity is available.

11.7 From distribution functions to the bounded Fabius function

The limiting probability law has density up. Its cumulative distribution, after the repository's translation convention, is the bounded Fabius function. The proof integrates the density over a half-line, uses support to reduce to a compact interval, and applies the fold relation between F and up.

This gives an independent conceptual construction of the same object as the Banach fixed point. In the formal dependency graph it is normally developed after the canonical function is available, because identification is easier than rebuilding the entire `IsFabius` interface from measure convergence. Mathematically, however, it explains why the function is a distribution function.

11.8 Exact finite Taylor formulas and nonanalyticity

`OriginalPaperSupplement.lean` packages further theorems from the original paper. At centered dyadic points, sufficiently high derivatives vanish on one side because repeated scaling reaches the support boundary. Taylor's theorem becomes an exact finite polynomial identity on a small interval.

If the function were analytic at such a point, its Taylor series would terminate and the function would be a polynomial in a neighborhood. The differential equation and support properties rule this out unless the function were trivial. Thus the exact local truncation contributes to a nowhere-analyticity theorem.

The formal proof distinguishes several notions:

- `ContDiff` smoothness;
- existence of derivatives of all orders at a point;
- equality to the Taylor series on a neighborhood; and
- `mathlib`'s analytic-at predicate.

A proof that merely observes all derivatives vanish at a boundary point establishes flatness, not by itself nonanalyticity everywhere. The module supplies the additional translation and dyadic-density argument needed for the global conclusion.

11.9 Corrections recorded by the original-paper aggregator

`Paper05442.lean` and its supplement explicitly document several source-sensitive points. Among them are:

- finite convolution must be stated at the finite stage rather than with an already limiting object;
- step-function endpoints require half weights for pointwise identities;
- a factor of the transform variable is needed in one displayed Fourier equation;
- a formula involving a positive order requires the hypothesis $n > 0$; and
- a global dyadic identity needs the signed extension or an adjusted normalization.

These are not hidden as comments in low-level proofs. The aggregator treats them as part of the formal audit of the paper.

The two paper formalizations as navigable APIs

12.1 The first paper: `Paper05442.lean`

The module named after `arXiv:1702.05442` imports the original uniqueness proof, probability representation, weak convergence, step approximation, Poisson summation, and supplementary source theorems. Its purpose is to certify coverage of the paper as a whole.

A useful reading order is:

1. `OriginalUniqueness.lean` for the defining characterization;
2. `ProbabilityRepresentation.lean` for the random/convolution model;
3. `WeakConvergence.lean` for the measure limit;
4. `StepApproximationLimit.lean` for pointwise approximation;
5. `PoissonSummation.lean` for the sampled Fourier identity; and
6. `OriginalPaperSupplement.lean` for moments, Taylor formulas, rationality, and nonanalyticity.

The aggregator’s theorem names are source-facing, but the mathematical objects remain those of the common core. This avoids a second, paper-specific definition of the function.

12.2 The arithmetic paper: `Paper06487.lean`

The second aggregator, named after `arXiv:1702.06487`, imports `Paper06487Supplement.lean`, which in turn draws on `PaperStatements.lean` and the arithmetic/analytic bridge modules. It covers the paper’s propositions on moments, exact dyadic values, denominators, parity, and two-adic valuations.

The internal proof structure is more extensive than the paper numbering suggests:

exact recurrence $\longrightarrow$ normalized numerator $\longrightarrow$ parity/divisibility $\longrightarrow$ reduced rational theorem.

Each arrow is represented by one or more modules. The final paper theorem is often a cast or notation wrapper around an internal theorem with a stronger domain or a more explicit divisibility conclusion.

12.3 Statements that remain questions or conjectures

A formalization should not turn an open question into a theorem. `PaperStatements.lean` records the paper’s Question 5, Definition 12, and Conjecture 16 as definitions or propositions expressing the claimed predicate, not as proofs of the unresolved assertion. This is part of the source-coverage discipline: every numbered item is represented, but its logical status is preserved.

The surrounding proved theorems may establish initial cases, necessary conditions, or denominator bounds relevant to the conjecture. Their names make clear when the result is a theorem about the conjectural predicate rather than a proof of the conjecture itself.

12.4 Why a separate coverage document matters

The repository’s `PAPER_COVERAGE.md` maps source theorem numbers to Lean declarations. This is more than project management. It helps distinguish three questions:

1. Has the mathematical claim been formalized?
2. Where is the strongest internal theorem proved?
3. Which wrapper corresponds to the source numbering and notation?

Appendix C reproduces the navigational content of that map in compact form and adds the surrounding module families.

Thue–Morse and q-binomial identities

13.1 Why q-binomial algebra appears

The Thue–Morse sign converts binary digit structure into alternating sums. When a finite Taylor formula is applied repeatedly across dyadic cells, the resulting coefficients naturally involve Gaussian, or q-binomial, coefficients at $q = 1/2$. The repository develops this relationship in a series of modules ranging from exact finite identities to global analytic series.

The core finite identity has the flavor

$$\sum_{k=0}^{2^n-1} \varepsilon(k)(x+k)^m,$$

which vanishes for low degree and factors in a controlled way at the critical degree. q-binomial coefficients provide a compact description after binary decomposition. The Fabius values enter when the polynomial identity is inserted into a scaled finite Taylor expansion.

13.2 The finite q-Pochhammer infrastructure

`HalfQBinomial.lean` develops exact q-binomial algebra at $q = 1/2$ over $\mathbb{Q}$. The finite q-Pochhammer product is represented without analytic convergence issues:

$$(a; q)_n = \prod_{j=0}^{n-1} (1 - aq^j).$$

At $q = 1/2$, clearing powers of two turns many factors into Mersenne numbers. This is why the Mersenne products introduced in `Arithmetic.lean` reappear.

The module defines a rational `halfQBinomial` and proves its q-Pascal recurrence. It then proves a finite q-binomial theorem by induction. The induction step is not a black-box appeal to an imported polynomial theorem; it matches the precise normalization required downstream. Summands are shifted and paired so the q-Pascal recurrence telescopes into the next product.

13.3 Why prove a specialized q-binomial theorem?

Mathlib contains general polynomial and combinatorial infrastructure, but a specialized theorem at $q = 1/2$ gives the repository:

- exact rational coefficients with predictable denominators;
- direct compatibility with existing Mersenne products;
- a recurrence usable by kernel computation; and
- statements in the same finite-sum form as the Fabius formulas.

A general theorem would still require a substantial specialization layer. Proving the specialized identity directly keeps casts and nonvanishing side conditions under control.

13.4 Raw, scalar, and polished formulas

The q-binomial family is intentionally split into several modules:

- `FabiusRawQBinomialFormula.lean` and `FabiusRawQBinomialScalar.lean` establish algebraic sums before all Fabius-specific simplifications;
- `FabiusQBinomialFormula.lean` proves the exact finite identity in its principal form;
- `FabiusQBinomialScalarFormula.lean` specializes coefficient-valued statements to scalar values; and
- `FabiusDyadicQBinomialScalar.lean` inserts the exact dyadic evaluator.

This decomposition protects the core polynomial identity from unnecessary analytic dependencies. It also allows translation invariance to be proved at the polynomial level and reused after evaluation.

13.5 The unrestricted finite identity

A notable strength of the formal result is that it is not restricted to a reduced dyadic representative or to $0 \leq m \leq 2^n$. It is stated for arbitrary natural parameters in the range supported by the algebraic expression. The signed global extension absorbs the translation behavior that would otherwise force an artificial range hypothesis.

The proof proceeds by induction on the binary depth. Splitting a range into even and odd indices transforms the Thue–Morse sign into a difference of two scaled copies. The induction hypothesis applies to each copy, and the q-Pascal recurrence combines the coefficients. Polynomial degree bounds control vanishing terms.

Proof idea

The even/odd split is the common engine behind Thue–Morse cancellation and q-binomial recursion. In Lean, the decisive step is a reindexing theorem that expresses a sum over `Finset.range(2*N)` as sums over $2k$ and $2k + 1$. Once that lemma has the right shape, the sign and power transformations simplify automatically.

13.6 Translation invariance as a polynomial theorem

One of the strongest internal statements says that the relevant finite q -expression is independent of a translation parameter because it is a polynomial of degree below the cancellation threshold. This is proved before specializing the parameter to real or complex values. As a result, common translation invariance holds over a broad coefficient ring and later applies uniformly to $\mathbb{R}$ and $\mathbb{C}$.

This is a recurring proof-design strategy: prove a fact at the most algebraic level where it is true, then transport it through evaluation homomorphisms. Analytic specializations become one-line consequences of polynomial extensionality.

13.7 Taylor expansion in the q parameter

`FabiusQBinomialTaylor.lean` analyzes the finite expression as a polynomial or power series in q . Coefficients are finite and exact; no convergence theorem is needed. The Taylor expansion supplies estimates for the later discrete-limit theorem when a complex shift parameter approaches its limiting value.

The source separates coefficient extraction from norm estimates. First it proves an exact coefficient formula. Then it bounds each coefficient by a combinatorial majorant, and finally sums the majorant. This ordering produces reusable algebraic theorems and prevents analytic inequalities from obscuring finite identities.

13.8 The global binary-reduction series

`FabiusBinaryReductionSeries.lean` and `FabiusGlobalQBinomialSeries.lean` pass from finite identities to an infinite series representing the signed extension. The binary reduction follows the same highest-bit philosophy as the exact evaluator, but now the steps are organized as a global q -series.

A subtle endpoint correction places the outer index at $m = 0$, not $m = 1$. At the initial endpoint the missing term is not negligible; it carries the base value that makes the telescoping identity exact. The formal proof reveals this immediately because a finite partial-sum formula leaves a nonzero boundary term.

The convergence argument uses strong coefficient decay inherited from products of powers of two and factorials. Uniform convergence on compact sets permits termwise continuity and, where required, differentiation. The final equality is first proved at dyadic points using exact correctness and then extended by continuity, or derived directly by telescoping finite approximations depending on the module.

13.9 Parity-power series

`FabiusParityPowerSeries.lean` packages a related expansion in which the coefficients are controlled by binary parity. The Thue–Morse sign is represented as an exact coefficient sequence, and finite cancellation gives a high-order zero at the origin. Bounds on the remaining coefficients yield an entire or locally uniform series as appropriate.

This module is a bridge between combinatorics and complex analysis. The coefficient identities are exact, while convergence is proved with norms and comparison tests. Keeping these two layers separate makes it possible to reuse the parity theorem in purely arithmetic settings.

Discrete limits and complex shifts

14.1 The shape of the limit theorem

The discrete-limit project starts from finite q -binomial/Thue–Morse rows depending on a real location x and a complex parameter q . For nondyadic x , an individual finite row can depend genuinely on q . The theorem is that, after the prescribed normalization and as the row depth tends to infinity, the limit exists and is independent of q , equaling the relevant Fabius expression.

This distinction between finite-stage dependence and limiting independence is explicitly formalized. It prevents an overstrong claim that each approximant is constant in the parameter.

14.2 Uniform spline approximants

`FabiusUniformSpline.lean` introduces centered finite Thue–Morse splines. These are piecewise-polynomial functions whose coefficients encode a finite convolution row. The splines are uniformly bounded and supported in a common compact region after scaling.

The source proves continuity at knots by comparing adjacent polynomial pieces. Finite-difference cancellation ensures derivatives up to the expected order also match. Rather than relying on a generic spline package, the proof works directly with the explicit pieces because their coefficients must later be transformed into the q -binomial sum.

14.3 Complex-shift control

`FabiusComplexShiftSpline.lean` and `FabiusDiscreteLimitComplexShift.lean` control evaluating a finite spline at a complex displacement. A Taylor expansion separates the real centered value from correction terms. Uniform derivative bounds, together with the scale of the shift, show the corrections vanish in the limit.

The proof is careful not to use order arguments on $\mathbb{C}$. Norm bounds replace real inequalities, and factorial denominators are estimated in $\mathbb{R}$ before being cast into complex norms. The finite nature of the Taylor expansion is again important: each finite spline has bounded degree, so no complex analytic remainder theorem is required.

14.4 The Toeplitz averaging mechanism

`FabiusDiscreteLimitToeplitz.lean` rewrites the normalized row as a Toeplitz-type weighted average of centered spline values. The weights are nonnegative or absolutely summable after taking norms, their total tends to one, and mass far from the central region vanishes. A general Toeplitz convergence principle then transfers pointwise or uniform convergence of the splines to convergence of the averages.

The formal theorem isolates three hypotheses:

1. a uniform bound on the input sequence;
2. convergence on every fixed central window; and
3. concentration and normalization of the weights.

This abstraction is useful beyond the specific Fabius application and keeps the final limit proof from becoming a nested epsilon argument.

14.5 Integration and identification of the limit

`FabiusDiscreteLimitIntegration.lean` is the final integration layer. Uniform convergence of cumulative distributions or spline primitives permits passage to the limit under an interval integral. Boundary terms telescope into the global binary-reduction series, which has already been identified with `extendedFabius`.

The result is stated for $x \geq 0$ and complex q in the supported domain. The restriction on x reflects the normalization of the finite rows; global signed-extension identities can then translate or reflect additional cases.

14.6 Proof flow of the discrete-limit family

A recommended source order is:

1. `FabiusRawQBinomialFormula.lean`;
2. `FabiusQBinomialFormula.lean`;
3. `FabiusBinaryReductionSeries.lean`;
4. `FabiusUniformSpline.lean`;
5. `FabiusComplexShiftSpline.lean`;
6. `FabiusDiscreteLimitToeplitz.lean`;
7. `FabiusDiscreteLimitComplexShift.lean`; and
8. `FabiusDiscreteLimitIntegration.lean`.

The sequence moves from exact finite algebra, through real approximation, to complex perturbation, averaging, and final analytic identification.

Formalization-sensitive point

When reading these files, distinguish the row index from the polynomial degree, the dyadic scale, and the q -parameter. Informal drafts often use a single letter for more than one role. The Lean development uses separate variables and explicit type annotations, which can make theorem statements longer but prevents illegal substitutions.

The logarithmic scale near the flat endpoint

15.1 Why ordinary Taylor asymptotics fail

Every derivative of the bounded Fabius function vanishes at zero, yet $F(x) > 0$ for every $x > 0$. Thus no nonzero power of x is the leading term. The correct scale is logarithmic and, at first order, quadratic in $-\log x$.

The asymptotic development introduces

$$\phi(t) = F(2^{-t}), \quad g(t) = -\log \phi(t).$$

As $x \downarrow 0$, the variable $t = -\log_2 x$ tends to infinity. This turns dyadic scaling into unit translation and is the natural coordinate for the delay equation.

15.2 The exact logarithmic delay identity

`FabiusLogScale.lean` differentiates $\phi(t) = F(2^{-t})$. For $t \geq 1$, the argument 2^{-t} lies in the left branch where the differential equation applies. The chain rule gives

$$\phi'(t) = -(\log 2)2^{-t}F'(2^{-t}) = -2(\log 2)2^{-t}F(2^{1-t}).$$

Since $\phi(t-1) = F(2^{1-t})$, this becomes a delay relation. Rewriting in terms of $g = -\log \phi$ and its derivative produces the exact identity formalized in the module:

$$g(t) - g(t-1) = \log g'(t) - \log(\log 2) - (1-t)\log 2, \quad t \geq 1.$$

The source first proves positivity of $F(2^{-t})$ and the relevant logarithmic slope, so every logarithm is legal.

Formalization-sensitive point

The threshold $t \geq 1$ is essential. The bounded function's derivative law does not hold on the same branch for all real t . An informal draft treated the delay identity as global; the Lean theorem records the correct domain and downstream asymptotic arguments are formulated with eventual filters.

15.3 Positivity near zero

Taking a logarithm requires strict positivity, not merely nonnegativity. The repository obtains positivity from monotonicity and exact dyadic values. Given $x > 0$, choose a sufficiently large inverse power of two below x . Exact arithmetic shows $F(2^{-n}) > 0$, and monotonicity gives $F(x) \geq F(2^{-n}) > 0$.

This proof is representative of the interaction between exact and analytic layers: a qualitative analytic property at every positive real point is obtained from exact values on a dense cofinal sequence.

15.4 The quadratic logarithmic law

`FabiusLogSquaredAsymptotic.lean` proves

$$\frac{g(t)}{t^2} \longrightarrow \frac{\log 2}{2}.$$

Equivalently,

$$\log F(x) \sim -\frac{(\log(1/x))^2}{2 \log 2} \quad (x \downarrow 0).$$

The proof first establishes sufficiently sharp estimates at natural values of t , where the exact inverse-dyadic recurrence is available. It then extends them to real t by monotonicity and floor/ceiling comparison.

If $n \leq t < n + 1$, monotonicity gives

$$g(n) \leq g(t) \leq g(n + 1)$$

with the inequality direction determined by the decreasing behavior of $F(2^{-t})$. Dividing by t^2 and using $n/t \rightarrow 1$ transfers the natural asymptotic to real arguments.

15.5 A quantitative coarse bound

The same module proves a bound of the form

$$\left| g(t) - \frac{\log 2}{2} t^2 \right| \leq Ct \log t$$

for all sufficiently large t . This is weaker than the eventual sharp formula but strong enough to establish flatness and compare F with elementary scales.

Such a concrete inequality is more useful in Lean than a bare $o(t^2)$ statement. It can be inserted into exponential comparisons with explicit thresholds, and it supports later proofs that every power of x dominates $F(x)$ while every $e^{-c/x}$ is dominated by $F(x)$.

15.6 Filter formulations

The repository states asymptotics both in the $t \rightarrow \infty$ coordinate and as $x \rightarrow 0^+$. Conversion lemmas relate the filters under $x = 2^{-t}$. A typical proof establishes a limit along `atTop`, composes with the continuous map $t \mapsto 2^{-t}$, and then identifies the image filter with `nhdsWithin0(Set.Ioi0)`.

Explicit conversion lemmas are preferable to ad hoc substitutions in each theorem. They ensure that positivity domains and logarithm definitions are carried through uniformly.

Negative Laplace analysis and the periodic correction

16.1 The negative moment generating function

Let

$$G(-s) = \int_{\mathbb{R}} e^{-sx} \operatorname{up}_+(x) \, dx$$

denote the relevant negative Laplace transform in the repository's normalization. The exact moment-generating product gives, for $s > 0$,

$$G(-s) = \prod_{j \geq 1} \frac{1 - e^{-s/2^j}}{s/2^j}.$$

The product's logarithm is

$$L(s) = \sum_{j \geq 1} \left[\log(1 - e^{-s/2^j}) - \log(s/2^j) \right].$$

`NegativeLaplace.lean` constructs this series carefully and proves absolute convergence after grouping terms in a form adapted to small and large arguments.

16.2 The kernel and its bounds

The central scalar kernel is the logarithm of a normalized Bose factor. For small u , the quotient $(1 - e^{-u})/u$ tends to one, so its logarithm is $O(u)$. For large u , the exponentially small correction $\log(1 - e^{-u})$ is controlled by a geometric series.

The source proves elementary inequalities before summing over dyadic scales. Typical estimates use

$$0 \leq -\log(1 - v) \leq \frac{v}{1 - v} \quad (0 \leq v < 1),$$

with $v = e^{-u}$. The resulting majorants are summable after the dyadic split at the scale where $s/2^j$ crosses one.

16.3 The exact dilation equation

The product shifts cleanly under $s \mapsto 2s$. Removing the first factor and reindexing the rest gives an exact functional equation for $L(s) = \log G(-s)$. Solving the associated difference equation in the logarithmic variable $u = \log_2 s$ produces a quadratic polynomial plus a periodic remainder.

The formal decomposition is

$$L(s) = -\frac{(\log s)^2}{2 \log 2} + \frac{1}{2} \log s + R(\log_2 s) + E(s),$$

where $R(u+1) = R(u)$ and the forward-tail error satisfies an explicit exponential bound. A representative bound proved in the module is

$$|E(s)| \leq \frac{e^{-s}}{(1 - e^{-s})^2},$$

and for s beyond a fixed threshold this is simplified to a constant multiple of e^{-s} .

16.4 How the periodic function is constructed

The periodic term is not guessed from numerics. The code builds it by comparing the exact logarithm with a particular quadratic solution of the dilation equation. The difference is invariant under doubling up to a forward tail. Iterating the dilation toward infinity makes that tail summable; adding its convergent correction yields an exactly invariant function of $\log_2 s$.

Concretely, one introduces a defect

$$D(s) = L(s) + \frac{(\log s)^2}{2 \log 2} - \frac{1}{2} \log s.$$

The dilation identity shows that $D(2s) - D(s)$ is an exponentially small kernel. A forward sum of these defects corrects D into a function satisfying $P(2s) = P(s)$. Writing $R(u) = P(2^u)$ gives period one.

Proof idea

The periodic correction is the homogeneous solution of a dyadic difference equation. The quadratic logarithmic terms form a particular solution. Exponentially small forward tails determine which periodic solution corresponds to the actual infinite product.

16.5 Continuity and regularity

`PeriodicCorrection.lean` proves continuity of R . The defining correction is a locally uniformly convergent series on positive compact intervals. The proof covers a compact interval $[a, b] \subset (0, \infty)$, bounds every tail term uniformly by a summable geometric sequence depending on a , and applies a Weierstrass-type theorem.

Later modules strengthen regularity. `PeriodicRegularity.lean` differentiates the correction series term by term after proving uniform convergence of derivative tails. `PeriodicMean.lean` computes or normalizes its mean. `PeriodicFourier.lean` identifies Fourier coefficients through Mellin-transform data.

16.6 Mellin–Bose analysis

`MellinBose.lean` studies the Mellin transform of the Bose kernel that appears in the logarithm. The transform has factors involving the Gamma and zeta functions. Poles encode the polynomial logarithmic terms, while values on the imaginary lattice generated by $2\pi/\log 2$ encode the Fourier coefficients of the period-one correction.

The formal proof proceeds first in a vertical strip where the defining integral is absolutely convergent. It then relates the integral to known Gamma/zeta representations and uses analytic continuation or finite-part identities only in modules that import the required complex-analysis machinery.

16.7 Finite parts and Bromwich preparation

`MellinFinitePart.lean`, `BoseFinitePartIntegral.lean`, and related modules remove singular polynomial pieces from the Mellin inversion formula. A finite-part integral is represented as an ordinary convergent integral after subtracting the local Laurent terms explicitly.

This decomposition is important for formal contour arguments. Instead of manipulating a divergent integral symbolically, Lean sees a sum of:

- explicit residues;
- a convergent integral on a truncated contour; and
- error terms with proved norm bounds.

The periodic Fourier series then emerges from residues at the imaginary lattice.

16.8 Why the periodic term cannot be dropped

The oscillatory correction $R(\log_2 s)$ is bounded but nonconstant. Therefore it is lower order than the quadratic logarithmic term but not an error tending to zero. Any asymptotic formula claiming an $o(1)$ or $O(1/\log s)$ residual after omitting R is false.

`FabiusWikipediaObstruction.lean` and related audit modules formalize this obstruction. They prove nonconstancy through a nonzero Fourier coefficient or a pair of points with unequal values. Sampling along two dyadic phase subsequences then produces distinct limit points for the alleged residual.

16.9 Identification with the generating function

After constructing the product and its logarithm, `NegativeLaplace.lean` proves equality with the analytic generating function defined earlier. The route is:

1. establish the product formula for finite truncations;
2. prove convergence to the infinite product;
3. establish the same functional equation and normalization for the generating function; and
4. use uniqueness or direct limit passage to identify them.

This closes the loop between exact moment series, Fourier/probability products, and the asymptotic Laplace object.

Bromwich inversion and the sharp saddle point

17.1 From a transform to the endpoint density

The sharp asymptotic of $F(x)$ is obtained by inverting a Laplace transform. Schematically,

$$F(x) = \frac{1}{2\pi i} \int_{\sigma-i\infty}^{\sigma+i\infty} e^{sx} H(s) ds,$$

where H is built from the negative-Laplace product and normalization factors. The contour parameter $\sigma > 0$ is chosen at a saddle depending on x .

`BromwichSaddle.lean` and `FabiusBromwichInput.lean` provide the inversion identity and the analytic bounds needed to move to the saddle line. Formal contour integrals are represented through finite vertical segments followed by a limit. Tail bounds justify passage to the infinite line.

17.2 The Lambert phase

Differentiating the dominant exponent produces an equation in which the saddle scale appears both linearly and inside a logarithm. The exact solution is expressed through a lower Lambert-type function. The repository defines the relevant phase $\lambda(x)$ and proves it is the unique positive solution of the saddle equation in the required range.

The advantage of retaining the exact Lambert phase is that all-order expansions can be stated without repeatedly composing truncated asymptotic series. The leading behavior is

$$\lambda(x) = \log \frac{1}{x} + \log \log \frac{1}{x} + O(1),$$

but the exact implicit relation captures the corrections needed in both the exponential and Gaussian normalization.

`FabiusLambertPhase.lean`, `FabiusLambertRates.lean`, and `FabiusLambertSaddle.lean` establish monotonicity, growth, derivative bounds, and the exact saddle equation. Separate modules for small-argument expansions later translate λ into elementary logarithms to any fixed order.

17.3 Exact reduction at the saddle

`FabiusSharpExactReduction.lean` packages the inversion calculation into an exact error decomposition. In logarithmic form, it has the conceptual shape

$$\log F(x) - M(x) = \log(\text{normalized saddle mass}) + \text{forward-tail correction},$$

where $M(x)$ contains the explicit quadratic-logarithmic, Lambert, Gaussian, and periodic pieces.

This theorem is the critical interface between contour analysis and asymptotic algebra. All later approximations can focus on two controlled terms: a normalized central integral tending to one, and an exponentially flat tail inherited from the negative-Laplace product.

17.4 Central and tail decomposition

Write the vertical contour variable as $s = \sigma + iy$. The integral is split at a central radius $|y| \leq Y(\lambda)$:

$$I = I_{\text{central}} + I_{\text{tail}}.$$

Near the saddle, a Taylor expansion of the exponent begins with a negative quadratic term. Away from the saddle, a minor-arc estimate shows strict decay of the real part.

The module family includes `FabiusSaddleCentral.lean`, `FabiusSaddleTail.lean`, `FabiusLambertMinorArc.lean`, and reference-weight modules. The split allows different techniques:

- local cumulant expansions and Gaussian comparison in the center;
- coarse but uniform exponential inequalities in the tail.

Trying to use the local Taylor series on the entire contour would give an unjustified remainder.

17.5 The Gaussian reference weight

After rescaling y by the square root of the second derivative at the saddle, the leading central factor is $e^{-u^2/2}$. `FabiusSaddleReferenceWeight.lean` and `FabiusSaddleReferenceTail.lean` collect exact Gaussian integrals and tail bounds in the normalization used by the contour.

The normalized saddle mass is divided by the Gaussian main integral so its limit should be one. This normalization simplifies logarithms: once the ratio is $1 + O(1/\lambda)$, its logarithm is controlled by a standard $\log(1 + u)$ estimate.

17.6 Cumulants and derivative bounds

The logarithm of the negative-Laplace product has derivatives expressible as dyadic sums of kernel derivatives. `LaplaceCumulantAsymptotics.lean`, `FabiusLambertDerivativeBounds.lean`, and related modules prove uniform estimates for these cumulants at the saddle.

The second derivative determines the Gaussian width. Higher derivatives determine correction polynomials. The source proves bounds of the form

$$\kappa_r(\lambda) = O(\lambda^{1-r/2})$$

after the chosen normalization. Exact exponents vary with convention, but the hierarchy guarantees that each additional Taylor order contributes another inverse power of the large phase.

17.7 Minor arcs and tail decay

A complex product may have oscillatory peaks away from zero. `FabiusLambertMinorArc.lean` proves that, outside the central window, enough sinc/Bose factors lose modulus to force decay. The proof divides the frequency axis into ranges:

1. a near-central range where a quadratic cosine or exponential inequality applies;
2. an intermediate range where a fixed block of dyadic factors supplies uniform loss; and
3. a far range controlled by elementary transform decay and integration by parts.

Each range has a separate numerical inequality, making the final integral estimate a finite case split rather than an opaque global bound.

17.8 The first correction

`FabiusFirstSaddleCorrection.lean` computes the first non-Gaussian term. Cubic and quartic derivatives of the phase contribute through Gaussian moments. Odd terms integrate to zero against the even Gaussian; the first surviving coefficient combines the quartic term and the square of the cubic term.

The code represents the correction as a polynomial in the rescaled central variable. Exact Gaussian moments are evaluated recursively or through double factorials. The resulting rational coefficient is then inserted into the normalized saddle mass expansion.

17.9 The corrected sharp formula

`FabiusSharpAsymptotic.lean` states the sharp logarithmic asymptotic. In schematic form,

$$\log F(x) = -\frac{\lambda(x)^2}{2 \log 2} + A\lambda(x) + B \log \lambda(x) + R(\theta(x)) + C + O\left(\frac{1}{\lambda(x)}\right),$$

where $\lambda(x)$ is the exact lower-Lambert phase and R is the nonconstant period-one correction evaluated at an exact phase $\theta(x)$. The constants and phase shifts are fixed by the repository's transform normalization.

Exponentiating gives an asymptotic equivalence for $F(x)$ itself. The proof first bounds the logarithmic error, then uses continuity of the exponential and the fact that $e^{O(1/\lambda)} = 1 + O(1/\lambda)$.

17.10 Failure of an uncorrected elementary formula

A simpler formula obtained by dropping the periodic term may still reproduce the leading $(\log x)^2$ scale. It does not have a residual of order $1/(-\log x)$. The repository proves this by choosing two phase subsequences along which the periodic correction approaches different values. The residual therefore has distinct limit points and cannot tend to zero.

Similarly, a proposed quotient of elementary exponentials is shown to decay on the wrong scale. `FabiusQuotientExponentialMismatch.lean` compares logarithms and proves the quotient is not asymptotically equivalent to F , even if a finite numerical range looks persuasive.

Architectural point

The sharp theorem does not emerge from manipulating the delay equation alone. The delay equation explains the coarse quadratic scale. The periodic term and correct constant require the exact transform product; the Gaussian factor and inverse-phase corrections require Bromwich inversion and saddle analysis.

All-order asymptotic expansions

18.1 Why the exact phase is retained

An all-order expansion can be expressed either in the exact saddle variable $\lambda(x)$ or directly in powers of $-\log x$ and iterated logarithms. The repository takes the exact-phase expansion as primary. This avoids a common error: composing two finite asymptotic series and claiming an arbitrary-order remainder without proving uniform control of the composition.

The exact-phase theorem has the form

$$\mathcal{M}(x) \sim \sum_{j \geq 0} \frac{c_j}{\lambda(x)^j},$$

where $\mathcal{M}(x)$ is the normalized saddle mass. For every N , the difference from the first N terms is $O(\lambda^{-N})$.

18.2 Formal logarithm of the Laplace product

`FabiusLambertFormalLog.lean` extracts a formal series for the local phase. It treats logarithms, exponentials, and reciprocal series in a truncated polynomial ring. The goal is algebraic: compute coefficients that must occur if a sufficiently smooth analytic remainder is controlled.

Separating formal algebra from analytic remainder estimates is essential. Formal power-series identities can be checked by ring normalization and coefficient extensionality. Their analytic validity is supplied later by bounds on the omitted terms.

18.3 Higher small-argument expansion

`FabiusLambertHigherExpansion.lean` and `FabiusLambertAllOrderSmallArgument.lean` prove the expansion of the exact Lambert phase in terms of

$$L = \log(1/x), \quad \ell = \log L.$$

For each fixed order, λ is represented by a polynomial in $1/L$ whose coefficients are polynomials in ℓ , plus a rigorously bounded remainder.

The proof uses the implicit saddle equation. An approximate phase is substituted into the equation; formal algebra shows cancellation through the desired order. Monotonicity and a derivative lower bound for the implicit function turn the equation residual into a bound on the phase error.

18.4 Gaussian polynomial contractions

The central exponential has an expansion

$$e^{-u^2/2} \exp \left(\sum_{r \geq 3} \lambda^{1-r/2} a_r (iu)^r \right).$$

Truncating the second exponential produces polynomials in u and $1/\sqrt{\lambda}$. Odd monomials vanish after integration; even monomials contract to Gaussian moments. The modules `GaussianPolynomialContraction.lean`, `GaussianPolynomialWholeIntegral.lean`, and the tail modules formalize this mechanism.

A polynomial is represented by finitely supported coefficients. The contraction functional maps u^{2k} to $(2k - 1)!!$ and odd powers to zero. Proving that contraction commutes with finite sums and scalar multiplication turns the central coefficient calculation into exact algebra.

18.5 Coefficient recurrences

`FabiusSaddleCoefficientRecurrence.lean`, `FabiusSaddlePolynomialCoefficients.lean`, and `FabiusSaddleExpansionCoefficients.lean` organize the coefficients recursively. Each order depends only on finitely many cumulants and lower coefficients. The source avoids committing to a single giant closed form; a recurrence is more executable and easier to prove correct.

The recurrence is justified by differentiating or multiplying truncated formal series and comparing coefficients. Lean’s finite-support polynomial representation ensures every coefficient sum is finite. Index constraints are handled with `Finset.antidiagonal` or explicit bounded ranges.

18.6 Uniform central remainders

Formal coefficients alone do not prove an asymptotic expansion. `FabiusSaddleCentralAllOrders.lean` bounds the difference between the exact central integrand and its truncated polynomial approximation uniformly on the growing central window.

The window grows slowly enough that the normalized Taylor remainder tends to zero at the requested inverse-phase rate, but fast enough that the omitted Gaussian tail is negligible. Choosing this window is a balancing act. The proof records explicit inequalities among its powers of λ , so later estimates reduce to comparisons of monomials.

18.7 All-order tails

`FabiusSaddleTailAllOrders.lean`, `GaussianPolynomialTailAllOrders.lean`, and `FabiusLambertAllOrderRemainder.lean` show that tail errors are smaller than every prescribed

inverse power. Exponential decay is converted into polynomial decay using standard eventual inequalities:

$$e^{-c\lambda^\alpha} = O(\lambda^{-N}) \quad \text{for every fixed } N.$$

The source proves such comparisons with explicit thresholds and reuses them through Big-O transitivity.

18.8 Assembling the normalized mass expansion

`FabiusSaddleMassAllOrders.lean` combines central coefficient extraction and tail flatness. For each N , it proves

$$\mathcal{M}(x) - \sum_{j=0}^{N-1} \frac{c_j}{\lambda(x)^j} = O(\lambda(x)^{-N}).$$

The theorem is formulated through a structure or predicate such as `HasAsymptoticExpansion`, allowing generic lemmas for truncation, addition, multiplication, and logarithms.

18.9 From mass to logarithm

Since $c_0 = 1$, the normalized mass is eventually close to one. A formal logarithm of the coefficient series gives

$$\log \mathcal{M}(x) \sim \sum_{j \geq 1} \frac{d_j}{\lambda(x)^j}.$$

The analytic proof uses a Taylor theorem for $\log(1 + u)$ with a uniform remainder on a small neighborhood. Positivity of the exact mass, inherited from the original integral representation, ensures the logarithm is well-defined.

18.10 Transfer to $F(x)$

`FabiusFullAsymptoticExpansion.lean` inserts the logarithmic mass expansion into the exact saddle reduction, adds the periodic and explicit main terms, and absorbs the exponentially small forward tail. The result is an all-order logarithmic expansion for $F(x)$ at zero.

The primary remainder is $O(\lambda^{-N})$. A weaker but simpler corollary uses $\lambda(x) \asymp -\log x$ to obtain

$$O((-\log x)^{-N}).$$

The line breaks in this display correspond to ordinary $-\log x$; they are separated here only to emphasize that positivity of the logarithmic scale is part of the eventual hypothesis.

18.11 First-order and Wikipedia-facing corollaries

Modules with names such as `FabiusWikipediaMain.lean` and `FabiusWikipediaExpansion.lean` package concise formulas suitable for comparison with public summaries. They separate what is correct at leading order from what is false at the claimed next order. The obstruction module demonstrates that a nonconstant dyadic-periodic correction cannot be absorbed into a vanishing error.

18.12 The proof-audit role of the asymptotic aggregator

`PaperFabiAsymptotic.lean` is not just a theorem list. Its header records which informal claims are supported, which need corrected domains or periodic terms, and which are disproved. The formal development therefore functions as an audit trail:

- coarse log-squared behavior is validated;
- the exact delay relation is restricted to its true domain;
- a claimed overly small residual is refuted;
- the periodic Lambert-phase formula is proved independently through transforms and saddles;
and
- arbitrary-order corrections are established at the exact phase.

Legendre expansions and polynomial approximation

19.1 Why Legendre polynomials are a natural basis

The up-function is supported on $[-1, 1]$, is even, and is infinitely differentiable. Legendre polynomials are orthogonal on exactly this interval with respect to Lebesgue measure, so they provide a canonical spectral expansion:

$$\text{up}(x) = \sum_{n \geq 0} a_n P_n(x).$$

Evenness forces every odd coefficient to vanish. The repository does not stop at a formal L^2 expansion: it evaluates the even coefficients exactly through moments, proves absolute and uniform convergence, translates the series back to the global Fabius function, and proves least-squares optimality of finite truncations.

19.2 Building the polynomial basis

`LegendrePolynomial.lean` develops the required Legendre facts in the normalization used by the project. The polynomials are defined through Rodrigues' formula or an equivalent derivative representation:

$$P_n(x) = \frac{1}{2^n n!} \frac{d^n}{dx^n} (x^2 - 1)^n.$$

The module proves degree, parity, endpoint values, differential equation, and derivative bounds.

A bespoke development is useful because later coefficient formulas require explicit monomial coefficients. A black-box orthogonal-polynomial theorem would provide orthogonality but not necessarily the exact finite sum compatible with the moment sequence.

19.3 Sturm–Liouville orthogonality

The Legendre differential equation is

$$((1 - x^2)P'_n(x))' + n(n + 1)P_n(x) = 0.$$

Multiplying the equation for P_n by P_m , the equation for P_m by P_n , subtracting, and integrating gives

$$(n(n+1) - m(m+1)) \int_{-1}^1 P_n(x)P_m(x) \, dx = 0.$$

Boundary terms vanish because of the factor $1 - x^2$. For $m \neq n$, the eigenvalues differ, proving orthogonality.

`FabiusLegendreSeries.lean` carries out this argument using interval integration by parts. The proof explicitly verifies differentiability of the polynomial expressions and endpoint vanishing. The eigenvalue inequality is discharged in natural arithmetic and cast to reals only at the final multiplication step.

19.4 Norms and pointwise bounds

The exact norm is

$$\int_{-1}^1 P_n(x)^2 \, dx = \frac{2}{2n+1}.$$

The source derives it from Rodrigues' formula and repeated integration by parts, or from a recurrence plus normalization depending on the helper theorem used. This norm fixes the Fourier–Legendre coefficient:

$$a_n = \frac{2n+1}{2} \int_{-1}^1 \text{up}(x)P_n(x) \, dx.$$

A bound $|P_n(x)| \leq 1$ on $[-1, 1]$ is proved from the differential equation and an extremum argument. This simple uniform bound is the last ingredient needed to convert absolute convergence of coefficients into uniform convergence of the function series.

19.5 Vanishing of odd coefficients

The parity theorem states

$$P_n(-x) = (-1)^n P_n(x).$$

Since `up` is even, the integrand $\text{up}(x)P_n(x)$ is odd when n is odd. A symmetric-interval integral of an odd integrable function is zero. In Lean, the proof either invokes a generic odd-integral lemma or performs the substitution $x \mapsto -x$ and adds the two integral identities.

The resulting series is indexed naturally by P_{2n} . This is not only cosmetic: the exact coefficient formula then uses the even moment M_{2k} , for which the arithmetic normalization and denominator theory are available.

19.6 Exact coefficient evaluation

`FabiusLegendreCoefficients.lean` expands P_{2n} in monomials:

$$P_{2n}(x) = \sum_{k=0}^n c_{n,k} x^{2k}.$$

Finite linearity of the integral gives

$$a_{2n} = \frac{4n+1}{2} \sum_{k=0}^n c_{n,k} M_{2k}.$$

Every M_{2k} is the exact rational moment value, so a_{2n} is rational. The source also substitutes formulas expressing moments through inverse-power Fabius values, producing a coefficient formula entirely in terms of the exact dyadic table.

The proof is layered:

1. establish the explicit polynomial coefficient theorem;
2. turn polynomial evaluation into a finite sum;
3. commute the finite sum with the integral;
4. replace each monomial integral by the analytic moment;
5. identify the moment with the exact rational sequence; and
6. simplify casts and normalizations.

At no point is an infinite series exchanged with the integral during coefficient evaluation.

19.7 Generic convergence infrastructure

`LegendreSeriesConvergence.lean` develops a theorem for smooth functions whose repeated derivatives satisfy suitable endpoint or norm bounds. Integration by parts transfers derivatives from the function to the Legendre eigenstructure, yielding decay of coefficients faster than any fixed power of n .

For the Fabius up-function, compact support and smoothness supply these hypotheses to every order. Consequently the coefficient sequence is absolutely summable. Since $|P_n(x)| \leq 1$, the Weierstrass M-test gives uniform convergence on $[-1, 1]$.

The generic theorem is separated from Fabius-specific coefficient arithmetic. This permits future reuse for other compactly supported smooth functions and keeps the main convergence proof independent of exact moment formulas.

19.8 Identifying the uniform limit

Orthogonality first shows that the partial sums are the L^2 projections of up . Absolute/uniform convergence produces a continuous limit S . To prove $S = up$, the source may use one of two equivalent routes:

- completeness of Legendre polynomials in $L^2[-1, 1]$, followed by equality of continuous representatives; or
- a generic convergence theorem already identifying the pointwise limit for sufficiently smooth functions.

The final theorem states a `HasSum` at each point, then upgrades to uniform convergence of the function series.

Endpoints are included. This matters because pointwise convergence theorems for orthogonal series often state only interior convergence. The absolute coefficient bound and $|P_n(\pm 1)| = 1$ handle endpoints directly.

19.9 Translated series for the global Fabius function

`FabiusTranslatedLegendreSeries.lean` transports the up-function expansion to an interval such as $[0, 2]$ where a translate agrees with the signed global Fabius function. The transformation replaces $P_{2n}(x)$ by $P_{2n}(x - 1)$ or the corresponding affine normalization.

The module also expands translated Legendre polynomials into ordinary monomials. This produces an explicit power-polynomial series in the translated coordinate, with exact rational coefficients. Uniform convergence follows from the original series under composition with the affine map.

19.10 Least-squares optimality

`FabiusLegendreLeastSquares.lean` proves that the partial Legendre sum is the unique least-squares approximant among polynomials of the corresponding degree. Let

$$S_N = \sum_{n=0}^N a_n P_n.$$

For any polynomial p in the same finite-dimensional span,

$$\|\text{up} - p\|_2^2 = \|\text{up} - S_N\|_2^2 + \|S_N - p\|_2^2.$$

This Pythagorean identity follows because $\text{up} - S_N$ is orthogonal to every basis polynomial in the span.

The formal proof represents the finite span either as a submodule of L^2 -type functions or directly through polynomial coefficients. It proves orthogonality term by term and then expands the square of the norm. Uniqueness follows because equality of errors forces $\|S_N - p\|_2 = 0$, hence equality almost everywhere; polynomial continuity upgrades this to pointwise polynomial equality.

19.11 Gaunt coefficients and finite Rvachev Gram sums

The current Legendre layer continues beyond one-function coefficient extraction. `LegendreGaunt.lean` defines an executable rational triple product

$$G_{ijk}^{\mathbb{Q}} = \sum_{a=0}^i \sum_{b=0}^j \sum_{c=0}^k [X^a] P_i [X^b] P_j [X^c] P_k \mu_{a+b+c}^L,$$

where $\mu_n^L = 2/(n+1)$ for even n and zero for odd n . Scalar extension identifies it with

$$G_{ijk} = \int_{-1}^1 P_i(x) P_j(x) P_k(x) \, dx, \quad P_i P_j = \sum_{k=0}^{i+j} \frac{2k+1}{2} G_{ijk}^{\mathbb{Q}} P_k.$$

The first module proves symmetry and the parity/triangle support zeros. `LegendreGauntClosedForm.lean` sharpens support to an equivalence: admissibility means even total degree together with

the three weak triangle inequalities. Equivalently $(i, j, k) = (b + c, a + c, a + b)$. On this support the squared zero-row datum has the closed form

$$W_{b+c, a+c, a+b}^2 = \frac{\binom{2a}{a} \binom{2b}{b} \binom{2c}{c}}{(2(a + b + c) + 1) \binom{2(a+b+c)}{a+b+c}},$$

and

$$G_{ijk}^{\mathbb{Q}} = 2W_{ijk}^2, \quad \frac{2k+1}{2}G_{ijk}^{\mathbb{Q}} = (2k+1)W_{ijk}^2.$$

Thus the Gaunt coefficients and product-linearization coefficients are nonnegative everywhere and positive exactly on the admissible support.

`FabiusLegendreGaunt.lean` and its closed-form companion combine this linearization with the exact Rvachev Legendre coefficients. Products of basis polynomials can therefore be integrated against the up-law by a finite rational sum, yielding exact Gram entries and matrix formulas. The boundary is again clear: the formalization concerns the squared zero-row integer-index datum and does not claim a signed Wigner symbol, magnetic indices, recoupling theory, or asymptotics of general $3j$, $6j$, or $9j$ symbols.

19.12 Computational significance

The exact coefficient theorem gives a certified way to produce rational polynomial approximants. The least-squares theorem certifies optimality in L^2 , while uniform convergence certifies pointwise approximation. This is a rare combination:

- exact arithmetic coefficients;
- kernel-checked analytic convergence; and
- an optimality theorem for every finite truncation.

The evaluator and moment recurrences can be used to compute coefficients without numerical integration.

The k-fold Thue–Morse draft audit

20.1 Purpose of the audit

`PaperKFoldThueMorse.lean` collects a separate line of work concerning k-fold Thue–Morse constructions and a related informal draft. Its role is both constructive and diagnostic: it proves exact identities and corrected approximation statements, but it also formalizes counterexamples to several literal claims in the draft.

This part of the repository illustrates a mature use of formalization. A failed proof is not treated as an obstacle to be papered over; the surrounding hypotheses are tested, false versions are refuted, and repaired versions are isolated.

20.2 Exact finite identities

The Thue–Morse modules prove prefix-sum cancellation, long zero runs of finite differences, convolution formulas, and generating-series identities. Repeated finite difference annihilates polynomials below a threshold degree, which explains the high-order cancellation of the sign sequence.

The formal proofs use induction on binary depth and repeated even/odd splitting. Convolution associativity is kept finite, so all sums are over `Finset.range` and can be reindexed explicitly. Generating polynomials turn convolution into multiplication, allowing coefficient comparison to replace nested sum manipulations.

20.3 Polygonal interpolation and corrected approximation

A piecewise-linear or polygonal interpolant converts finite coefficient data into a function. The draft’s intended limit is recovered only after choosing a normalization and pointwise representative compatible with the mesh. The corrected theorem makes these choices explicit and proves convergence on the appropriate domain.

As in the step-approximation project, knot values require special treatment. Lean forces a decision about whether adjacent pieces are left-closed, right-closed, or averaged at the boundary. The corrected interpolation chooses a consistent convention and proves continuity directly.

20.4 Formal counterexamples

`DraftCounterexamples.lean` records counterexamples to claims about normalization, local and global errors, an unbounded maximum, and a missing linear term. A typical counterexample evaluates the proposed formula at the smallest nontrivial depth or at a symmetry point, where the exact recurrence reduces to rational arithmetic. `norm_num` or a short ring proof then contradicts the claimed equality or inequality.

Other false claims are asymptotic. The code constructs a sequence of inputs for which the normalized residual has a nonzero or divergent limit, using the already-established coarse Fabius asymptotic. The counterexample theorem is stated independently of prose, so its logical content is unambiguous.

20.5 Repairing the lower Lambert branch

The draft uses a lower branch of a Lambert-type inverse. The repository develops this branch with a correct domain, monotonicity, and inversion theorem. Endpoint behavior and branch separation are proved explicitly; no appeal is made to an ambiguous multivalued inverse.

These lemmas then support the corrected asymptotic comparisons in both the k-fold project and the principal Fabius saddle analysis.

20.6 Decay comparisons

`FabiusFlatness.lean` and `FabiusDecayComparison.lean` convert the log-squared law into two useful endpoint comparisons. For every $m > 0$,

$$F(x) = o(x^m) \quad (x \downarrow 0),$$

so the function is smaller than every power. On the other hand, for every $c > 0$,

$$e^{-c/x} = o(F(x)),$$

so it is much larger than an $e^{-c/x}$ flat function.

Taking logarithms reduces these statements to comparing $(\log(1/x))^2$ with $\log(1/x)$ and with $1/x$. The code packages positivity and eventual monotonicity so the exponential comparison can use standard filter lemmas.

Formal combinatorics and power-series infrastructure

21.1 Why a Fabius project owns so much combinatorial algebra

Exact Fabius arithmetic and saddle asymptotics repeatedly ask for the same operations: convert powers to falling-factorial bases, pass between moments and cumulants, exponentiate and logarithmize formal series, compose series, invert triangular transforms, and identify coefficients after a geometric specialization. Early versions could prove each identity close to its first use. The current tree instead factors the algebra into reusable modules. This both shortens analytic proofs and makes ring assumptions explicit.

The first large facade stratum contains complete homogeneous symmetric polynomials, elementary symmetric functions, Newton power sums, Bell polynomials, both kinds of Stirling numbers, Eulerian and Lah numbers, Bernoulli, Euler, Genocchi, Nørlund, Touchard, Catalan, Narayana, and associated or r -Stirling families. Their purpose is not encyclopedic accumulation. They form a conversion network among four common coordinate systems:

1. ordinary monomials x^n ;
2. falling or rising factorials;
3. exponential generating functions; and
4. moment/cumulant or elementary/complete symmetric coordinates.

21.2 Triangular transforms as the organizing abstraction

A finite triangular transform has the schematic form

$$b_n = \sum_{k \leq n} T(n, k) a_k.$$

The files `FiniteTriangularTransform.lean`, `TriangularKernelInverse.lean`, `BinomialInversion.lean`, and the Stirling reverse-column and reverse-row modules prove inversion at the kernel level before any Fabius specialization. This is important in Lean: a theorem about a triangular matrix over a ring can later be transported through a ring homomorphism, whereas a proof written directly over $\mathbb{R}$ tends to bury the finite algebra under coercions.

The normal pattern is:

```
-- algebraic layer
show transformedCoeffs T (transformedCoeffs Tinv a) = a

-- specialization layer
simp [specificKernel] using generic_inverse_theorem ...
```

The specialized theorem then reads like a classical binomial, Stirling, or Bell identity, but the proof owes its stability to the generic kernel statement.

21.3 Bell polynomials and cumulants

The modules `PartialBellPolynomials.lean`, `BellGeneratingFunctions.lean`, `BellComposition.lean`, `CumulantBellFormula.lean`, and `MomentCumulantAlgebra.lean` package the finite coefficient formulas behind

$$M(t) = \exp K(t), \quad K(t) = \log M(t).$$

For probability laws, this converts cumulants of independent weighted summands into moments. For the saddle problem, the same algebra converts the Taylor jet of the local logarithmic phase into the Taylor coefficients of its exponential. The repository therefore avoids maintaining two nearly identical Bell engines for probability and asymptotics.

The centered-moment modules exploit symmetry before expansion. Odd centered moments and odd cumulants vanish, so many coefficient sums are empty by parity. In Lean, proving that parity statement upstream is far more effective than asking `ring` to cancel a large expanded expression at every order.

21.4 Endpoint-transfer polynomials and finite expectations

The current tree isolates a further piece of endpoint asymptotic algebra that older versions left embedded in a special-purpose calculation. `EndpointTransferPolynomials.lean` defines universal polynomials $P_r \in \mathbb{Q}[X]$ by the formal identity

$$\exp \left(- \sum_{j \geq 2} \frac{X^j t^{j-1}}{j} \right) = \sum_{r \geq 0} P_r(X) t^r.$$

The recursive coefficient theorem, the weighted-partition formula, and the formal exponential identity are proved separately. Evaluation and mapping lemmas then transport the construction to every commutative rational algebra. This is deliberately a formal statement: it has no analytic radius, branch choice, or remainder estimate.

`PolynomialExpectationCumulant.lean` supplies the matching finite integration interface. Integrating a polynomial evaluation is reduced first to a finite sum of coefficients times raw moments; those moments can then be rewritten by the complete Bell transform of formal cumulants. When the zeroth moment is normalized to one—in particular for a probability measure—the constant-term mass correction disappears. Applying this bridge to P_r provides exactly the algebra needed by an endpoint-transfer calculation, while leaving Taylor control, contour estimates, and analytic tails to the later asymptotic modules.

Architectural point

Formal endpoint transfer is a composition of two reusable finite APIs:

$$\begin{aligned} \text{universal polynomial coefficient} &\longrightarrow \text{finite moment functional} \\ &\longrightarrow \text{Bell-cumulant coordinates} \end{aligned}$$

None of these arrows by itself is an asymptotic theorem. The separation is one of the clearest examples of the repository’s formal/analytic boundary.

21.5 Formal logarithms, exponentials, and inversion

`ExpLog.lean`, `ExpAddLog.lean`, `SquareRootSeries.lean`, `LagrangeInversion.lean`, `ImplicitPowerSeries.lean`, and `QuadraticCompositionalInverse.lean` provide the formal mechanisms later used in Lambert expansions, inverse germs, and endpoint corrections. The essential distinction is between:

- a coefficient identity in a formal power-series ring, where convergence is irrelevant; and
- an analytic expansion, where a radius, a remainder, and a domain are required.

The project normally proves the coefficient theorem first, then supplies a separate analytic transfer module. This split prevents a formal calculation from being silently advertised as an asymptotic theorem.

21.6 Symmetric functions and geometric alphabets

The complete homogeneous polynomial $h_n(x_1, \dots, x_m)$ is especially natural on a geometric alphabet $(1, q, \dots, q^{m-1})$. The modules `GeometricCompleteHomogeneous.lean` and `GeometricLagrangeCompleteHomogeneous.lean` identify these values with Gaussian-binomial expressions and interpolation residuals. The result is a three-way bridge:

$$\text{symmetric functions} \longleftrightarrow \text{geometric interpolation} \longleftrightarrow \text{q-binomial coefficients}.$$

This bridge later explains why q-binomial weights appear in dyadic extrapolation and in the Fabius–Thue–Morse formulas, rather than treating the appearance of $q = 1/2$ as a coincidence.

The finite generating-series layer is also explicit. For a finite alphabet $a = (a_i)$, `CompleteHomogeneousGenerating.lean` packages $\sum_{n \geq 0} h_n(a) t^n$ as a formal power series and proves the adjoining-variable recurrence. Clearing the linear factors gives, in a commutative ring,

$$\left(\prod_i (1 - a_i t) \right) \sum_{n \geq 0} h_n(a) t^n = 1.$$

No values must be distinct and no analytic convergence is asserted. `CompleteHomogeneousBell.lean` gives the complementary division-free Bell identity. With $p_k = \sum_i a_i^k$, $\kappa_0 = 0$, and $\kappa_{k+1} = k! p_{k+1}$, it proves

$$\text{Bell}_n(\kappa) = n! h_n(a).$$

On the geometric alphabet this specializes over every commutative semiring to

$$h_n(1, q, \dots, q^p) = \begin{bmatrix} n+p \\ p \end{bmatrix}_q, \quad h_n(c, cq, \dots, cq^p) = c^n \begin{bmatrix} n+p \\ p \end{bmatrix}_q.$$

The residual-moment theorem is correspondingly sharp. If weights w_k are exact on the scaled nodes cq^k through degree p , then for $r \geq 0$ and $m > 0$,

$$\sum_{k=0}^p w_k (cq^k)^{p+1+r} = c^{p+1+r} (-1)^p q^{\binom{p+1}{2}} \begin{bmatrix} p+r \\ p \end{bmatrix}_q,$$

$$\sum_{k=0}^p w_k (cq^k)^m = c^m (-1)^p q^{\binom{p+1}{2}} \begin{bmatrix} m-1 \\ p \end{bmatrix}_q.$$

Stating exactness directly on the scaled nodes is stronger than proving an unscaled result and cancelling powers of c : it remains meaningful when c is zero or a zero divisor.

21.7 What to import

For a new finite coefficient identity, begin as high in the algebraic stack as possible. Typical starting points are:

- `CoefficientRules.lean` and `FiniteTriangularTransform.lean` for coefficient extraction;
- `PartialBellPolynomials.lean` and `MomentCumulantAlgebra.lean` for exponential composition;
- `StirlingBasisChange.lean` for monomial/factorial conversion;
- `SymmetricFunctionTransform.lean` for elementary/complete/power-sum changes; and
- `LagrangeInversion.lean` or `ImplicitPowerSeries.lean` for formal inverses.

Only after the algebraic theorem is stable should one import analytic convergence or a Fabius-specific bridge.

Weighted-uniform laws, transforms, and geometric filters

22.1 The generic random-series layer

The probability construction of the Fabius function is a special case of an absolutely summable weighted series of independent uniform coordinates. The current modules `WeightedUniformSeries.lean`, `WeightedUniformDistribution.lean`, and `WeightedUniformSupport.lean` separate three questions:

1. does the random series converge;
2. what probability law and affine recursion does it satisfy; and
3. what is the exact support or range of the deterministic series map?

The separation matters for signed weights. A distributional reflection or independent-copy equation can hold without the support being the unit interval. The support theorem is therefore stated for the actual weight hypotheses, not smuggled into the notation.

For geometric weights $(1 - q)q^n$, the files `GeometricUniformLaw.lean`, `GeometricUniformMultisection.lean`, `GeometricUniformUniqueness.lean`, and `GeometricUniformCDF.lean` provide a named law, multisection identities, uniqueness for the affine independent-copy equation, and a CDF/density interface. At $q = 1/2$ the CDF and its affine density recover the bounded Fabius and Rvachev objects. The generic theorem is stronger than this specialization because it isolates which parts survive for negative q , for $q = 0$, or without an a priori support assumption.

22.2 Affine independent-copy laws and the continuous-CDF bridge

`AffineIndependentCopy.lean` packages the generic transformation before any geometric or Fabius specialization. Given a digit law ρ , a measurable digit map d , a real scale q , and a candidate law μ on a second-countable real inner-product space, it forms

$$\mathcal{T}_{\rho,d,q}(\mu) = (d_*\rho) * ((q \cdot)_*\mu).$$

For s-finite measures this convolution is proved equal to the direct product pushforward under $(u, x) \mapsto d(u) + qx$. Characteristic functions therefore factor as

$$\widehat{\mathcal{T}_{\rho,d,q}(\mu)}(t) = \widehat{d_*\rho}(t) \widehat{\mu}(qt).$$

A fixed point can be iterated finitely:

$$\widehat{\mu}(t) = \prod_{k=0}^{n-1} \widehat{d_*\rho}(q^k t) \widehat{\mu}(q^n t).$$

This is the common engine behind product formulas and uniqueness proofs. The geometric-uniform modules provide the contraction and tail arguments needed to pass from the finite identity to the infinite law.

`ContinuousCDF.lean` closes three measure-theoretic steps that are easy to blur in prose. Null singleton masses imply continuity of a real CDF; reflection invariance under $y \mapsto a - y$ gives $H(a - x) = 1 - H(x)$; and an everywhere pointwise derivative $H'(x) = h(x)$ identifies the law exactly with Lebesgue measure having density $\text{ofReal} \circ h$. The final identification does not assume in advance that h is continuous or that the law is absolutely continuous: monotonicity of the CDF supplies the needed nonnegativity and local integrability inside the proof.

22.3 CDFs, quantiles, and Cauchy/Stieltjes transforms

`ContinuousCDF.lean` packages regularity of distribution functions under the relevant atomlessness hypotheses. The later `QuantileTransport.lean` module gives a support-free inverse-CDF transport principle from an almost-everywhere sublevel identity. The Fabius specialization transports uniform measure through the clamped inverse and provides Banach-valued quantile substitution.

The transform side is intentionally layered:

- `MeasureCauchyTransform.lean` defines and controls a Cauchy transform for a measure;
- `MeasureCauchyMomentLaurent.lean` expands it at infinity in terms of moments;
- `StieltjesCauchyTransform.lean` and `StieltjesHerglotz.lean` record real-line and half-plane structure;
- `StieltjesInversion.lean`, `PoissonApproximateIdentity.lean`, and `StieltjesPeron.lean` recover mass from boundary data; and
- `GeometricUniformCauchy.lean`, `CauchyCDF.lean`, and `CauchyRenormalization.lean` return to the geometric law.

Thus a moment computation, a probability-law identity, and a transform fixed point can be proved independently and then connected.

22.4 Finite filters on arbitrary and geometric nodes

The filtering library begins with finite weights w_j and nodes r_j . If

$$\sum_j w_j r_j^k = \delta_{k0} \quad (0 \leq k < m),$$

then the affine combination $\sum_j w_j f(r_j x)$ reproduces the constant term and annihilates the next $m - 1$ Taylor modes. The modules `FinitePolynomialFilterExactness.lean`, `FinitePowerSeriesFilter.lean`, and `AnalyticSeriesFilter.lean` formalize this at polynomial, formal-series, and convergent-series levels. `FiniteAffineCombinationBounds.lean` keeps the norm estimate separate from exactness.

On geometric nodes, Lagrange weights have explicit q -binomial forms. `GeometricLagrange.lean`, `GeometricLagrangeWeights.lean`, `GeometricLagrangeQBinomial.lean`, and `GeometricLagrangeQMoments.lean` prove the row identities. `GeometricPowerSeriesFilter.lean` diagonalizes the action on coefficients, and `GeometricResidualMoments.lean` computes the residual multipliers. The same machinery appears under the numerical-analysis name Richardson extrapolation in `GeometricRichardson.lean`.

Proof idea

A geometric extrapolation proof should not expand a Vandermonde determinant at each use. Prove the interpolation row once, evaluate its moments, and let the formal-series filter theorem act coefficientwise. Analytic error bounds are a later corollary of summability and the residual multiplier estimate.

22.5 Gram matrices and orthogonal reconstruction

A finite moment functional L determines a Gram matrix $G_{ij} = L(x^{i+j})$. The modules `FinitePolynomialFunctional.lean`, `PolynomialMomentFunctional.lean`, `FiniteMomentGram.lean`, and `PolynomialMomentGramDeterminant.lean` package this construction. `FiniteMomentJacobi.lean` and `GramStieltjes.lean` connect the moment matrix to Jacobi data and continued-fraction/Stieltjes information. Naturality lemmas ensure that exact rational data can be scalar-extended to $\mathbb{R}$ or $\mathbb{C}$ without reproving the determinant identities.

For the Fabius/Rvachev law this infrastructure supports rational Gram values, orthogonal polynomial construction, parity, three-term recurrences, Jacobi matrices, and exact Legendre/Hankel determinants. It also supports the polynomial-synthesis modules discussed in Chapter 24.

22.6 Activation Taylor jets, square-summability, and effective dimension

The hyperbolic-activation family is a separate analytic specialization of the weighted-series engine. It begins with the totalized quotient

$$h(x) = \begin{cases} \tanh(x)/x, & x \neq 0, \\ 1, & x = 0, \end{cases} \quad p(x) = 1 - h(x).$$

`HyperbolicActivation.lean` proves that the totalization is canonical: h is the divided slope of $\tanh$ at zero. The defect p is continuous, even, nonnegative, strictly positive exactly away from zero, and satisfies the global sharp-enough domination

$$0 \leq p(x) < 1, \quad p(x) \leq \min\left(1, \frac{x^2}{3}\right).$$

Consequently

$$p(at) \leq \frac{t^2}{3}a^2.$$

For any index type and any square-summable real family w_i , the series $D_w(t) = \sum_i p(w_i t)$ is summable and obeys

$$D_w(t) \leq \frac{t^2}{3} \sum_i w_i^2.$$

No separate countability, positivity, or ℓ^1 assumption is imposed; summability itself carries countable support, and zero or signed weights are allowed.

The local coefficient is proved independently of the finite Taylor theorem:

$$\lim_{t \rightarrow 0, t \neq 0} \frac{p(at)}{t^2} = \frac{a^2}{3}.$$

The domination above then permits a Tannery/dominated-sum argument,

$$\lim_{t \rightarrow 0, t \neq 0} \frac{D_w(t)}{t^2} = \frac{1}{3} \sum_i w_i^2.$$

The separate module `ActivationTaylor.lean` proves the stronger finite jet

$$\begin{aligned} \tanh x &= x - \frac{x^3}{3} + \frac{2x^5}{15} - \frac{17x^7}{315} + \frac{62x^9}{2835} + O(|x|^{11}), \\ p(x) &= \frac{x^2}{3} - \frac{2x^4}{15} + \frac{17x^6}{315} - \frac{62x^8}{2835} + O(x^{10}). \end{aligned}$$

The proof differentiates $\tanh x \cosh x = \sinh x$, rather than repeatedly expanding a quotient; the totalized value then patches the punctured-neighborhood calculation at zero.

For geometric weights, the normalized sum of squares gives the coefficient

$$\frac{1 - q}{3(1 + q)} \quad (|q| < 1),$$

which equals $1/9$ at $q = 1/2$. The family illustrates a recurring design principle: weighted summation, domination, and limit exchange are reusable, but each nonlinear activation must supply its own local coefficient and global majorant.

The expanded Thue–Morse formalization

23.1 From one sign sequence to a digital-analysis library

The signed Thue–Morse sequence

$$\varepsilon_n = (-1)^{s_2(n)}$$

first enters the Fabius project through the signed global extension and exact dyadic values. The present corpus treats it as a central digital object. The 108-module facade stratum from `ThueMorse` `eBasicLemmas.lean` through `RieszSharpness.lean` develops mutually reinforcing algebraic, arithmetic, combinatorial, automatic, Fourier, Mellin, and asymptotic views.

The basic recursion

$$\varepsilon_{2n} = \varepsilon_n, \quad \varepsilon_{2n+1} = -\varepsilon_n$$

is encoded early, but downstream modules prefer more structural forms: Boolean-cube products, parity characters, digit-floor formulas, Rademacher functions, and products indexed by bit positions. These forms let finite sums be attacked by ring factorization, character orthogonality, or induction on the binary word length, whichever best matches the theorem.

23.2 The all-rings product identity and Prouhet cancellation

The finite identity

$$\prod_{j < m} (1 - X^{2^j}) = \sum_{n < 2^m} \varepsilon_n X^n$$

is the main algebraic engine. It implies vanishing of the first m power moments after differentiation at $X = 1$, giving the Prouhet partition of the binary block. The modules `ThueMorseBooleanCube.lean`, `BaseDigitProuhet.lean`, `GeneralizedProuhetBlock.lean`, and `ProuhetBlockZeroOrder.lean` generalize the cancellation and track the first nonzero order.

Recent closing modules `MixedDifferenceForwardDiff.lean`, `GeometricProuhetBlock.lean`, and `GeneralLinearThueMorseSum.lean` connect the Boolean-cube sum to mixed and forward differences and to general linear forms. This is the appropriate API for repeated summation and polynomial-diagonal questions: the result is a finite-difference theorem first, a Fabius theorem only after a specialization.

23.3 Valuations and Pascal multiplicities

Binary digit sums also control two-adic valuations. The modules `ThueMorseValuation.lean`, `DyadicZeroMultiplicity.lean`, `PascalDyadicMultiplicity.lean`, `CentralBinomialValuation.lean`, and `PrimePowerBinomialValuation.lean` formalize carry-counting and prime-power row identities. The prime-power theorem keeps its strict interior index hypothesis: an endpoint outside the relevant binomial row evaluates to zero and has a different valuation status. The dyadic wrappers then connect these valuations to zero multiplicities in products and denominator exponents in exact Fabius values.

23.4 Automatic and algebraic structure

`ThueMorseAperiodicity.lean`, `ThueMorseOverlapFree.lean`, and `WordComplexity.lean` cover combinatorics on words. The recurrence, Mahler, and Artin–Schreier modules encode the generating series in characteristic zero and characteristic two. `ThueMorseLucasSupport.lean` and `ThueMorseBooleanMobius.lean` connect digit restrictions to Lucas support and Boolean Möbius inversion. These are not decorative side results: they provide exact finite support descriptions used by sparse sums, submask-density asymptotics, and q-binomial specializations.

23.5 Fourier, aliasing, and cyclic characters

The Fourier family contains the finite exponential product, sine product, inversion, Parseval, half-integer odd DFT, cyclic alias identities, and explicit alias error bounds. The sequence’s dyadic recursion turns Fourier frequencies into a doubling orbit; character-sum modules isolate rational frequencies and residue classes. The bridge `CenteredRvachevThueMorseFourier.lean` later identifies the same products inside the Rvachev Fourier transform.

A useful reading path is:

1. `ThueMorseComplexExponential.lean`;
2. `ThueMorseFourier.lean` and `ThueMorseSineProduct.lean`;
3. `HalfIntegerOddDFT.lean` and `HalfIntegerAliasIdentity.lean`;
4. `AliasDyadicBridge.lean`; and
5. `ThueMorseParseval.lean`.

The order exposes the finite product before any analytic limiting argument.

23.6 Centered sine and sinh products

The 3 September module `ThueMorseSinhProduct.lean` adds the real hyperbolic twin of the finite sine product. With $c_m = (2^m - 1)/2$, it proves

$$\sum_{n < 2^m} \varepsilon_n \exp((n - c_m)t) = (-2)^m \prod_{j < m} \sinh(2^j t/2).$$

The proof factors $1 - e^u = -2e^{u/2} \sinh(u/2)$ in every Boolean-cube factor; the centering cancels the accumulated exponential prefactor. The principal public declaration is `sum_thueMorseSign_e`

`xp_centered_eq_sinh_prod`. This form is tailored to Laplace-side moment calculations because it is real and symmetric.

23.7 Infinite products, Mellin transforms, and natural boundaries

The finite product feeds an extensive analytic branch: `ThueMorseInfiniteProduct.lean`, `ThueMorseMellin.lean`, `ThueMorseWoodsRobbins.lean`, `ThueMorseMasterProduct.lean`, `ThueMorseEntireContinuation.lean`, and `ThueMorseGammaTower.lean`. Frullani integrals and reciprocal Gamma jets supply the analytic regularization. The disc-series and `ThueMorseNaturalBoundary.lean` modules separate the domain of an ordinary power series from the entire continuation of a different normalized product; this prevents a continuation theorem from being misread as convergence of the original series across its natural boundary.

23.8 Newman, Gelfond, Riesz, and Hankel directions

The Newman modules analyze residue-class and quantitative signed sums. The Gelfond modules prove the classical exponent bound and a sharpness statement, while `PeriodicOrbitGelfondBound.lean` and `GelfondLogisticBound.lean` give orbit-based refinements. `PeriodDoublingHankel.lean` and `ThueMorseHankel.lean` treat Hankel structure, and the Riesz modules connect lacunary products to integrals and sharpness. These families meet the Rvachev Fourier-decay program through the same doubling dynamics, but they remain independently usable digital-analysis theorems.

23.9 Bridges back to Fabius and q-series

The final facade leaves include `FabiusDyadicGaussianForm.lean`, `FabiusGeneralLinearDenominator.lean`, `ThueMorseQPochhammer.lean`, `ThueMorseQPochhammerInfinite.lean`, `ThueMorseEulerFunction.lean`, and `ThueMorsePentagonal.lean`. Together with `BitPositionQBinoomial.lean`, they show how the finite digital product, Gaussian coefficients, q-Pochhammer dissections, and exact Fabius denominators are different projections of the same binary factorization.

Generalized Rvachev products, Fourier decay, and synthesis

24.1 Beyond the classical dyadic sinc product

The classical Fourier transform of the up-function is an infinite dyadic sinc product. The generalized layer replaces the constant exponent sequence by an admissible natural weight $a(h)$ and studies products schematically of the form

$$\Phi_a(z) = \prod_{h \geq 0} \text{sinc}(2^{-h}z)^{a(h)}.$$

`GeometricScaleProducts.lean`, `GeneralizedRvachevProduct.lean`, `WeightLinearityProducts.lean`, and `GeneralizedCanonicalForm.lean` prove convergence, finite natural-linear laws, scaling/refinement, and canonical-product representations. Natural coefficients are a deliberate boundary: signed exponent identities require additional cancellation and are not silently inferred from the natural-weight API.

24.2 Zeros, analytic order, and identifiability

The zero set alone does not determine multiplicity. The modules `WeightedScaleMultiplicity.lean`, `BaseDigitMultiplicity.lean`, `DyadicGammaOrder.lean`, `IntegerZeroAnalyticOrder.lean`, and `GeneralizedZeroDivisor.lean` compute exact orders at structured points. `GeneralizedRvachevIdentifiability.lean` then proves that the exponent sequence can be recovered from the analytic orders at dyadic points and that, under the stated summability hypotheses, equality of the generalized entire products is equivalent to equality of exponent sequences.

The key logical distinction is:

$$\text{bare zero set} \neq \text{zero divisor with multiplicities}.$$

Multiplying every exponent by two preserves the zero set but doubles every order. The Lean theorem therefore uses analytic order data, not merely pointwise vanishing.

24.3 Lobe geometry and local factorization

The files from `LobeSignLaw.lean` through `OnePeakPerLobe.lean` formalize signs, logarithmic factorization, derivative formulas, concavity, and uniqueness of peaks on central and side lobes. Local zero factorization and a leading-jet theorem make the order statement analytic rather than combinatorial. `ThueMorseLobeSign.lean` and `LobeSignSelfSimilarity.lean` add digital sign laws and renormalization. These results support both plotting intuition and rigorous envelope estimates, but the proofs are stated in terms of factor signs, logarithmic derivatives, and concavity, not sampled graphs.

24.4 Fourier decay as a dynamical problem

A dyadic product evaluated on a large frequency can be written as an additive cocycle over the doubling map. The corpus develops this systematically:

- `LacunaryProductToSum.lean` and its cosine variant take logarithms and expose the cocycle;
- `LogSineMeanZero.lean`, `LogCosineFourier.lean`, and related modules compute means and Fourier coefficients;
- `martingale`, `covariance`, `Gordin`, and `Birkhoff` modules control variance and moments;
- `SincProductShells.lean`, `ShellEnvelopeExplicit.lean`, and `GlobalDecayEnvelope.lean` convert orbit estimates to product envelopes; and
- `peak-ray` and `fixed-mantissa` modules isolate exceptional sequences where generic decay is not representative.

This architecture explains why one global power-law slogan is inadequate. The product has typical behavior, arithmetic rays, peak rays, and shell maxima, which require different quantifiers.

24.5 Transfer operators and certified spectral bounds

`DoublingMapTransfer.lean`, `TransferOperatorStep.lean`, `LasotaYorkeIteration.lean`, `TransferPositivity.lean`, `PerronRootExistence.lean`, and `PerronEigenvalueUnique.lean` package the positive-operator side. `CollatzWielandt.lean` supplies the variational comparison, and `BernsteinPositivity.lean` turns polynomial positivity on subintervals into exact certificates. The enclosure modules obtain a kernel-checked interval for the Perron root and related constants. Decimal endpoints are not accepted as floating-point evidence; they are rational inequalities discharged by finite polynomial certificates.

24.6 Moment matrices, orthogonal polynomials, and exact synthesis

The Rvachev law's rational moments feed `MomentHankelMatrix.lean`, `OrthogonalPolynomialConstruction.lean`, `OrthogonalPolynomialGramBridge.lean`, and the rational Jacobi/Gram modules. The resulting orthogonal polynomials have exact values, parity, and three-term recurrences.

The synthesis branch asks a different question: can a polynomial be represented or integrated exactly using shifted/scaled copies of u ? `RvachevPolynomialSynthesis.lean` and `LagrangeRvach`

`evSynthesis.lean` construct the coefficients. `CompositeMeshExactness.lean`, `CombDefectSeries.lean`, and `PolynomialCombExactness.lean` analyze comb quadrature. `RvachevSuperconvergentSynthesis.lean` selects parity-dependent endpoint or quarter phases and proves exactness through degree $v_2(M) + 1$ for every nonzero natural mesh, not only dyadic meshes.

24.7 Quarter-cell and inverse-germ technology

The quarter-cell modules form a compact local laboratory. They prove exact finite spline evaluations, a quarter quantile, local polynomial pieces, two-sided spline matching, a Catalan inverse germ, and geometric Richardson acceleration. `AlgebraicInverseGerm.lean` and its analytic/binomial companions separate formal coefficient inversion from analytic convergence. The Fabius-specific module `FabiusInverseQuarterJet.lean` then transports this general germ calculus to the inverse near the exact quarter value.

Inverse theory, Volterra calculus, non-elementarity, and computability

25.1 The totaled inverse as an order-theoretic object

`FabiusInverse.lean` begins with the order isomorphism `fabiusOrderIso` of the unit interval. Its inverse is extended by clamping the argument to $[0, 1]$, producing a total real function `fabiusInv`. The principal API includes `fabiusReal_fabiusInv`, `fabiusInv_fabiusReal`, `continuous_fabiusInv`, `monotone_fabiusInv`, `strictMonoOn_fabiusInv`, and `bijOn_fabiusInv`.

This order-isomorphism construction is preferable to an unbundled `Classical.choose`: continuity and monotonicity follow from the order topology, and comparison theorems become exact equivalences. Reflection gives

$$F^{-1}(1 - y) = 1 - F^{-1}(y),$$

including the exact midpoint. Exact dyadic evaluation of F immediately produces exact inverse values at the corresponding rational ordinates.

25.2 Endpoint steepness and the exact smoothness locus

Effective and sharp flatness estimates for F invert into lower bounds for F^{-1} . In filter form the API proves

$$\frac{F^{-1}(y)}{y} \longrightarrow +\infty \quad (y \downarrow 0),$$

and the reflected quotient diverges at one. The interior derivative also tends to infinity at both clamping values. Because the totaled inverse is constant outside $[0, 1]$ but has infinite interior slope, it is not differentiable at zero or one; it is smooth at every other real point. This precise statement is stronger and safer than the vague phrase “the inverse is singular at the endpoints.”

25.3 Sharp interval-mass and inverse-gap rigidity

`InverseModulus.lean` does more than prove a weak modulus inequality. Its current equality layer classifies the extremizers. For a nonnegative interval length h , the interval-mass profile has zero tails, and for $0 < h \leq 1$ it is strictly increasing on the left branch $[-h, (1 - h)/2]$ and strictly decreasing

on the right branch $[(1 - h)/2, 1]$, with constant tails beyond those ranges. Inside the unit interval, an interval of length h has the endpoint mass $F(h)$ exactly in the degenerate case $h = 0$, when its left endpoint is zero, or when its right endpoint is one.

Transport through the order isomorphism gives sharp inverse-gap statements. For $0 \leq u \leq v \leq 1$, equality in the ordered inverse-gap bound occurs exactly when $u = v$, $u = 0$, or $v = 1$. In the absolute version, equality for $u, v \in [0, 1]$ means that the inputs coincide or at least one input is an endpoint. The unit-interval hypotheses are essential: the totalized inverse has constant exterior tails and hence additional equality configurations on all of $\mathbb{R}$.

This is a useful Lean lesson. A global inequality may be most convenient for composition, while its equality theorem must expose the clamping geometry explicitly. Proving only the inequality would lose the rigidity information needed by later uniqueness and sharpness arguments.

25.4 Moduli and effective inversion

`InverseModulus.lean`, `FabiusInverseEffectiveContinuity.lean`, and `FabiusInverseLogarithmicModulus.lean` turn endpoint estimates into quantitative continuity. The logarithmic module defines the least scale index meeting the required endpoint bound and proves primitive-recursive reciprocal continuity witnesses at both ends. Its scope is deliberately narrower than the whole computability chain: it does not itself prove the endpoint-mass ceiling, tolerant bisection, sequential computability, the combined computable-real-function theorem, or a bit-complexity asymptotic.

`EffectiveMonotoneInverse.lean` implements a tolerant bisection scheme: approximate comparisons either safely choose a half interval or certify that the bracket is already narrow enough. Exact equality of computable reals is never required.

`EffectiveGapInverse.lean` packages a reusable alternative hypothesis: computable positive rational lower bounds on dyadic forward gaps. Together with `FabiusComputableSpline.lean`, `FabiusGapInversion.lean`, and `FabiusInverseComputable.lean`, it proves computability of the clamped inverse. The proof separates three notions that informal numerical accounts often mix:

1. exact rational evaluation at dyadic inputs;
2. uniform approximation with a proved modulus; and
3. an asymptotic running-time bound in the input bit length.

The facade proves the first two ingredients needed for computable analysis; it does not claim the third.

25.5 Normalized Volterra operators

The integration library is generic before it is Fabius-specific. For a continuous Banach-valued input f , the normalized integer-order Volterra operator has kernel

$$(V_n f)(x) = \frac{1}{(n-1)!} \int_a^x (x-t)^{n-1} f(t) \, dt \quad (n \geq 1).$$

`NormalizedVolterra.lean` proves the literal iteration law, strict fundamental-theorem identities, order addition, Taylor reconstruction, and finite polynomial commutators under the weakest appropriate normed-space or completeness assumptions.

The Fabius modules `FabiusAntiderivatives.lean` and `FabiusFractionalVolterra.lean` specialize these formulas to the signed global extension, the bounded function on its maximal valid range, and the folded up-function. A theorem written for the signed extension can therefore use the pure global dilation equation; a bounded-function theorem must retain the domain where the two agree.

25.6 Fractional order

`FractionalVolterra.lean` replaces the factorial by a Gamma normalization and allows positive real order. `FractionalVolterraCalculus.lean` proves order-raising and covariance statements, while `FractionalVolterraSemigroup.lean` establishes additive composition on ordered compact intervals under continuity and completeness. The `FractionalCDFLayerCake.lean` and `FabiusFractionalIntegral.lean` leaves connect this operator calculus to CDF and Fabius identities.

The formal boundary is explicit. Positive real orders with stated endpoint order and integrability hypotheses are proved. Complex-order formulas, unrestricted analytic continuation in the order, and iterated claims not represented by a named theorem remain outside the proved surface.

25.7 Layer-cake and inverse-pair integration

`SubgraphFubini.lean`, `SurvivalLayerCake.lean`, and `CDFLayerCake.lean` establish measure-theoretic identities by integrating indicator subgraphs. `InverseLayerCake.lean` and `InversePairIntegral.lean` combine two absolutely continuous monotone clocks on a rectangle, avoiding repeated ad hoc changes of variables. The Fabius inverse-pair formulas are then direct specializations. This is a good example of proof migration: a theorem first motivated by one special function becomes a generic measure/calculus tool.

25.8 Analyticity and non-elementarity

`NowhereAnalytic.lean` states the exact analytic locus for the bounded function rather than merely proving failure at a dense set. The non-elementarity chain `AlgebraicBranch.lean`, `ElementaryFunction.lean`, and `NotElementary.lean` formalizes a specific differential-algebraic obstruction with explicit hypotheses. `InverseBranch.lean` and `InverseNotElementary.lean` transport the obstruction to inverse branches. These results must not be paraphrased as broader transcendence claims: the formal theorem concerns the repository's defined elementary-function class and the stated branch domains.

The current q-series corpus

26.1 Why q-series are in the public Fabius facade

The dyadic parameter $q = 1/2$ appears in Gaussian-binomial expressions for Fabius values, geometric interpolation, and infinite sinc/Pochhammer products. The repository generalized those identities until the supporting q-calculus became a substantial library. The final 145 direct imports should therefore be read as a q-series subproject with Fabius bridges, not as 145 separate facts about the bounded function.

26.2 Finite q-binomial algebra

`FiniteQBinomialCore.lean` gives the division-free Gaussian coefficient. Reciprocity, inversion, Vandermonde, Cauchy, q-Pascal, specialization at $q = 1$ and $q = -1$, continuity, polynomial structure, palindromicity, and bounds are distributed across focused modules. Division-free statements are preferred when possible because they work over semirings and avoid nonzero denominator obligations. Quotient forms are supplied later under explicit field and nonvanishing hypotheses.

The universal polynomial and cyclotomic layers prove degree, monicity, constant coefficient, palindromicity, factorization, q-Lucas congruences, and carry criteria. q-Catalan polynomials and binary-word inversion statistics supply combinatorial interpretations. Theorems at roots of unity retain the primitive-root and integral-domain assumptions needed for cancellation.

26.3 Infinite q-Pochhammer products

`QPochhammerInfinite.lean` and `QPochhammerInfiniteBounds.lean` establish convergence and quantitative prefix control for $|q| < 1$. `QPochhammerEntire.lean` proves that, for a fixed strict contraction, the infinite product is entire in its symbol variable and identifies its zeros and analytic orders. Locally uniform convergence modules justify termwise analytic operations; logarithmic and order derivatives are handled separately.

`QPochhammerComplexOrder.lean` defines a ratio-based complex-order symbol. Its domain requires a nonzero nome and nonvanishing products. The facade does not erase these hypotheses by pretending the ratio is automatically entire in all parameters. Multiplicative dissections, integer-index formulas, partial fractions, modular asymptotics, and classical limits are each given their own proof layer.

26.4 q-gamma and Jackson calculus

`QGamma.lean`, `QGammaLogDerivative.lean`, `QGammaMultiplication.lean`, `QGammaLogConvex.lean`, `QGammaMeromorphic.lean`, and `QBohrMollerup.lean` build the q-gamma function from the infinite product through differential, convexity, meromorphic, and characterization results. `JacksonIntegral.lean` and `JacksonFundamentalExtension.lean` formalize the q-integral, both directions of the fundamental theorem, integration by parts, and rational certificates. `QBetaIntegral.lean` evaluates the positive-parameter q-beta integral and proves its recurrences under $0 < q < 1$.

26.5 Basic hypergeometric transformations

The modules `BasicHypergeometricSeries.lean`, `HeineTransformation.lean`, `QGaussSummation.lean`, `QGaussFullDomain.lean`, `QChuVandermonde.lean`, `QPfaffSaalschutz.lean`, `TwoPhiOneReversal.lean`, and `RamanujanOnePsiOne.lean` cover terminating, unilateral, and bilateral summations. Algebraic terminating proofs and analytic infinite proofs are kept separate. `BilateralSeriesConvergence.lean` supplies the convergence engine needed before an identity of bilateral sums can be asserted.

26.6 Theta products and sums of squares

`JacobiTripleProduct.lean` and its uniform companion provide the product identity, while `ThetaQuasiPeriodicity.lean`, `ThetaHeatEquation.lean`, `ThetaSimpleZeros.lean`, `JacobiImaginaryTransform.lean`, and `ThetaProductIdentities.lean` build the analytic theta API. Jacobi's cubic identity, the quintuple product, Schröter formulas, and Gaussian Fourier theta identities are separate leaves. The two-square and four-square modules then derive arithmetic generating-function consequences with their coefficient extraction made explicit.

26.7 Partitions, Bailey chains, and Rogers–Ramanujan

Partition modules formalize bounded parts, multiplicities, distinct/odd bijections, Durfee decompositions, Euler recurrences, stabilization, and generating functions. The Bailey family introduces pairs, inversion, lowering, chains, finite bounds, and infinite limits. It culminates in `RogersRamanujan.lean`, `AndrewsGordon.lean`, and `AndrewsGordonOne.lean`. The facade order reflects proof construction: finite algebra and bounds precede the infinite identities.

26.8 Finite geometry and cyclic sieving

Gaussian coefficients also count subspaces over finite fields. The subspace, Möbius, rank-matrix, and general-linear order-product modules connect the q-polynomials to finite geometry. `CyclicSievingSubsets.lean` evaluates appropriate Gaussian polynomials at roots of unity. These modules demonstrate why keeping the Gaussian coefficient division-free and functorial pays off: the same formal object supports interpolation, partitions, and finite-field enumeration.

26.9 Return bridges to Fabius and Thue–Morse

The closing modules specialize the general q-theory back to the project: `FabiusDyadicGaussianForm.lean` packages exact dyadic values; `FabiusGeneralLinearDenominator.lean` records denominator control; `ThueMorseQPochhammer.lean` and its infinite companion identify digital products; and `QPochhammerMultiplicativeDissection.lean`, `ThueMorseEulerFunction.lean`, and `ThueMorsePentagonal.lean` connect those products to classical q-series. This direction of dependency is important: the general identity is proved once, then the binary parameter is a corollary.

Lambert coordinates and phase-locked extrapolation

27.1 Two real branches with explicit domains

The repository defines the principal branch W_0 on $[-e^{-1}, \infty)$ and the lower branch W_{-1} on $[-e^{-1}, 0)$, with inverse equations, monotonicity, first derivatives, and endpoint/asymptotic bounds. `LambertBranchDichotomy.lean` and the branch geometry modules keep the two solutions separate at negative arguments. `LambertWAnalytic.lean` uses the analytic inverse-function theorem to prove real analyticity exactly where the forward derivative is nonzero: the open principal domain and the open lower-branch interval.

27.2 A generic autonomous-ODE engine

The new `AutonomousIteratedDeriv.lean` isolates a reusable chain-rule principle. Suppose on an open set

$$W'(x) = \varphi(W(x)).$$

If $G_0(w) = w$ and $G_{n+1}(w) = G'_n(w)\varphi(w)$, then

$$W^{(n)}(x) = G_n(W(x)).$$

The theorem `AutonomousODE.iteratedDeriv_eq_comp` asks for differentiability only on the range actually visited by W . This formulation is crucial for Lambert W , where the recurrence contains a denominator $1 + w$ and the proof must know that the chosen branch does not equal -1 on its open domain.

27.3 All derivatives of both Lambert branches

`LambertWHigherDerivatives.lean` defines polynomials P_n by

$$P_0(w) = 1, \quad P_{n+1}(w) = (1 + w)P'_n(w) - ((n + 1)w + 3n + 2)P_n(w).$$

It proves, for either real branch on its open domain,

$$W^{(n+1)}(x) = \frac{e^{-(n+1)W(x)} P_n(W(x))}{(1 + W(x))^{2n+1}}.$$

The same polynomial family works for both branches; only the first-derivative and range facts are branch-specific. Public endpoint checks include $P_1(w) = -(w + 2)$ and $P_2(w) = 2w^2 + 8w + 9$. The final declarations are `iteratedDeriv_principalLambertW` and `iteratedDeriv_lowerLambertW`.

27.4 Lambert coordinates in the Fabius saddle problem

The Fabius small-argument saddle is most naturally parameterized by an exact lower-Lambert coordinate. The modules `FabiusLambertPhase.lean`, `FabiusLambertRates.lean`, and `FabiusLambertSaddle.lean` establish the implicit phase and its comparison scales. Formal logarithm and all-order remainder modules then expand it in powers of $1/\log(1/x)$ with $\log \log(1/x)$ coefficients. The exact-phase theorem is primary: replacing the phase by a truncated elementary-log series is a separate composition step with its own remainder.

27.5 Phase-locked Richardson and Bell extraction

`LambertPhaseLockedRichardson.lean` applies geometric extrapolation to quantities whose error is naturally an expansion in the reciprocal Lambert phase rather than a simple power of the original variable. `LambertPhaseLockedBell.lean` and `LambertReciprocalAsymptotics.lean` supply the coefficient algebra. `FabiusLambertPhaseLockedPullback.lean`, `FabiusLambertPhaseExtraction.lean`, and its Bell companion pull the filtered result back to the Fabius endpoint variable.

The methodological lesson is broad: choose nodes on the asymptotic coordinate where the expansion is polynomial, not necessarily on the original x -axis. Lean makes the choice visible because the pullback, filter exactness, and remainder estimate live in separate declarations.

27.6 Power-exponential inverse calculus

The files from `PowerExponentialLambert.lean` through `PowerExponentialLambertFabiusCurvature.lean` apply Lambert branches to inverses of functions of the shape $x \mapsto x^\alpha e^{\beta x}$ and to the curvature of the resulting inverse coordinates. The generalized-coordinate and perturbed-lower-Lambert modules handle deformations needed by endpoint models. These modules should be read as reusable inverse-function calculus; the Fabius-named leaves are specializations at the phase parameters arising in the saddle equation.

Proof status, documentation boundaries, and maintenance

28.1 Kernel-checked, executable, conditional, and frontier claims

The project distinguishes several statuses that should not be collapsed:

Kernel-checked theorem. A declaration with a proof term accepted by Lean. Its assumptions, including typeclass and nonvanishing hypotheses, are part of the result.

Executable evaluator with correctness theorem. Code may be run, and a theorem identifies its output with a mathematical object. Execution alone is not the certificate.

Generated certificate checked by a theorem. A table or polynomial certificate lies outside the kernel until a proved checker consumes it.

Conditional theorem. A theorem is fully proved from a visible hypothesis whose truth may be open or separately unformalized.

Research-frontier statement. LaTeX may contain a conjecture, heuristic, numerical observation, or partial proof, but it is not represented as established by the primary Lean-backed exposition.

The facade imports claim-level audits and formal counterexamples to false draft claims. Importing an audit does not convert every sentence of the associated research document into a theorem.

28.2 Repository invariants

The project policy requires `set_optionautoImplicitfalse` in every file, a documentation comment on every non-private declaration, registration of new modules in the facade, and no `sorry`, `admit`, `axiom`, or `opaque`. The intended axiom set remains the ordinary classical quotient boundary recorded by the repository. These invariants are architectural: they make the long public surface searchable and prevent hidden placeholders from being mistaken for completed mathematics.

28.3 How to update this walkthrough again

A future update should begin from machine-checkable inventories rather than page counts in old PDFs:

1. pin the exact commit;
2. extract and count the direct facade imports;
3. diff the facade and its documentation comment against the previous pinned commit;
4. inspect new module headers and the commits that introduced them;
5. update the family narrative and the exact import appendix;
6. check date-stamped README and coverage counts before repeating them; and
7. compile the LaTeX three times, inspect the rendered PDF, and verify the fonts.

The appendix is deliberately generated from the facade order so that a missing new leaf is detectable as a count or diff error. Narrative completeness then becomes a review of families and bridges, not an unreliable attempt to remember hundreds of filenames.

28.4 What this walkthrough does not claim

This document is not a replacement for the primary mathematical synthesis, a complete API reference with every declaration type, or a proof that every file in a moving development has been built on one machine at the latest commit. It does provide a complete ledger of the pinned facade and a detailed map of the mathematical engines exposed there. Exact theorem statements, universe levels, implicit arguments, and local namespace choices must still be read from Lean.

Recurring proof-engineering patterns

29.1 Strong induction as the default arithmetic engine

Many sequences in the project depend on all smaller indices, not only on the predecessor. The idiomatic proof begins with strong induction:

```
refine Nat.strong_induction_on n ?_
intro n ih
-- ih : forall m < n, P m
```

A finite sum over `Finset.rangen` then supplies the inequality needed to invoke `ih`. It is often worth proving a helper lemma that converts membership in the range directly into the strict inequality expected by the hypothesis.

Strong induction also matches the highest-bit evaluator, whose recursive remainder is smaller but not necessarily one less. Using the same well-founded order for definition and correctness proof keeps the program and theorem structurally aligned.

29.2 Finite sums and index transport

The repository moves frequently among:

- `Finset.rangen`;
- subtype indices `Fin`;
- integer intervals;
- list positions; and
- antidiagonals describing coefficient convolution.

Rather than rewriting all indices inside a large expression, the code usually proves an equivalence between finite index types and invokes a reindexing theorem. A clean bijection proof has four parts: the forward map, inverse map, membership preservation, and left/right inverse equalities.

For even/odd splits, a specialized theorem is especially valuable:

$$\sum_{j < 2N} f(j) = \sum_{k < N} f(2k) + \sum_{k < N} f(2k + 1).$$

Once established, it drives Thue–Morse identities, convolution recurrences, and parity counts.

29.3 Casts as explicit interfaces

Exact theorems live in $\mathbb{N}$, $\mathbb{Z}$, or $\mathbb{Q}$; analytic theorems live in $\mathbb{R}$ or $\mathbb{C}$. Lean will insert canonical casts, but it will not silently prove that a finite sum commutes with the cast or that a natural exponent has the intended target type.

The code therefore proves and reuses lemmas of the forms

$$\begin{aligned} \left(\sum_{k \in s} q_k : \mathbb{Q} \right) \uparrow^{\mathbb{R}} &= \sum_{k \in s} (q_k : \mathbb{R}), \\ (2^n : \mathbb{Q}) \uparrow^{\mathbb{R}} &= (2 : \mathbb{R})^n, \\ \left(\binom{n}{k} : \mathbb{N} \right) \uparrow^{\mathbb{Q}} &= \binom{n}{k} \text{ interpreted in } \mathbb{Q}. \end{aligned}$$

In actual Lean syntax the upward arrows are ordinary coercions. Tactics such as `norm_cast`, `exact_mod_cast`, and `push_cast` are used after the expression is put in a form they recognize.

Lean pattern

Do not ask `norm_cast` to solve a goal containing unreduced divisions, nested sums, and conditionals. First rewrite the exact recurrence, move casts through finite sums, simplify powers, and only then invoke cast normalization. The repository’s helper lemmas reflect this staged approach.

29.4 Pointwise derivatives instead of `raw deriv`

A theorem of type `HasDerivAt f f' x` carries both differentiability and the derivative value. It composes under addition, multiplication, affine maps, logarithms, and exponentials. By contrast, an equality involving `deriv f x` often creates a side goal proving `DifferentiableAt f x` before it can be rewritten.

Accordingly, the calculus modules first prove `HasDerivAt` identities and derive `deriv` corollaries only when a source theorem is stated in that notation. Iterated derivatives are handled through `iteratedDeriv` or `ContDiff` lemmas with explicit order parameters.

29.5 Piecewise functions and boundary glue

The bounded function, fixed-point transform, splines, and endpoint representatives are all piecewise. The source follows a stable pattern:

1. define each branch as a total function;
2. prove branch continuity or differentiability;
3. prove equality of branch values at the boundary;
4. apply a piecewise glue theorem; and

5. derive simplified branch lemmas for later rewriting.

The simplified lemmas are often tagged for the simplifier only when their conditions are syntactically obvious. Overly aggressive `simp` tags on overlapping branches can create loops or choose an unintended normal form.

29.6 Local finiteness before infinite-sum algebra

For translate sums, the code first proves finite support at a point or on a neighborhood. Only then does it reindex or differentiate a `tsum`. This is safer than invoking general unconditional summability theorems on a series whose convergence is really trivial by support.

A typical theorem states that the summand is zero outside a finite integer interval. From this it derives summability, rewrites the `tsum` as a `Finset.sum`, performs the algebra, and converts back only if the public theorem uses `tsum` notation.

29.7 Integrability from support and bounds

The most common integrability proof has three ingredients:

- continuity or measurability of the integrand;
- compact support; and
- a uniform bound on that support.

The repository builds reusable support lemmas for F , up , their derivatives, and affine transforms. Polynomial or exponential multipliers remain bounded on a compact support, so moment and transform integrability follows from a generic compact-support theorem.

This style avoids global estimates that are stronger than needed. For instance, $x^n \text{up}(x)$ is integrable not because x^n is globally bounded, but because the product vanishes outside $[-1, 1]$.

29.8 Interchanging sums, limits, derivatives, and integrals

Every interchange in the project is supported by a named convergence theorem:

- finite sums commute with everything by algebra;
- infinite sums commute with integrals under a summable integrable majorant;
- limits pass under integrals by dominated or bounded convergence;
- derivatives pass through series under local uniform convergence of derivative series; and
- contour limits use explicit uniform tail bounds.

The proof normally constructs the majorant first, proves its summability or integrability, and only then invokes the interchange theorem. This order makes failures local: if an informal series is not uniformly controlled, the missing hypothesis appears before the main identity is attempted.

29.9 Big-O, little-o, and eventual domains

Asymptotic modules use filters rather than epsilon notation. A theorem of the form

$$f(x) = O(g(x)) \quad (x \rightarrow 0^+)$$

is represented through `IsBigO` at a within-neighborhood filter. The proof often begins with

```
filter_upwards [eventually_pos, eventually_small] with x hxpos hxsmall
```

and establishes a pointwise norm inequality.

Eventual positivity is particularly important for logarithms, divisions, and Lambert phases. The source proves threshold lemmas once and then includes them in filter-upwards blocks. Transitivity of Big-O and equivalence under asymptotically equal scales are used heavily to transfer results from the exact phase $\lambda(x)$ to $-\log x$.

29.10 Polynomial identities

Exact finite identities are proved at the polynomial level whenever possible. The usual techniques are:

- extensionality by coefficients;
- evaluation at an arbitrary point followed by `ring`;
- induction using polynomial addition and multiplication; and
- degree bounds plus a roots theorem.

For translation-invariant Thue–Morse sums, a degree bound and finite-difference theorem often give a cleaner proof than expanding every binomial coefficient.

29.11 Choosing automation deliberately

The development uses automation, but proofs are structured so each tactic sees an appropriate goal:

```
simp
  unfolds branch lemmas, finite-range membership, and standard algebraic identities.

norm_num
  closes concrete rational and natural computations, especially counterexamples and base cases.

ring / ring_nf
  handles polynomial identities after divisions and casts have been normalized.

linarith / nlinarith
  combines real inequalities once transcendental terms have been abstracted as variables with
  known bounds.

positivity
  proves positivity of products, powers, factorials, exponentials, and denominators.
```

`omega`

handles Presburger arithmetic for indices, ranges, and exponents.

`field_simp`

clears rational or field denominators after nonzero hypotheses are established.

`aesop`

resolves local structural goals, but is not used as a substitute for choosing the main induction or analytic theorem.

29.12 Public wrappers and internal strength

Theorem-producing modules tend to prove results in a strong reusable form: unrestricted numerators, arbitrary candidate functions, general coefficient rings, or explicit error constants. Paper-facing wrappers then add notation and specialize hypotheses.

When extending the library, search for the strongest internal theorem before reproving a source-shaped special case. Names ending in `_all`, `_global`, `_scalar`, `_correct`, `_transfer`, or `_of_i` `sFabi` often signal the reusable layer.

29.13 No placeholders and explicit corrections

The repository states that the development builds without `sorry`. More importantly, corrected claims are not represented by axioms that merely assume the missing fact. When a source formula is false, the project either adds the necessary hypothesis, changes the object to the signed extension, weakens the error term, or proves a counterexample.

This makes the dependency graph epistemically meaningful. A theorem about the Fabius function ultimately reduces to `mathlib`, Lean’s kernel, explicit definitions, and proved lemmas—not to a paper citation embedded as an axiom.

How to read and extend the code

30.1 Four recommended reading paths

Different goals call for different entry points.

30.1.1 Path A: existence and uniqueness

Read, in order:

1. `Basic.lean`;
2. `Differential.lean`;
3. `DyadicClosedForm.lean`;
4. `DyadicAnalytic.lean`; and
5. `Existence.lean`.

This path explains the abstract specification, generic derivative consequences, dyadic rigidity, and fixed-point implementation.

30.1.2 Path B: exact evaluation

Read:

1. `Arithmetic.lean`;
2. `DyadicCorrectness.lean`;
3. `DyadicClosedForm.lean`;
4. `DyadicAnalytic.lean`; and
5. `GlobalDyadic.lean`.

Keep an editor evaluation buffer open and inspect the types of table, Horner, and wrapper correctness theorems.

30.1.3 Path C: the historical Rvachev theory

Read:

1. `GlobalExtension.lean`;
2. `FourierProduct.lean`;
3. `FourierAnalytic.lean`;
4. `OriginalUniqueness.lean`;
5. `ProbabilityRepresentation.lean`; and
6. `StepApproximationLimit.lean`.

This path emphasizes compact support, transform refinement, convolutions, and pointwise approximants.

30.1.4 Path D: sharp asymptotics

Read:

1. `FabiusLogScale.lean`;
2. `FabiusLogSquaredAsymptotic.lean`;
3. `NegativeLaplace.lean`;
4. `PeriodicCorrection.lean`;
5. `MellinBose.lean`;
6. `FabiusLambertPhase.lean`;
7. `BromwichSaddle.lean`;
8. `FabiusSharpExactReduction.lean`;
9. `FabiusSharpAsymptotic.lean`; and
10. `FabiusFullAsymptoticExpansion.lean`.

Use the specialized central/tail and all-order modules as dependencies are encountered.

30.2 Adding a new exact identity

Suppose a new conjectured formula concerns $F(a/2^n)$. A productive workflow is:

1. state and test the identity entirely in $\mathbb{Q}$ using `fabiusDyadicValue`;
2. normalize representations and determine whether the bounded or global extension is intended;
3. prove the finite arithmetic identity, preferably through existing closed-form or q-binomial lemmas;
4. only then transport it to the analytic function using dyadic correctness; and
5. add a paper-facing wrapper if the notation comes from an external source.

This approach keeps exploratory computation cheap and avoids starting with real analysis when the content is combinatorial.

30.3 Adding a new moment theorem

For a new moment recurrence:

1. derive the analytic recurrence in a generic `IsFabius` context;
2. define an exact rational sequence only if computation or arithmetic consequences justify it;
3. prove equality by strong induction;
4. design a division-free numerator recurrence before attempting parity or divisibility; and
5. separate reduced-denominator conclusions into a later module.

The existing normalized moment files are templates for common-denominator induction.

30.4 Adding an asymptotic correction

A proposed correction term should be attached to the exact saddle reduction rather than inferred from numerical fitting. The robust route is:

1. identify which cumulant or periodic Fourier coefficient contributes at the desired order;
2. extend the formal coefficient recurrence;
3. prove a uniform central remainder at that order;
4. prove the tail is smaller than the target scale;
5. update the normalized mass expansion; and
6. transfer through logarithm and the exact phase.

If the desired final formula is in elementary logarithms, derive it as a corollary of the exact-phase theorem using `FabiusLambertAllOrderSmallArgument.lean`.

30.5 Testing a suspected source error

Formalization is most efficient when the claim is stress-tested early.

- Check endpoint and smallest-index cases by exact evaluation.
- Verify domains of every differentiated functional equation.
- Compare bounded and signed-global versions under translation.
- For asymptotic claims, sample subsequences of fixed dyadic phase.
- For uniform errors, examine knots and support boundaries.
- For q -dependent finite rows, test nondyadic locations before claiming parameter independence.

A small counterexample theorem is often more informative than a long stalled proof attempt.

30.6 Maintaining dependency hygiene

Before importing the umbrella module into a new file, ask which interface is actually needed. An exact lemma should normally depend on `Arithmetic.lean` or a narrow `q`-binomial module, not on `Existence.lean`. A generic analytic lemma should depend on `Basic.lean` and the relevant calculus modules, not on the canonical instance unless the theorem truly uses it.

A useful check is to run the import-graph tooling included through `mathlib`'s dependencies. Unexpected paths from asymptotic modules into paper aggregators or from arithmetic modules into analysis are signs that a helper theorem belongs in a lower layer.

30.7 Naming and theorem granularity

The repository's long module names encode the proof stage: `Raw`, `Scalar`, `ExactReduction`, `Transfer`, `Central`, `Tail`, `AllOrders`, and so on. New declarations should preserve this vocabulary where possible.

A theorem should expose one conceptual transition. Good boundaries include:

- closed formula equals Horner program;
- analytic recurrence equals exact recurrence;
- central integral equals Gaussian main term plus error;
- exact phase equals truncated elementary expansion plus error; and
- source statement follows from the stronger internal theorem.

Large proofs become navigable when these transitions have names.

30.8 A minimal downstream example

A downstream file that only needs exact evaluation and the analytic correctness theorem might begin as follows:

```
import FabiusFunction.DyadicAnalytic
import FabiusFunction.Existence

open Fabius

example (a n : Nat) :
  fabiusReal fabius (a / (2^n : Real)) =
    (fabiusDyadicValue a n : Real) := by
  -- apply the repository's dyadic correctness theorem,
  -- then discharge the chosen normalization side conditions
  simpa using fabius_dyadic_correct a n
```

The exact names and argument order should be confirmed in the editor; this schematic example illustrates the intended dependency level. Importing `FabiusFunction.lean` would also work, but would hide which layer supplies the result.

30.9 What to trust when prose and code differ

The kernel-checked theorem type is the final authority. Module headers and README tables explain intent and source correspondence, while the proof term certifies the exact statement. When a paper formula differs from a Lean wrapper, inspect:

1. whether the paper silently assumes an index is positive;
2. whether a removable singularity has been totalized;
3. whether a global extension replaces a bounded function;
4. whether endpoint representatives differ only almost everywhere; and
5. whether an asymptotic periodic term has been omitted.

The repository usually records the reason in the aggregator header or a nearby correction theorem.

Concluding perspective

The current `FabiusFunction` library is best understood as a formal mathematical ecosystem generated by one unusually fertile smooth function. Its central proof spine still has a clear shape: exact dyadic arithmetic and an abstract analytic specification are developed independently; existence and uniqueness identify the canonical bounded function; folding and signed translation produce the Rvachev and global forms; probability, Fourier, moment, and paper-facing APIs converge on the same object; and negative Laplace plus Lambert saddle analysis resolve the flat endpoint.

Around that spine, the repository has extracted the generic mechanisms that the proofs repeatedly need. Bell and Stirling transforms organize coefficient algebra. Weighted uniform series isolate the probability law. Geometric interpolation explains q -binomial extrapolation. Thue–Morse becomes a digital analysis library rather than a single sign trick. Generalized sinc products lead to zero divisors, lobe geometry, and transfer operators. Gram/Stieltjes machinery turns exact moments into orthogonal reconstruction. Volterra and inverse-pair calculus package repeated integration. Computable analysis turns exact evaluators and moduli into certified real functions. Lambert and q -series modules become reusable subprojects with named bridges back to Fabius.

This growth changes how the code should be read. Source order and facade order are release mechanics, not pedagogy. A reader should choose a mathematical bridge, descend to its generic engine, and then return to the specialization. The exhaustive ledger in [Appendix A](#) ensures that the map is current at the pinned commit; the chapters explain why those hundreds of leaves belong together.

Exact public-import census at the pinned commit

This appendix is the completeness ledger for the current walkthrough. It is a literal transcription of the 740 direct imports in `Analysis/FabiusFunction/Lean/FabiusFunction.lean` at commit `83428e0939114407e02b329621563f835d3e7c20`. The order and numbering are preserved. A module listed here may import many lower-level files that are not themselves direct facade imports; conversely, being listed proves public exposure, not a particular theorem count.

No.	Directly imported module
Cross-cutting seed modules (imports 1–7)	
1	<code>FabiusFunction.IntervalParseval</code>
2	<code>FabiusFunction.IntSumFolding</code>
3	<code>FabiusFunction.PascalTailIdentity</code>
4	<code>FabiusFunction.PeriodicOrbitGelfondBound</code>
5	<code>FabiusFunction.PascalParityPeriodic</code>
6	<code>FabiusFunction.RvachevHalfIntegerEnergy</code>
7	<code>FabiusFunction.LambertWElementaryBounds</code>
Symmetric functions, Bell–Stirling algebra, and formal series (imports 8–95)	
8	<code>FabiusFunction.CompleteHomogeneous</code>
9	<code>FabiusFunction.CompleteHomogeneousGenerating</code>
10	<code>FabiusFunction.CompleteHomogeneousBell</code>
11	<code>FabiusFunction.CompleteHomogeneousAsymptotics</code>
12	<code>FabiusFunction.SymmetricFunctionOrthogonality</code>
13	<code>FabiusFunction.FiniteTriangularTransform</code>
14	<code>FabiusFunction.TriangularKernelInverse</code>
15	<code>FabiusFunction.BinomialInversion</code>
16	<code>FabiusFunction.StirlingBasisChange</code>
17	<code>FabiusFunction.LahNumbers</code>
18	<code>FabiusFunction.BellStirling</code>
19	<code>FabiusFunction.StirlingGeneratingFunctions</code>
20	<code>FabiusFunction.EulerianNumbers</code>
21	<code>FabiusFunction.PartialBellPolynomials</code>
22	<code>FabiusFunction.BellGeneratingFunctions</code>
23	<code>FabiusFunction.BellComposition</code>
24	<code>FabiusFunction.BernoulliStirling</code>
25	<code>FabiusFunction.BellHomogeneity</code>
26	<code>FabiusFunction.StirlingOrdinaryGF</code>
27	<code>FabiusFunction.EulerianGeneratingFunctions</code>
28	<code>FabiusFunction.ExponentialRiordan</code>
29	<code>FabiusFunction.ExponentialFormula</code>
30	<code>FabiusFunction.OrderedBell</code>
31	<code>FabiusFunction.EulerianStirling</code>
32	<code>FabiusFunction.NewtonExpansion</code>

No.	Directly imported module (continued)
33	FabiusFunction.ComplementaryBell
34	FabiusFunction.StirlingTriangularExplicit
35	FabiusFunction.BellShiftEGF
36	FabiusFunction.OrdinaryBellComposition
37	FabiusFunction.EulerianEGF
38	FabiusFunction.BernoulliAppell
39	FabiusFunction.StirlingShiftedEvaluations
40	FabiusFunction.StirlingTransformEGF
41	FabiusFunction.StirlingSummations
42	FabiusFunction.StirlingNormalOrder
43	FabiusFunction.BinomialType
44	FabiusFunction.BellLeadingZeros
45	FabiusFunction.GenocchiNumbers
46	FabiusFunction.EulerianAlternating
47	FabiusFunction.EulerPolynomials
48	FabiusFunction.EulerReflection
49	FabiusFunction.FaulhaberOffset
50	FabiusFunction.RaabeMultiplication
51	FabiusFunction.TouchardCongruence
52	FabiusFunction.TouchardShiftEGF
53	FabiusFunction.TouchardPolyCongruence
54	FabiusFunction.TypeBEulerian
55	FabiusFunction.SecondOrderEulerian
56	FabiusFunction.ShiftOperatorCharP
57	FabiusFunction.BellUmbra
58	FabiusFunction.StirlingParity
59	FabiusFunction.CatalanGeneratingFunction
60	FabiusFunction.StirlingFirstReverse
61	FabiusFunction.BernoulliNewtonBasis
62	FabiusFunction.NorlundPolynomials
63	FabiusFunction.BellDerivative
64	FabiusFunction.AssociatedStirling
65	FabiusFunction.RStirling
66	FabiusFunction.FallingFactorialSeries
67	FabiusFunction.CauchyPolynomials
68	FabiusFunction.CoefficientRules
69	FabiusFunction.IteratedLeibniz
70	FabiusFunction.NorlundDiagonal
71	FabiusFunction.NarayanaNumbers
72	FabiusFunction.StirlingFirstModH
73	FabiusFunction.LagrangeInversion
74	FabiusFunction.LambertWSeries
75	FabiusFunction.FussCatalanSeries
76	FabiusFunction.InverseBellCoefficients
77	FabiusFunction.StirlingFirstDiagonal
78	FabiusFunction.DiamondPower
79	FabiusFunction.OrdinaryBellBivariate
80	FabiusFunction.BinomialInversionEGF
81	FabiusFunction.EulerianPolynomialRecurrence
82	FabiusFunction.BellFactorialRowSum
83	FabiusFunction.BellInversionTwo
84	FabiusFunction.StirlingSecondReverseColumn
85	FabiusFunction.ElementarySymmetricBell
86	FabiusFunction.ExpLog
87	FabiusFunction.CumulantBellFormula
88	FabiusFunction.NewtonPowerSumBell
89	FabiusFunction.ExpAddLog
90	FabiusFunction.SquareRootSeries
91	FabiusFunction.PowerSumOrdinaryBell
92	FabiusFunction.StirlingParityBitwise
93	FabiusFunction.StirlingSecondReverseRow
94	FabiusFunction.SymmetricFunctionTransform
95	FabiusFunction.SymmetricFunctionGenerating
Finite q-binomial algebra and geometric filters (imports 96–127)	
96	FabiusFunction.FiniteQBinomialCore
97	FabiusFunction.QBinomialReciprocity
98	FabiusFunction.QBinomialInversion

No.	Directly imported module (continued)
99	FabiusFunction.QPochhammerElementaryIdentities
100	FabiusFunction.QDifferenceAnnihilation
101	FabiusFunction.QBinomialTransform
102	FabiusFunction.QBinomialVandermonde
103	FabiusFunction.QBinomialCauchy
104	FabiusFunction.GaussianBinomialAtOne
105	FabiusFunction.GaussianBinomialAtNegOne
106	FabiusFunction.GaussianBinomialAtNegOneDerivative
107	FabiusFunction.GeometricCompleteHomogeneous
108	FabiusFunction.GeometricLagrange
109	FabiusFunction.LagrangeResidualMoments
110	FabiusFunction.GeometricResidualMoments
111	FabiusFunction.FinitePowerSeriesFilter
112	FabiusFunction.GeometricPowerSeriesFilter
113	FabiusFunction.AnalyticSeriesFilter
114	FabiusFunction.RvachevQBinomialFilter
115	FabiusFunction.GeometricQBinomialLagrange
116	FabiusFunction.QBinomialInversionSpecializations
117	FabiusFunction.GeometricRichardson
118	FabiusFunction.GeometricLagrangeWeights
119	FabiusFunction.GeometricLagrangeQBinomial
120	FabiusFunction.GeometricLagrangeQMoments
121	FabiusFunction.GeometricLagrangeCompleteHomogeneous
122	FabiusFunction.LambertPhaseLockedRichardson
123	FabiusFunction.LambertPhaseLockedBell
124	FabiusFunction.LambertReciprocalAsymptotics
125	FabiusFunction.FabiusLambertPhaseLockedPullback
126	FabiusFunction.FabiusLambertPhaseExtraction
127	FabiusFunction.FabiusLambertPhaseExtractionBell
Lambert branches and power-exponential calculus (imports 128–144)	
128	FabiusFunction.PrincipallLambertW
129	FabiusFunction.PrincipallLambertWAtTop
130	FabiusFunction.AutonomousIteratedDeriv
131	FabiusFunction.LambertWHigherDerivatives
132	FabiusFunction.LowerLambertW
133	FabiusFunction.GeneralizedLambertCoordinate
134	FabiusFunction.LambertBranchDichotomy
135	FabiusFunction.LambertWCurvature
136	FabiusFunction.LambertWBranchPointGeometry
137	FabiusFunction.LambertWBranchPointAsymptotics
138	FabiusFunction.PowerExponentialLambert
139	FabiusFunction.PowerExponentialLambertCalculus
140	FabiusFunction.PowerExponentialLambertCurvature
141	FabiusFunction.PowerExponentialLambertInverse
142	FabiusFunction.PowerExponentialLambertAsymptotics
143	FabiusFunction.PowerExponentialLambertFabius
144	FabiusFunction.PowerExponentialLambertFabiusCurvature
Weighted laws, facades, moments, Gram data, and activation (imports 145–199)	
145	FabiusFunction.WeightedUniformSeries
146	FabiusFunction.WeightedUniformDistribution
147	FabiusFunction.WeightedUniformSupport
148	FabiusFunction.ContinuousCDF
149	FabiusFunction.AffineIndependentCopy
150	FabiusFunction.GeometricUniformLaw
151	FabiusFunction.GeometricUniformMultisection
152	FabiusFunction.GeometricUniformUniqueness
153	FabiusFunction.GeometricUniformCDF
154	FabiusFunction.Paper05442
155	FabiusFunction.Paper06487
156	FabiusFunction.PaperFabiusAsymptotic
157	FabiusFunction.FinitePolynomialFilterExactness
158	FabiusFunction.FiniteAffineCombinationBounds
159	FabiusFunction.FabiusDiscreteLimitRates
160	FabiusFunction.EndpointTransferPolynomials
161	FabiusFunction.PerturbedLowerLambert
162	FabiusFunction.PaperKFoldThueMorse
163	FabiusFunction.FinitePolynomialFunctional

No.	Directly imported module (continued)
164	FabiusFunction.PolynomialMomentFunctional
165	FabiusFunction.FiniteMomentGram
166	FabiusFunction.GramStieltjes
167	FabiusFunction.FiniteMomentJacobi
168	FabiusFunction.GramStieltjesNaturality
169	FabiusFunction.PolynomialMomentGramDeterminant
170	FabiusFunction.LegendrePolynomialRational
171	FabiusFunction.WeightedScaleMultiplicity
172	FabiusFunction.BaseDigitMultiplicity
173	FabiusFunction.PartitionDefect
174	FabiusFunction.SpectralZetaWeighted
175	FabiusFunction.ShiftDifferenceWeights
176	FabiusFunction.NewtonMultiplicityAssembly
177	FabiusFunction.NewtonBasisGeneratingFunction
178	FabiusFunction.NewtonSpectralZeta
179	FabiusFunction.NewtonCoefficientUniqueness
180	FabiusFunction.BellPolynomialInversion
181	FabiusFunction.AppellSequence
182	FabiusFunction.MomentCumulantAlgebra
183	FabiusFunction.SaddleLogProductAlgebra
184	FabiusFunction.SaddleLogExpansionPowerSeries
185	FabiusFunction.CenteredMomentCumulants
186	FabiusFunction.CenteredMomentParity
187	FabiusFunction.SinhDivBernoulliLog
188	FabiusFunction.HyperbolicActivation
189	FabiusFunction.ActivationSeries
190	FabiusFunction.ActivationAsymptotics
191	FabiusFunction.ActivationTaylor
192	FabiusFunction.ActivationSeriesAsymptotics
193	FabiusFunction.GeometricActivationDimension
194	FabiusFunction.GeometricActivationAsymptotics
195	FabiusFunction.ReciprocalExponentialGenerating
196	FabiusFunction.RvachevMomentAppell
197	FabiusFunction.ImplicitPowerSeries
198	FabiusFunction.QuadraticCompositionalInverse
199	FabiusFunction.PolynomialExpectationCumulant
Thue–Morse and digital analysis (imports 200–307)	
200	FabiusFunction.ThueMorseBasicLemmas
201	FabiusFunction.ThueMorseBitSupport
202	FabiusFunction.AffineDifferenceOrbit
203	FabiusFunction.ThueMorseBooleanCube
204	FabiusFunction.ThueMorseComplexExponential
205	FabiusFunction.BitPositionGenerating
206	FabiusFunction.ParityCharacter
207	FabiusFunction.ParityCharacterKernel
208	FabiusFunction.DigitCharacterCongruence
209	FabiusFunction.BinaryDigitFloor
210	FabiusFunction.RademacherSine
211	FabiusFunction.BaseDigitProduct
212	FabiusFunction.SignedRowSum
213	FabiusFunction.BaseDigitProuhet
214	FabiusFunction.RulerLogDerivative
215	FabiusFunction.RulerLogSeries
216	FabiusFunction.BaseBProduct
217	FabiusFunction.ThueMorseCubature
218	FabiusFunction.ThueMorseAutocorrelationLimit
219	FabiusFunction.BellPolynomialMoments
220	FabiusFunction.GeneralizedProuhetBlock
221	FabiusFunction.ProuhetBlockZeroOrder
222	FabiusFunction.BitPositionQBinomial
223	FabiusFunction.BinaryPartitionGreenInverse
224	FabiusFunction.DigitDefectCounting
225	FabiusFunction.DigitDefectBitForm
226	FabiusFunction.ThueMorseValuation
227	FabiusFunction.DyadicZeroMultiplicity
228	FabiusFunction.PascalDyadicMultiplicity
229	FabiusFunction.CentralBinomialValuation

No.	Directly imported module (continued)
230	FabiusFunction.PrimePowerBinomialValuation
231	FabiusFunction.ThueMorseDigits
232	FabiusFunction.ThueMorseEnumerators
233	FabiusFunction.ThueMorseRarefied
234	FabiusFunction.ThueMorseAperiodicity
235	FabiusFunction.ThueMorseSparseProuhet
236	FabiusFunction.ThueMorseBlockAlgebra
237	FabiusFunction.ThueMorseProductIdentity
238	FabiusFunction.ApproximationLadder
239	FabiusFunction.ThueMorseMoments
240	FabiusFunction.ThueMorseArtinSchreier
241	FabiusFunction.ThueMorseEulerTransform
242	FabiusFunction.ThueMorseLucasSupport
243	FabiusFunction.DigitalDensity
244	FabiusFunction.SubmaskDensityAsymptotic
245	FabiusFunction.ThueMorseWalsh
246	FabiusFunction.ThueMorseComplexProductBridge
247	FabiusFunction.ThueMorseSparseMoments
248	FabiusFunction.ThueMorseMixedDifference
249	FabiusFunction.ThueMorseSymmetricDifference
250	FabiusFunction.ThueMorseAutocorrelation
251	FabiusFunction.ThueMorseOverlapFree
252	FabiusFunction.ThueMorseRecurrence
253	FabiusFunction.ThueMorseMahler
254	FabiusFunction.ThueMorseBooleanMobius
255	FabiusFunction.ThueMorseMultinomial
256	FabiusFunction.ThueMorseFourier
257	FabiusFunction.ThueMorseSineProduct
258	FabiusFunction.ThueMorseFourierInversion
259	FabiusFunction.HalfIntegerOddDFT
260	FabiusFunction.SummableCyclicAlias
261	FabiusFunction.HalfIntegerAliasIdentity
262	FabiusFunction.AliasDyadicBridge
263	FabiusFunction.LambertShiftInverse
264	FabiusFunction.CyclicCharacterSums
265	FabiusFunction.AliasErrorBound
266	FabiusFunction.DyadicDerivativeFiltration
267	FabiusFunction.ThueMorseParseval
268	FabiusFunction.ThueMorseComplexity
269	FabiusFunction.ThueMorseArithmeticFilter
270	FabiusFunction.RealZPowProduct
271	FabiusFunction.ToeplitzHessenberg
272	FabiusFunction.ThueMorseHessenberg
273	FabiusFunction.ExponentialPartition
274	FabiusFunction.ThueMorsePartition
275	FabiusFunction.ExponentialBell
276	FabiusFunction.ThueMorseIntegerLift
277	FabiusFunction.ThueMorseBoundaryFlatness
278	FabiusFunction.ThueMorseDirichlet
279	FabiusFunction.ThueMorseCatalanSeries
280	FabiusFunction.ThueMorseCatalanConvolution
281	FabiusFunction.ThueMorseIntegerLiftEquation
282	FabiusFunction.ThueMorseIntegerLiftRecurrence
283	FabiusFunction.ThueMorseLacunaryQuotient
284	FabiusFunction.ThueMorseInfiniteProduct
285	FabiusFunction.ThueMorseMellin
286	FabiusFunction.ThueMorseWoodsRobbins
287	FabiusFunction.ThueMorseMasterProduct
288	FabiusFunction.ThueMorseBlockProducts
289	FabiusFunction.ThueMorseEntireContinuation
290	FabiusFunction.FrullaniIntegral
291	FabiusFunction.ReciprocalGammaJets
292	FabiusFunction.ThueMorseGDirichlet
293	FabiusFunction.ThueMorseGammaTower
294	FabiusFunction.ThueMorseQuarterProduct
295	FabiusFunction.ThueMorseDiscSeries
296	FabiusFunction.ThueMorseNaturalBoundary
297	FabiusFunction.ThueMorseComputability

No.	Directly imported module (continued)
298	FabiusFunction.WordComplexity
299	FabiusFunction.ThueMorseNewman
300	FabiusFunction.ThueMorseNewmanQuantitative
301	FabiusFunction.ThueMorseNewmanResidues
302	FabiusFunction.ThueMorseGelfond
303	FabiusFunction.ThueMorseGelfondSharpness
304	FabiusFunction.PeriodDoublingHankel
305	FabiusFunction.ThueMorseHankel
306	FabiusFunction.LacunaryRieszIntegral
307	FabiusFunction.RieszSharpness
Generalized Rvachev products, transforms, and synthesis (imports 308–390)	
308	FabiusFunction.GelfondLogisticBound
309	FabiusFunction.GeometricScaleProducts
310	FabiusFunction.GeneralizedRvachevProduct
311	FabiusFunction.WeightLinearityProducts
312	FabiusFunction.WeierstrassProductBound
313	FabiusFunction.SincLowerBound
314	FabiusFunction.SincProductPositive
315	FabiusFunction.RelativeHarmonicDominance
316	FabiusFunction.PlateauLocalization
317	FabiusFunction.PlateauDegree
318	FabiusFunction.PlateauConstant
319	FabiusFunction.UniformSplineStrictMono
320	FabiusFunction.QuarterCellEvaluation
321	FabiusFunction.PlateauLimit
322	FabiusFunction.QuarterQuantile
323	FabiusFunction.StieltjesInversion
324	FabiusFunction.PoissonApproximateIdentity
325	FabiusFunction.PoissonMassSwap
326	FabiusFunction.StieltjesPerron
327	FabiusFunction.GeneralQConditionNumber
328	FabiusFunction.GeometricToeplitzBridge
329	FabiusFunction.LimitConditionNumber
330	FabiusFunction.CumulantGridConstant
331	FabiusFunction.MultipleAngleBounds
332	FabiusFunction.TentMoment
333	FabiusFunction.FaberSchauderDefect
334	FabiusFunction.GeometricConvolutionTails
335	FabiusFunction.SmoothingOperatorInversion
336	FabiusFunction.SmoothingOperatorExponential
337	FabiusFunction.AlgebraicInverseGerm
338	FabiusFunction.AlgebraicInverseGermAnalytic
339	FabiusFunction.AlgebraicInverseGermBinomial
340	FabiusFunction.FabiusInverseQuarterJet
341	FabiusFunction.GeometricCgfTails
342	FabiusFunction.GeometricTailDictionary
343	FabiusFunction.GeometricUniformDictionary
344	FabiusFunction.GeometricSincFactorization
345	FabiusFunction.DyadicCombTrapezoid
346	FabiusFunction.IncompleteBetaMaster
347	FabiusFunction.MeasureCauchyTransform
348	FabiusFunction.MeasureCauchyMomentLaurent
349	FabiusFunction.MomentHankelMatrix
350	FabiusFunction.MomentHankelValues
351	FabiusFunction.StieltjesMomentLaurent
352	FabiusFunction.StieltjesCauchyTransform
353	FabiusFunction.StieltjesHerglotz
354	FabiusFunction.RefinementConditioning
355	FabiusFunction.EntireKernelCalculus
356	FabiusFunction.StieltjesLogFixedPoint
357	FabiusFunction.StieltjesResolventHierarchy
358	FabiusFunction.StieltjesGeneralizedOrder
359	FabiusFunction.OrthogonalPolynomialConstruction
360	FabiusFunction.OrthogonalPolynomialGramBridge
361	FabiusFunction.RvachevRationalJacobi
362	FabiusFunction.FabiusLegendreHankelDeterminant
363	FabiusFunction.FabiusLegendreRationalGram

No.	Directly imported module (continued)
364	FabiusFunction.FabiusLegendreRationalGramValues
365	FabiusFunction.OrthogonalPolynomialValues
366	FabiusFunction.OrthogonalPolynomialParity
367	FabiusFunction.OrthogonalPolynomialRecurrence
368	FabiusFunction.OrthogonalPolynomialJacobi
369	FabiusFunction.CompositeMeshExactness
370	FabiusFunction.RvachevPolynomialSynthesis
371	FabiusFunction.LagrangeRvachevSynthesis
372	FabiusFunction.CombFirstDefect
373	FabiusFunction.CompositeMeshSharpness
374	FabiusFunction.CombDefectSeries
375	FabiusFunction.RvachevSuperconvergentSynthesis
376	FabiusFunction.CauchySurvival
377	FabiusFunction.CauchyHigherPowers
378	FabiusFunction.MonomialCombFourier
379	FabiusFunction.MonomialCombDerivatives
380	FabiusFunction.MonomialCombAlias
381	FabiusFunction.MonomialCombExactness
382	FabiusFunction.PolynomialCombExactness
383	FabiusFunction.SincProductShells
384	FabiusFunction.RvachevFixedMantissaRay
385	FabiusFunction.SharpGelfondBound
386	FabiusFunction.CyclicSineProductBound
387	FabiusFunction.SincProductPeakRay
388	FabiusFunction.DoublingCocycleIdentities
389	FabiusFunction.DiscreteLacunaryParseval
390	FabiusFunction.SineProductRootsUnity
Lacunary Fourier decay, entire products, lobes, and transfer (imports 391–488)	
391	FabiusFunction.SincMeanBracket
392	FabiusFunction.PeakRayEnvelope
393	FabiusFunction.DoublingTransferAdjoint
394	FabiusFunction.GeometricCesaro
395	FabiusFunction.LogSineMeanZero
396	FabiusFunction.LogTanMeanZero
397	FabiusFunction.SquareLogIntegrable
398	FabiusFunction.MartingaleOrthogonality
399	FabiusFunction.CocycleCovarianceHalving
400	FabiusFunction.DirichletKernelCotangent
401	FabiusFunction.LogSineFourierCoefficient
402	FabiusFunction.CocycleParseval
403	FabiusFunction.LogCosineFourier
404	FabiusFunction.GordinParseval
405	FabiusFunction.ClausenCorollaries
406	FabiusFunction.DoublingMapTransfer
407	FabiusFunction.CovarianceLadder
408	FabiusFunction.DistanceCounting
409	FabiusFunction.BirkhoffVariance
410	FabiusFunction.BirkhoffProductBridge
411	FabiusFunction.BirkhoffMoments
412	FabiusFunction.ShellLinearity
413	FabiusFunction.BoundaryLayerMax
414	FabiusFunction.VarianceBookkeeping
415	FabiusFunction.SharpGelfondEquality
416	FabiusFunction.CesaroProfileSkeleton
417	FabiusFunction.LogMeanSkeleton
418	FabiusFunction.SincEulerProduct
419	FabiusFunction.SincCanonicalProduct
420	FabiusFunction.RvachevPochhammerFactorization
421	FabiusFunction.QPochhammerEntire
422	FabiusFunction.GeometricPochhammerNormalConvergence
423	FabiusFunction.GeneralizedCanonicalForm
424	FabiusFunction.LobeSignLaw
425	FabiusFunction.CanonicalIntegerPoint
426	FabiusFunction.LobeSignNegative
427	FabiusFunction.GeneralizedRvachevEntire
428	FabiusFunction.AlternatingNewtonFamily
429	FabiusFunction.AlternatingNewtonZeta

No.	Directly imported module (continued)
430	FabiusFunction.GeneralizedZeroDivisor
431	FabiusFunction.GeneralizedRvachevIdentifiability
432	FabiusFunction.LobeSignComplete
433	FabiusFunction.GeneralizedRealBound
434	FabiusFunction.GeneralizedExponentialType
435	FabiusFunction.ClassicalGrowth
436	FabiusFunction.RMSTransferEigenfunctions
437	FabiusFunction.LogFactorConcavity
438	FabiusFunction.CentralLobeConcavity
439	FabiusFunction.LobeDerivativeFormulas
440	FabiusFunction.CentralLobeOnePeak
441	FabiusFunction.CentralLobePeakAtZero
442	FabiusFunction.LobeLogFactorization
443	FabiusFunction.SideLobeConcavity
444	FabiusFunction.OnePeakPerLobe
445	FabiusFunction.RvachevProductContinuity
446	FabiusFunction.KappaZeroInMeasure
447	FabiusFunction.LogProductMoments
448	FabiusFunction.BaselineDecay
449	FabiusFunction.FluctuationTightness
450	FabiusFunction.LatticeFiber
451	FabiusFunction.ZeroOrderTail
452	FabiusFunction.PhiZeroOrder
453	FabiusFunction.PhiRealSign
454	FabiusFunction.LobeSignCount
455	FabiusFunction.ValleyZeroOrders
456	FabiusFunction.ThueMorseLobeSign
457	FabiusFunction.LobeSignSelfSimilarity
458	FabiusFunction.RenormalizationIdentity
459	FabiusFunction.FourierLaplaceRotation
460	FabiusFunction.IntegerZeroLocalFactorization
461	FabiusFunction.LeaningJet
462	FabiusFunction.IntegerZeroAnalyticOrder
463	FabiusFunction.ScaledInfiniteProducts
464	FabiusFunction.GeometricReciprocalGamma
465	FabiusFunction.GeometricSincCharacteristicFunction
466	FabiusFunction.DyadicGammaOrder
467	FabiusFunction.ReciprocalIntegerGammaZeros
468	FabiusFunction.SincPrefixBound
469	FabiusFunction.GlobalDecayEnvelope
470	FabiusFunction.LacunaryMeanSharp
471	FabiusFunction.KappaDictionary
472	FabiusFunction.DyadicReflection
473	FabiusFunction.CollatzWielandt
474	FabiusFunction.LacunaryProductToSum
475	FabiusFunction.LacunaryProductToSumCosine
476	FabiusFunction.CenteredRvachevThueMorseFourier
477	FabiusFunction.LogSeriesMultiplicative
478	FabiusFunction.BaseLayerRegrouping
479	FabiusFunction.ShellEnvelopeExplicit
480	FabiusFunction.ThueMorseSinhProduct
481	FabiusFunction.RvachevFourierDecay
482	FabiusFunction.BoundednessSkeleton
483	FabiusFunction.TransferOperatorStep
484	FabiusFunction.LasotaYorkeIteration
485	FabiusFunction.TransferPositivity
486	FabiusFunction.PerronRootExistence
487	FabiusFunction.PerronEigenvalueUnique
488	FabiusFunction.DiagonalSelection
Measure, Laplace, Legendre, inverse, Volterra, and computability (imports 489–559)	
489	FabiusFunction.MeasureEqualization
490	FabiusFunction.NegativeLaplace
491	FabiusFunction.SubgraphFubini
492	FabiusFunction.SurvivalLayerCake
493	FabiusFunction.CDFLayerCake
494	FabiusFunction.ProbabilityLaplaceMoments
495	FabiusFunction.UnitLaplaceMomentBounds

No.	Directly imported module (continued)
496	FabiusFunction.PeriodicCorrection
497	FabiusFunction.MellinBose
498	FabiusFunction.MellinFinitePart
499	FabiusFunction.BoseFinitePartIntegral
500	FabiusFunction.PeriodicMean
501	FabiusFunction.PeriodicRegularity
502	FabiusFunction.LaplacePeriodicSecondOrder
503	FabiusFunction.EndpointLogDerivativeExpansion
504	FabiusFunction.LaplaceMomentBoundsSharp
505	FabiusFunction.FabiusTranslatedLegendreSeries
506	FabiusFunction.FabiusLegendreLeastSquares
507	FabiusFunction.LegendreGaunt
508	FabiusFunction.LegendreGauntClosedForm
509	FabiusFunction.FabiusLegendreEnergy
510	FabiusFunction.FabiusLegendreRationalEnergy
511	FabiusFunction.FabiusLegendreGaunt
512	FabiusFunction.FabiusLegendreGauntClosedForm
513	FabiusFunction.FabiusSquareEnergyFourier
514	FabiusFunction.FabiusLegendreTranslateBlocks
515	FabiusFunction.FabiusLegendreTranslateSeries
516	FabiusFunction.FabiusBinaryReductionSeries
517	FabiusFunction.FabiusComputability
518	FabiusFunction.Monotonicity
519	FabiusFunction.Regularity
520	FabiusFunction.Convexity
521	FabiusFunction.EffectiveFlatness
522	FabiusFunction.SharpFlatness
523	FabiusFunction.FabiusInverse
524	FabiusFunction.InverseModulus
525	FabiusFunction.FabiusInverseEffectiveContinuity
526	FabiusFunction.FabiusInverseLogarithmicModulus
527	FabiusFunction.MidpointEndpointTransfer
528	FabiusFunction.InverseMidpointDefect
529	FabiusFunction.InverseLayerCake
530	FabiusFunction.InversePairIntegral
531	FabiusFunction.QuantileTransport
532	FabiusFunction.InverseQuarterAnchor
533	FabiusFunction.QuarterCatalanGerm
534	FabiusFunction.QuarterCatalanRichardson
535	FabiusFunction.QuarterSplineLocalPolynomial
536	FabiusFunction.QuarterSplineTwoSided
537	FabiusFunction.GlobalBounds
538	FabiusFunction.NormalizedVolterra
539	FabiusFunction.FractionalVolterra
540	FabiusFunction.FractionalVolterraCalculus
541	FabiusFunction.FractionalVolterraSemigroup
542	FabiusFunction.FractionalCDFLayerCake
543	FabiusFunction.FabiusFractionalIntegral
544	FabiusFunction.FabiusFractionalVolterra
545	FabiusFunction.FabiusAntiderivatives
546	FabiusFunction.RvachevDerivativeDistribution
547	FabiusFunction.BoundedDerivatives
548	FabiusFunction.NowhereAnalytic
549	FabiusFunction.AlgebraicBranch
550	FabiusFunction.ElementaryFunction
551	FabiusFunction.NotElementary
552	FabiusFunction.InverseBranch
553	FabiusFunction.InverseNotElementary
554	FabiusFunction.FabiusComputableSpline
555	FabiusFunction.EffectiveMonotoneInverse
556	FabiusFunction.PrimrecNatPow
557	FabiusFunction.EffectiveGapInverse
558	FabiusFunction.FabiusGapInversion
559	FabiusFunction.FabiusInverseComputable
Closed saddle data, zeta calculus, enclosures, and probability bridges (imports 560–595)	
560	FabiusFunction.FabiusSaddleJetClosedForm
561	FabiusFunction.FabiusSaddleExponentClosedForm

No.	Directly imported module (continued)
562	FabiusFunction.FabiusSaddleJetStirling
563	FabiusFunction.FabiusSaddleLeadingCoefficient
564	FabiusFunction.FabiusSecondSaddleCorrection
565	FabiusFunction.FabiusSecondSaddleExpansion
566	FabiusFunction.EulerLogTransform
567	FabiusFunction.WeightedEulerTransform
568	FabiusFunction.EvenZetaSeries
569	FabiusFunction.EvenZetaValues
570	FabiusFunction.SincZetaSeries
571	FabiusFunction.SincZetaDyadic
572	FabiusFunction.GeometricSincZeta
573	FabiusFunction.LaplaceEndpointAllOrders
574	FabiusFunction.GeneralizedSincZeta
575	FabiusFunction.AlternatingNewtonCumulantKernel
576	FabiusFunction.ExponentialDiskMaximality
577	FabiusFunction.SincZetaRemainder
578	FabiusFunction.BernsteinPositivity
579	FabiusFunction.PerronRootEnclosure
580	FabiusFunction.KappaOneEnclosure
581	FabiusFunction.KappaConstantEnclosures
582	FabiusFunction.MeasureRefinement
583	FabiusFunction.UniformDigitThueMorseBridge
584	FabiusFunction.RandomSeriesLaw
585	FabiusFunction.GeometricUniformCauchy
586	FabiusFunction.CauchyTransform
587	FabiusFunction.CauchyCDF
588	FabiusFunction.CauchyRenormalization
589	FabiusFunction.UniformSplineWeakConvergence
590	FabiusFunction.DyadicSpecializations
591	FabiusFunction.EvaluatorPopcount
592	FabiusFunction.PrefixKernelCounting
593	FabiusFunction.ThueMorseFormalProduct
594	FabiusFunction.GlobalBlockFold
595	FabiusFunction.NormalizationDictionary
q-series, theta, partitions, Bailey chains, and closing bridges (imports 596–740)	
596	FabiusFunction.QPochhammerDissection
597	FabiusFunction.QPochhammerInfinite
598	FabiusFunction.QBinomialTheoremInfinite
599	FabiusFunction.JacobiTripleProduct
600	FabiusFunction.QPascalSummation
601	FabiusFunction.GaussianBinomialContinuity
602	FabiusFunction.QuantumBinomial
603	FabiusFunction.RogersSzegoPolynomial
604	FabiusFunction.PolynomialQDerivative
605	FabiusFunction.PolynomialQLeibniz
606	FabiusFunction.QGamma
607	FabiusFunction.LambertSeriesLog
608	FabiusFunction.QPochhammerDerivative
609	FabiusFunction.ClassicalPochhammerLimit
610	FabiusFunction.QPochhammerIntegerIndex
611	FabiusFunction.QPartialFractions
612	FabiusFunction.PolynomialQTaylor
613	FabiusFunction.GaussianBinomialUniversal
614	FabiusFunction.GaussianBinomialPolynomialStructure
615	FabiusFunction.QPochhammerInfiniteBounds
616	FabiusFunction.HeineTransformation
617	FabiusFunction.QGaussSummation
618	FabiusFunction.QPochhammerComplexOrder
619	FabiusFunction.BasicHypergeometricSeries
620	FabiusFunction.QMultinomial
621	FabiusFunction.QExponential
622	FabiusFunction.JacksonIntegral
623	FabiusFunction.QPochhammerLogDerivative
624	FabiusFunction.QPochhammerOrderDerivative
625	FabiusFunction.JacobiCubic
626	FabiusFunction.CyclotomicFactorization
627	FabiusFunction.CentralQBinomialReduction

No.	Directly imported module (continued)
628	FabiusFunction.PrimitiveRootBlock
629	FabiusFunction.QLucas
630	FabiusFunction.CyclotomicDivisibility
631	FabiusFunction.QCatalan
632	FabiusFunction.QBetaIntegral
633	FabiusFunction.NewtonInterpolation
634	FabiusFunction.GaussianBinomialInteger
635	FabiusFunction.GaussianBinomialComplexOrder
636	FabiusFunction.QPfaffSaalschutz
637	FabiusFunction.QuantumMultinomial
638	FabiusFunction.GaussianBinomialBounds
639	FabiusFunction.BinaryWordInversions
640	FabiusFunction.BinaryWeightTotals
641	FabiusFunction.BoxPartitions
642	FabiusFunction.TelescopingCertificate
643	FabiusFunction.VandermondeAlternant
644	FabiusFunction.CyclotomicFactorList
645	FabiusFunction.QGammaMultiplication
646	FabiusFunction.ComplexGaussianBinomial
647	FabiusFunction.BilateralSeriesConvergence
648	FabiusFunction.QGammaLogDerivative
649	FabiusFunction.PowerSeriesUniqueness
650	FabiusFunction.DivisorSumLambert
651	FabiusFunction.ReciprocalQBinomialAnalytic
652	FabiusFunction.TwoPhiOneReversal
653	FabiusFunction.QGammaLogConvex
654	FabiusFunction.QChuVandermonde
655	FabiusFunction.GeometricFilterBound
656	FabiusFunction.GaussianBinomialCumulants
657	FabiusFunction.GeometricSimplexSum
658	FabiusFunction.IntegerParameterTranslation
659	FabiusFunction.MultisetInversions
660	FabiusFunction.GeometricSimplexReindex
661	FabiusFunction.RatioExpansion
662	FabiusFunction.ThetaHeatEquation
663	FabiusFunction.QPochhammerModularAsymptotic
664	FabiusFunction.FormalQBinomial
665	FabiusFunction.BorweinReciprocity
666	FabiusFunction.GaussianUnimodality
667	FabiusFunction.TwoSquareTheorem
668	FabiusFunction.FourSquareTheorem
669	FabiusFunction.JacksonVeryWellPoised
670	FabiusFunction.QSeriesTruncation
671	FabiusFunction.RogersRamanujanPartitions
672	FabiusFunction.GLOrderProduct
673	FabiusFunction.SchroeterFormula
674	FabiusFunction.SubspaceMobius
675	FabiusFunction.BasicBernoulliLog
676	FabiusFunction.GaussianFourierTheta
677	FabiusFunction.JacksonRationalCertificate
678	FabiusFunction.JacobiImaginaryTransform
679	FabiusFunction.ThetaProductIdentities
680	FabiusFunction.CyclicSievingSubsets
681	FabiusFunction.SchurPrincipalSpecialization
682	FabiusFunction.RankMatrixCount
683	FabiusFunction.QExponentialFactorization
684	FabiusFunction.LambertTailLimit
685	FabiusFunction.SubspaceCount
686	FabiusFunction.AndrewsGordonFreeIndices
687	FabiusFunction.ClassicalHypergeometricLimit
688	FabiusFunction.QGammaClassicalLimit
689	FabiusFunction.ComplexGaussianClassical
690	FabiusFunction.QuintupleProduct
691	FabiusFunction.QBabbage
692	FabiusFunction.ThetaSimpleZeros
693	FabiusFunction.LambertSeriesLocallyUniform
694	FabiusFunction.QGaussFullDomain
695	FabiusFunction.RamanujanOnePsiOne

No.	Directly imported module (continued)
696	FabiusFunction.QGammaMeromorphic
697	FabiusFunction.BasicHypergeometricDivergence
698	FabiusFunction.GaussianIntegerPrimes
699	FabiusFunction.HessenbergDeterminant
700	FabiusFunction.JacksonFundamentalExtension
701	FabiusFunction.SumsOfSquaresGeneratingFunction
702	FabiusFunction.JacobiTripleProductUniform
703	FabiusFunction.RogersContinuant
704	FabiusFunction.RogersSeries
705	FabiusFunction.BaileyLemmaFull
706	FabiusFunction.BaileyChainBounds
707	FabiusFunction.AndrewsGordon
708	FabiusFunction.AndrewsGordonOne
709	FabiusFunction.BaileyLowering
710	FabiusFunction.BaileyChain
711	FabiusFunction.BaileyInversion
712	FabiusFunction.BaileyAux
713	FabiusFunction.BaileyPairs
714	FabiusFunction.BaileyUnitPairs
715	FabiusFunction.BaileyLimitInfinite
716	FabiusFunction.RogersRamanujan
717	FabiusFunction.QBinomialLocallyUniform
718	FabiusFunction.EulerPartitionRecurrence
719	FabiusFunction.PartitionStabilization
720	FabiusFunction.DurfeeAll
721	FabiusFunction.PartitionDistinctOdd
722	FabiusFunction.PartitionGeneratingFunction
723	FabiusFunction.PartitionBoundedParts
724	FabiusFunction.QBohrMollerup
725	FabiusFunction.PartitionMultiplicity
726	FabiusFunction.EulerTelescoping
727	FabiusFunction.GaussianBinomialPalindromic
728	FabiusFunction.ThetaQuasiPeriodicity
729	FabiusFunction.FabiusDyadicGaussianForm
730	FabiusFunction.FabiusGeneralLinearDenominator
731	FabiusFunction.ThueMorseComplexHalfAngle
732	FabiusFunction.ThueMorseQPochhammer
733	FabiusFunction.QPochhammerMultiplicativeDissection
734	FabiusFunction.ThueMorseEulerFunction
735	FabiusFunction.ThueMorseQPochhammerInfinite
736	FabiusFunction.ThueMorsePentagonal
737	FabiusFunction.LambertWAnalytic
738	FabiusFunction.MixedDifferenceForwardDiff
739	FabiusFunction.GeometricProuhetBlock
740	FabiusFunction.GeneralLinearThueMorseSum

A.1 Reproduction command

From a checkout at the pinned commit, the ledger can be regenerated by:

```
git show 83428e0939114407e02b329621563f835d3e7c20:\
Analysis/FabiusFunction/Lean/FabiusFunction.lean \
| grep '^import FabiusFunction\.' \
| sed 's/^import //' > fabius-public-imports.txt
wc -l fabius-public-imports.txt
```

The expected line count is 740, with first entry `FabiusFunction.IntervalParseval` and last entry `FabiusFunction.GeneralLinearThueMorseSum`.

Detailed dependency map

B.1 Core spine

The shortest conceptual spine from definitions to the canonical function is:

$$\boxed{\text{Arithmetic.lean}} \longrightarrow \boxed{\text{Basic.lean}} \longrightarrow \boxed{\text{Differential.lean}} \\ \longrightarrow \boxed{\text{DyadicAnalytic.lean}} \longrightarrow \boxed{\text{Existence.lean}}.$$

This display omits two important side inputs. `DyadicClosedForm.lean` supplies the exact finite polynomial matched by `DyadicAnalytic.lean`, while `DyadicCorrectness.lean` supplies evaluator correctness and representation invariance. The resulting canonical function is then consumed by moment, Fourier, probability, global-extension, and asymptotic modules.

B.2 Arithmetic spine

The main arithmetic chain is:

$$\text{Arithmetic.lean} \longrightarrow \left\{ \begin{array}{l} \text{NormalizedMoments.lean} \\ \text{NormalizedEvenMoments.lean} \\ \text{Parity.lean} \end{array} \right. \\ \longrightarrow \left\{ \begin{array}{l} \text{HalfMomentDenominator.lean} \\ \text{TwoAdic.lean} \\ \text{DenominatorBound.lean} \end{array} \right. \longrightarrow \text{PaperStatements.lean}.$$

`AnalyticMoments.lean` and `ExactInversePower.lean` connect this chain to the canonical function before the source-facing wrappers are assembled.

B.3 Original-paper spine

The original Rvachev paper is represented by two parallel paths:

$$\begin{array}{l} \text{differential/refinement law} \rightarrow \text{Fourier recurrence} \rightarrow \text{sinc product} \rightarrow \text{uniqueness,} \\ \text{finite elementary laws} \rightarrow \text{convolutions} \rightarrow \text{weak convergence} \rightarrow \text{step approximants.} \end{array}$$

Their principal modules are `OriginalUniqueness.lean`, `FourierProduct.lean`, `ProbabilityRepresentation.lean`, `WeakConvergence.lean`, and `StepApproximationLimit.lean`. `OriginalPaperSupplement.lean` adds the remaining numbered theorems and prose claims.

B.4 Asymptotic spine

The sharp endpoint proof has the following high-level flow:

$$\begin{aligned}
 \text{AnalyticMoments.lean} &\rightarrow \text{NegativeLaplace.lean} \rightarrow \left\{ \begin{array}{l} \text{PeriodicCorrection.lean} \\ \text{MellinBose.lean} \end{array} \right. \\
 &\rightarrow \text{BromwichSaddle.lean} \rightarrow \text{FabiLambertPhase.lean} \\
 &\rightarrow \left\{ \begin{array}{l} \text{central modules} \\ \text{tail modules} \end{array} \right. \rightarrow \text{FabiSharpExactReduction.lean} \\
 &\rightarrow \text{FabiSharpAsymptotic.lean} \rightarrow \text{FabiFullAsymptoticExpansion.lean}.
 \end{aligned}$$

The actual source graph is finer because the all-order proof separates formal coefficient algebra, small-argument Lambert expansion, Gaussian contraction, central uniformity, tail flatness, and transfer lemmas.

Paper theorem map

C.1 Arias de Reyna, arXiv:1702.05442

Source item	Lean declaration or family	Principal proof location
Theorem 1	IsOriginalFabius; existence-and-uniqueness wrapper	OriginalUniqueness.lean, with canonical existence from the core development
Lemma 1	finite-convolution probability convergence variants	ProbabilityRepresentation.lean, WeakConvergence.lean
Theorem 2	stepApproximant_tendsto_rv achevUp and endpoint-corrected variants	StepApproximationLimit.lean
Theorem 3	probability representation of the up-function	ProbabilityRepresentation.lean
Theorem 4	declarations grouped as original_ theorem_four_a, _b, _c	OriginalPaperSupplement.lean, Fourier and differential helpers
Nonanalyticity	smoothness plus failure of analyticity	OriginalPaperSupplement.lean, differential and Taylor modules
Theorem 5	Poisson-summation identity	PoissonSummation.lean
Theorem 6	moment formulas	AnalyticMoments.lean, OriginalPaperSupplement.lean
Theorem 7	global rationality/dyadic statement	GlobalDyadic.lean, OriginalPaperSupplement.lean

C.2 Arias de Reyna, arXiv:1702.06487

Source item	Lean declaration or family	Principal proof location
Proposition 1	proposition_one	PaperStatements.lean; exact moment foundations
Proposition 2	proposition_two and real/removable-singularity version	MomentPowerSeries.lean, PaperStatements.lean

Source item	Lean declaration or family	Principal proof location
Proposition 3	<code>proposition_three</code>	analytic/exact moment recurrence modules
Proposition 4	<code>proposition_four</code>	moment and generating-function modules
Question 5	<code>reshetnikovQuestion</code>	recorded as a predicate/question in <code>PaperStatements.lean</code>
Proposition 6	<code>proposition_six</code>	exact arithmetic and dyadic modules
Theorem 7	<code>theorem_seven</code>	<code>DyadicClosedForm.lean</code> , <code>DyadicAnalytic.lean</code>
Proposition 8	<code>proposition_eight</code>	differential and dyadic Taylor modules
Theorem 9	<code>theorem_nine</code> , strengthened all-parameter form	<code>Paper06487Supplement.lean</code> , global/differential helpers
Lemma 1	<code>lemma_one</code> , with corrected order condition	<code>PaperStatements.lean</code> and arithmetic helpers
Proposition 10	<code>proposition_ten</code>	moment denominator normalization
Definition 12	<code>dyadicDenominator</code>	<code>PaperStatements.lean</code>
Theorem 13	<code>theorem_thirteen</code> and denominator bound	<code>DenominatorBound.lean</code>
Proposition 15	<code>proposition_fifteen</code>	normalized moments and divisibility
Conjecture 16	<code>conjecture_sixteen</code>	recorded without asserting a proof
Theorem 17	<code>theorem_seventeen</code>	parity and denominator modules
Proposition 18	<code>proposition_eighteen</code>	exact numerator/denominator analysis
Proposition 19	<code>proposition_nineteen</code>	exact arithmetic and parity
Theorem 20	<code>theorem_twenty</code>	<code>TwoAdic.lean</code>
Theorem 21	<code>theorem_twenty_one</code>	<code>TwoAdic.lean</code> ; inverse-dyadic valuation
Proposition 22	initial and full variants of <code>proposition_twenty_two</code>	<code>Paper06487Supplement.lean</code> , dyadic/global modules

C.3 How to use the map

The declaration named after a source theorem is normally the best citation target in downstream formal work. The principal proof location is the best learning target. When the two differ, the wrapper is preserving source notation while the internal file proves a stronger or more reusable theorem.

Annotated classical-core module guide

The following guide groups the principal files by role. The development contains many fine-grained helper modules; the descriptions emphasize the transition each module contributes rather than every local lemma.

Module	Role in the development
Core interfaces and construction	
<code>Arithmetic.lean</code>	Exact natural/rational sequences, binary weight, Thue–Morse signs, product normalizations, inverse-power values, and executable dyadic evaluators.
<code>Basic.lean</code>	The unit-interval codomain, bounded-function type, abstract IsFabius specification, up-function, signed extension names, transforms, moments, and generating functions.
<code>Differential.lean</code>	Derivative identities for any IsFabius candidate, including iterated scale formulas and reflected branches.
<code>Existence.lean</code>	Banach fixed-point construction on symmetric bounded continuous functions, calculus bootstrap, canonical object, and abstract uniqueness packaging.
<code>ScaleTranslation.lean</code>	Reusable chain-rule formulas for affine rescaling and translation of Fabius and up-functions.
<code>GlobalExtension.lean</code>	Local finiteness, smoothness, unit-interval agreement, and translation laws of the signed global extension.
<code>GlobalDyadic.lean</code>	Exact dyadic identities for the signed extension with unrestricted and nonreduced representatives.
Dyadic formulas and verified computation	
<code>DyadicClosedForm.lean</code>	Closed finite Taylor formulas, highest-bit decompositions, and equivalence among exact dyadic expressions.
<code>DyadicCorrectness.lean</code>	Termination-facing invariants, table-prefix stability, Horner correctness, refinement invariance, and evaluator representation independence.
<code>DyadicAnalytic.lean</code>	Finite Taylor recurrence for an abstract IsFabius function and proof that analytic dyadic values equal the exact evaluator.
<code>ExactInversePower.lean</code>	Links inverse-power Fabius values with exact moments and table entries.
<code>TaylorReduction.lean</code>	Finite Taylor-reduction identities used to move between dyadic cells.

Module	Role in the development
<code>FabiusDyadicLogBounds.lean</code>	Explicit logarithmic bounds for inverse-dyadic values, used in endpoint asymptotics.
<code>DyadicSharpConditional.lean</code>	Conditional sharp statements at dyadic points under specified recurrence estimates.
<code>DyadicSharpDecomposition.lean</code>	Exact decomposition of dyadic logarithms into main and correction pieces.
<code>FabiusDyadicSharpCumulant.lean</code>	Cumulant organization specialized to inverse-dyadic asymptotics.
<code>FabiusDyadicSharpAsymptotic.lean</code>	Sharp asymptotic statements along the inverse-power sequence.
Moments, arithmetic, and denominators	
<code>AnalyticMoments.lean</code>	Compact-support integration, total mass, moment and half-moment recurrences, and identification with exact rational sequences.
<code>MomentPowerSeries.lean</code>	Entire series associated with moments and inverse-power values; bridge between coefficients and analytic generating functions.
<code>NormalizedMoments.lean</code>	Division-free normalization theorem for half moments using factorial and Mersenne products.
<code>NormalizedEvenMoments.lean</code>	Natural numerator and common-denominator formula for even centered moments.
<code>Parity.lean</code>	Lucas-theorem parity analysis and oddness of natural moment numerators.
<code>TwoAdic.lean</code>	Reduced numerator/denominator parity and exact two-adic valuation consequences.
<code>DenominatorBound.lean</code>	Common-denominator divisibility and explicit denominator bounds for source-facing arithmetic results.
<code>HalfMomentDenominator.lean</code>	Denominator control specialized to half moments and inverse-power values.
<code>BernoulliRecurrences.lean</code>	Translation of moment recurrences into Bernoulli-number identities used by the arithmetic paper.
<code>LaplaceMoments.lean</code>	Moment identities in the Laplace-transform normalization.
<code>LaplaceMomentBounds.lean</code>	Uniform factorial/exponential bounds needed for transform series and differentiation.
<code>FabiusRecurrenceSequence.lean</code>	Auxiliary exact recurrence sequence used in asymptotic and dyadic estimates.
Fourier, probability, and the first paper	
<code>FourierProduct.lean</code>	Convergence and continuity of the infinite sinc product and compatibility with finite products.
<code>FourierAnalytic.lean</code>	Fourier transform formula for the up-function, inversion consequences, and moment-series identification.
<code>OriginalUniqueness.lean</code>	Fourier-refinement proof of uniqueness for Rvachev's original compactly supported characterization.
<code>ProbabilityRepresentation.lean</code>	Finite convolution laws, characteristic functions, and the probabilistic representation of the up-function.
<code>WeakConvergence.lean</code>	Convergence of finite probability measures against bounded continuous tests.

Module	Role in the development
<code>StepMeasureBridge.lean</code>	Relates discrete/step approximants to their associated finite measures.
<code>EarlyMeasureBridge.lean</code>	Foundational bridge between early convolution approximants and measure formulations.
<code>StepApproximationLimit.lean</code>	Coefficient monotonicity, endpoint-corrected step functions, and pointwise convergence by an interval-average squeeze.
<code>EarlyApproximants.lean</code>	Definitions and elementary properties of the finite approximants used before the limiting theorem.
<code>PoissonSummation.lean</code>	Poisson-summation theorem in the repository's Fourier normalization.
<code>OriginalPaperSupplement.lean</code>	Remaining numbered and prose results of the first paper, including exact Taylor formulas and nonanalyticity.
<code>Paper05442.lean</code>	Aggregator and coverage audit for arXiv:1702.05442.
The arithmetic paper	
<code>PaperStatements.lean</code>	Source-numbered propositions, theorems, question, definition, and conjecture for arXiv:1702.06487.
<code>Paper06487Supplement.lean</code>	Prose-level and strengthened arithmetic-paper consequences, with corrected hypotheses and bounded/global distinctions.
<code>Paper06487.lean</code>	Compact aggregator for the complete arithmetic-paper formalization.
Finite q-binomial and Thue–Morse algebra	
<code>HalfQBinomial.lean</code>	Exact q-binomial coefficients at $q=1/2$, q-Pascal recurrence, Mersenne normalization, and a finite q-binomial theorem.
<code>FabiusRawQBinomialFormula.lean</code>	Raw coefficient-valued finite q-binomial identity before scalar or analytic specialization.
<code>FabiusRawQBinomialScalar.lean</code>	Scalar evaluation of the raw finite identity.
<code>FabiusQBinomialFormula.lean</code>	Principal exact Thue–Morse/q-binomial formula for unrestricted natural parameters.
<code>FabiusQBinomialScalarFormula.lean</code>	Scalar-valued form of the principal q-binomial theorem.
<code>FabiusDyadicQBinomialScalar.lean</code>	Specialization using exact dyadic Fabius values.
<code>FabiusQBinomialTaylor.lean</code>	Finite Taylor/coefficient analysis in the q parameter and exact remainder organization.
<code>FabiusGlobalQBinomialSeries.lean</code>	Convergent global series identified with the signed extension.
<code>FabiusBinaryReductionSeries.lean</code>	Binary-reduction telescoping series, including the necessary initial endpoint term.
<code>FabiusParityPowerSeries.lean</code>	Power series whose coefficients reflect binary parity and Thue–Morse cancellation.
Discrete-limit family	
<code>FabiusUniformSpline.lean</code>	Centered finite Thue–Morse splines, support, knot compatibility, and uniform bounds.
<code>FabiusComplexShiftSpline.lean</code>	Taylor control of complex shifts of finite splines.

Module	Role in the development
<code>FabiusDiscreteLimitToeplitz.lean</code>	Toeplitz-type averaging principle for normalized finite rows.
<code>FabiusDiscreteLimitComplexShift.lean</code>	Vanishing of complex-shift corrections under the limit scaling.
<code>FabiusDiscreteLimitIntegration.lean</code>	Final integration/telescoping step identifying the discrete limit with the Fabius extension.
Coarse endpoint asymptotics	
<code>FabiusLogScale.lean</code>	Defines the dyadic logarithmic coordinate and proves the domain-correct delay identity for the logarithm.
<code>FabiusLogSquaredAsymptotic.lean</code>	Quadratic logarithmic leading term, floor/ceiling transfer to real scale, and an explicit $O(t \log t)$ remainder.
<code>FabiusSmallArgumentScale.lean</code>	Filter and scale conversions between x tending to zero and the large logarithmic variable.
<code>FabiusFlatness.lean</code>	Proof that the Fabius function is smaller than every positive power at zero.
<code>FabiusDecayComparison.lean</code>	Comparison with exponential-flat functions such as $\exp(-c/x)$.
<code>FabiusLogMainDefect.lean</code>	Definition and first bounds for the defect after subtracting the quadratic logarithmic main term.
Negative Laplace, Mellin, and periodicity	
<code>NegativeLaplace.lean</code>	Infinite product and absolutely convergent logarithm, exact dilation equation, quadratic decomposition, periodic correction, and exponential tail.
<code>PeriodicCorrection.lean</code>	Continuity of the multiplicatively periodic correction by locally uniform convergence.
<code>PeriodicMean.lean</code>	Mean value and normalization of the period-one logarithmic correction.
<code>PeriodicRegularity.lean</code>	Differentiability and higher regularity of the periodic correction.
<code>PeriodicFourier.lean</code>	Fourier coefficients and nonconstancy of the periodic correction.
<code>MellinBose.lean</code>	Mellin transform of the Bose kernel and Gamma/zeta factorization.
<code>MellinFinitePart.lean</code>	Finite-part continuation after subtraction of singular Laurent terms.
<code>BoseFinitePartIntegral.lean</code>	Convergent integral representation for the regularized Bose/Mellin contribution.
<code>GammaSecondOrder.lean</code>	Second-order Gamma-function expansions and bounds used in saddle normalization.
<code>LaplacePeriodicSecondOrder.lean</code>	Second-order transform asymptotics retaining the periodic correction.
Bromwich and Lambert saddle analysis	
<code>FabiusBromwichInput.lean</code>	Analytic and decay hypotheses needed to apply Bromwich inversion to the Fabius transform.
<code>BromwichSaddle.lean</code>	Vertical-contour inversion, saddle-line selection, and central/tail decomposition.
<code>FabiusLambertPhase.lean</code>	Exact implicit Lambert phase solving the saddle equation.

Module	Role in the development
<code>FabiusLambertRates.lean</code>	Growth, comparison, and eventual positivity estimates for the exact phase.
<code>FabiusLambertSaddle.lean</code>	Substitution of the Lambert phase into the exponent and saddle identities.
<code>FabiusLambertDerivativeBounds.lean</code>	Uniform bounds for derivatives/cumulants at the saddle scale.
<code>FabiusLambertMinorArc.lean</code>	Modulus loss away from the central saddle window.
<code>FabiusLambertTailFlat.lean</code>	Exponential tail bounds upgraded to every inverse-power scale.
<code>FabiusSharpConstant.lean</code>	Exact normalization constant in the sharp asymptotic formula.
<code>FabiusSharpLambertMain.lean</code>	Assembly of the explicit Lambert-phase main term.
<code>FabiusSharpLambertTransfer.lean</code>	Transfer of estimates stated in saddle variables to the small-x filter.
<code>FabiusSharpExactReduction.lean</code>	Exact decomposition into explicit main term, normalized saddle mass, and forward-tail error.
<code>FabiusExplicitSharpTransfer.lean</code>	Conversion between exact sharp expressions and source-facing elementary forms.
<code>FabiusSharpAsymptoticTransfer.lean</code>	General transfer lemmas for logarithmic and exponential asymptotic equivalence.
<code>FabiusSharpAsymptotic.lean</code>	Correct sharp formula with periodic phase, first error bound, and asymptotic equivalence.
Central integral and all-order machinery	
<code>FabiusSaddleReduction.lean</code>	Algebraic reduction of the inversion integral to a normalized central weight.
<code>FabiusSaddleCentral.lean</code>	Leading Gaussian approximation in the central window.
<code>FabiusSaddleCentralLambert.lean</code>	Central estimate expressed at the exact Lambert saddle.
<code>FabiusSaddleCentralRadiusAsymptotics.lean</code>	Growth and compatibility conditions for the chosen central radius.
<code>FabiusSaddleTail.lean</code>	Coarse tail bound outside the central window.
<code>FabiusSaddleReferenceWeight.lean</code>	Normalized Gaussian reference integrals and identities.
<code>FabiusSaddleReferenceTail.lean</code>	Gaussian reference tail estimates.
<code>FabiusFirstSaddleCorrection.lean</code>	Explicit first non-Gaussian correction from cubic and quartic cumulants.
<code>FabiusSaddleCoefficientRecurrence.lean</code>	Recursive definition and correctness of all-order expansion coefficients.
<code>FabiusSaddlePolynomialCoefficients.lean</code>	Polynomial coefficient extraction for the rescaled central exponent.
<code>FabiusSaddleExpansionCoefficients.lean</code>	Final scalar coefficients after Gaussian contraction.
<code>FabiusSaddleExponentAllOrders.lean</code>	Arbitrary-order expansion of the local exponent with uniform remainder.

Module	Role in the development
FabiusSaddleCentralAllOrders.lean	Uniform central-integral expansion to every inverse phase order.
FabiusSaddleTailAllOrders.lean	Tail smaller than every requested inverse phase power.
FabiusSaddleMassAllOrders.lean	All-order expansion of the normalized saddle mass.
GaussianPolynomialContraction.lean	Linear functional sending even monomials to Gaussian moments and odd monomials to zero.
GaussianPolynomialWholeIntegral.lean	Exact whole-line Gaussian integrals of coefficient polynomials.
GaussianPolynomialTail.lean	Tail bounds for fixed Gaussian polynomials.
GaussianPolynomialTailAllOrders.lean	Uniform tail bounds compatible with arbitrary expansion order.
Lambert formal algebra and final expansion	
FabiusLambertFormalLog.lean	Formal logarithm/exponential algebra for truncated saddle series.
FabiusLambertHigherExpansion.lean	Higher explicit approximants to the exact Lambert phase.
FabiusLambertAllOrderAlgebra.lean	Coefficient identities for arbitrary-order Lambert substitution.
FabiusLambertAllOrderRemainder.lean	Analytic control of the remainder left by formal Lambert algebra.
FabiusLambertAllOrderSmallArgument.lean	Expansion of the exact phase in $\log(1/x)$ and $\log \log(1/x)$ to arbitrary fixed order.
FabiusFullAsymptoticExpansion.lean	All-order expansion of $\log F$ at the exact phase and weakened elementary-log corollaries.
FabiusWikipediaMain.lean	Concise corrected main asymptotic in a public-summary format.
FabiusWikipediaExpansion.lean	Corrected higher expansion suitable for comparison with encyclopedic formulas.
FabiusWikipediaObstruction.lean	Proof that omission of the nonconstant periodic term invalidates a vanishing residual.
FabiusQuotientExponentialMismatch.lean	Rigorous mismatch between Fabius decay and a proposed quotient-of-exponentials fit.
PaperFabiusAsymptotic.lean	Aggregator and audit for the coarse, sharp, corrected, and all-order asymptotic program.
Legendre and approximation	
LegendrePolynomial.lean	Rodrigues formula, parity, differential equation, monomial coefficients, norms, and endpoint data for Legendre polynomials.
LegendreSeriesConvergence.lean	Generic orthogonal-series convergence and coefficient-decay infrastructure for smooth functions.
FabiusLegendreCoefficients.lean	Exact even Legendre coefficients expressed through moments and inverse-dyadic values; odd coefficients vanish.
FabiusLegendreSeries.lean	Orthogonality, absolute/uniform convergence, and pointwise identification of the up-function series.

Module	Role in the development
<code>FabiusTranslatedLegendreSeries.lean</code>	Affine translation to the signed Fabius function and monomial expansion of the translated series.
<code>FabiusLegendreLeastSquares.lean</code>	Pythagorean identity and unique least-squares optimality of finite Legendre truncations.
K-fold Thue–Morse audit and auxiliary asymptotics	
<code>ThueMorseApproximation.lean</code>	Corrected finite interpolation/approximation theorems for k-fold Thue–Morse data.
<code>ThueMorseBinomialLog.lean</code>	Binomial and logarithmic identities underlying the draft’s proposed normalization.
<code>DraftCounterexamples.lean</code>	Exact and asymptotic counterexamples to false literal claims in the informal draft.
<code>StirlingAsymptotics.lean</code>	Factorial and binomial asymptotics needed for the k-fold scale.
<code>LowerLambertW.lean</code>	Correctly defined lower Lambert-type inverse branch, domain, monotonicity, and asymptotics.
<code>PaperKFoldThueMorse.lean</code>	Aggregator separating exact results, repaired statements, and formal counterexamples.
Top-level packaging	
<code>FabiusFunction.lean</code>	Umbrella import exposing the two papers, asymptotic audit, k-fold audit, negative-Laplace theory, and Legendre developments.

Remark D.1. This annotated table is a pedagogical map of the classical Fabius, paper, asymptotic, and Legendre proof spine inherited from the earlier walkthrough. It is intentionally not the current file inventory. The exact current public ledger is [Appendix A](#). Duplicate-looking module names usually mark distinct proof stages—for example, raw algebra versus scalar specialization, a central estimate versus an all-order strengthening, or a generic engine versus its Fabius specialization.

Corrections and critical distinctions

Topic	Informal hazard	Formal treatment
Bounded versus global F	A translation or dyadic formula is written for a clamped function on all of $\mathbb{R}$.	Use extendedFabius for the signed global statement; prove agreement with the bounded function on the unit interval.
Original finite convolution	A limiting convolution object is used where a finite-stage identity is required.	State the convolution theorem for each finite row, then pass to the limit through weak convergence.
Step endpoints	Closed interval indicators count a shared endpoint twice.	Define a half-weight endpoint representative when the theorem is pointwise; retain ordinary indicators for almost-everywhere/integral statements.
Fourier equation factor	A transform variable or scaling factor is dropped during differentiation/dilation.	Derive the identity from mathlib's Fourier derivative and affine-map theorems under a fixed convention.
Positive index	A denominator, derivative order, or truncated product is used at $n = 0$ although the formula assumes $n > 0$.	Add the explicit order hypothesis and prove the zero case separately when meaningful.
Removable singularity	A quotient such as $(e^z - 1)/z$ is treated as globally defined.	Define a total function with the limiting value at zero and prove a global power-series theorem.
Dyadic representation	A formula is proved only for a reduced numerator even though later sums produce unreduced fractions.	Prove refinement invariance and formulate global correctness for arbitrary representatives.
Taylor series	Exact finite Taylor truncation is mistaken for local analyticity.	Use a finite-order Taylor theorem with vanishing remainder; separately prove the analytic-at predicate fails.
Logarithmic delay equation	A branch-dependent derivative law is differentiated without a domain.	State the transformed identity only for $t \geq 1$ and use eventual filters thereafter.

Topic	Informal hazard	Formal treatment
Periodic asymptotic term	A bounded log-periodic correction is absorbed into an $o(1)$ or inverse-log error.	Prove the correction is nonconstant and exhibit phase subsequences with different residual limits.
Binary-reduction series	The outer sum begins after the endpoint contribution.	Start at $m = 0$ or include the missing boundary term explicitly so finite telescoping is exact.
Finite q dependence	Independence of the limiting value is promoted to independence of every approximant.	Preserve the complex q parameter at finite depth; prove its effect tends to zero through complex-shift estimates.
Saddle expansion composition	A truncated Lambert expansion is substituted to all orders without a composition remainder proof.	State the primary all-order theorem at the exact phase and prove elementary-log expansions separately with implicit-function error bounds.
Quotient exponential fit	Numerical agreement over a finite range is treated as asymptotic equivalence.	Compare logarithmic scales and prove the proposed quotient has the wrong endpoint decay.
K-fold normalization	A draft normalization or global error is assumed from plots or low-depth data.	Evaluate exact small cases and formalize counterexamples before proving a repaired statement.
Facade count	A date-stamped module or declaration count is repeated after the public facade has changed.	Pin a commit and count the live facade or regenerate the declaration audit at that commit; label older totals historical.
Lake serialization	LAKE_JOBS=1, -j 1, or a multi-target command is assumed to force one worker.	On the current Lake version, run one module target per invocation after dependencies are built; do not set LEAN_NUM_THREADS=0.
Zero set versus divisor	Equality of zero locations is treated as equality of generalized products.	Recover the exponent sequence from analytic orders with multiplicity; a bare zero set loses scale.
Formal versus analytic series	A formal coefficient identity is promoted to a convergent or asymptotic expansion.	Add a separate convergence domain and remainder theorem before using the formal identity analytically.
Computability versus complexity	A computable-real certificate is presented as a polynomial-time or bit-complexity result.	State the evaluator and modulus theorem exactly; prove a cost model separately.
Research frontier	A theorem-shaped statement in a LaTeX frontier volume is described as formalized.	Require a named, proved Lean declaration with matching hypotheses and conclusion.

E.1 What these corrections teach

The corrections fall into four recurring classes. Domain errors arise when a local functional equation is used globally. Representation errors arise when a bounded object is substituted for a signed extension or an almost-everywhere representative is substituted in a pointwise claim. Normalization errors arise from Fourier, convolution, and dyadic scaling conventions. Asymptotic errors arise when a bounded oscillation is mistaken for a small remainder or when a formal series is composed without analytic control.

Lean detects each class differently. A domain error appears as an unprovable interval side condition. A representation error appears as a concrete endpoint contradiction. A normalization error leaves an unmatched scalar factor. An asymptotic error usually survives local algebra but fails at the limit theorem; the repository then proves a negative result by constructing a subsequence or comparing scales.

Lean and mathematical glossary

Term or identifier	Meaning in this development
<code>UnitInterval</code>	The subtype $\{x : \mathbb{R} \mid 0 \leq x \leq 1\}$, used as the codomain of the bounded function.
<code>BoundedFabi</code>	A real-input function whose values carry the unit-interval bound in their type.
<code>fabiReal</code>	Real-valued projection of a subtype-valued bounded function, suitable for calculus and integration.
<code>IsFabi</code>	Abstract specification combining tail values, smoothness, symmetry, and the left-half derivative law.
<code>fabi</code> / <code>fabi_spec</code>	The canonical constructed object and the theorem that it satisfies the specification.
<code>rvachevUp</code>	Compactly supported even twin used in Fourier and probability arguments.
<code>extendedFabi</code>	Signed globally defined extension assembled from Thue–Morse-weighted translates of the up-function.
<code>HasDerivAt</code>	Pointwise derivative proposition carrying differentiability and the derivative value together.
<code>ContDiff</code>	Smoothness predicate to finite or infinite order.
<code>tsum</code>	Infinite sum over a type; translate sums are proved locally finite before this is manipulated.
<code>HasSum</code>	Proposition that a series has a specified sum; often more convenient than rewriting a <code>tsum</code> .
<code>Finset.rangen</code>	Finite set $\{0, \dots, n - 1\}$, the dominant indexing object for recurrences and finite products.
<code>Nat.strong_induction_on</code>	Well-founded induction giving hypotheses for every smaller natural index.
<code>IsBigO</code> , <code>IsLittleO</code>	Filter-based asymptotic bounds and negligibility.
<code>atTop</code>	Filter describing a variable tending to positive infinity.
<code>nhdsWithin0(Set.Ioi)</code>	Right-hand approach to zero, the endpoint filter used for $x \downarrow 0$.
<code>eventually</code>	A property holding on some set belonging to a filter; used to encode large-variable thresholds.
<code>Summable</code>	Absolute or unconditional summability in the relevant normed additive group.

Term or identifier	Meaning in this development
<code>intervalIntegral</code>	Oriented integral over real endpoints, useful for FTC and integration by parts.
<code>MeasureTheory.Integrable</code>	Whole-space integrability; usually proved from compact support and continuity.
<code>SchwartzMap</code>	Smooth rapidly decreasing function type used for Fourier injectivity and Poisson summation.
<code>fabiusDyadicValue</code>	Executable exact rational evaluator for a dyadic input representation.
<code>fabiusDyadicUnitAux</code>	Well-founded highest-bit recursion at the core of unit-interval evaluation.
<code>halfMoment / moment</code>	Exact rational recurrence sequences later identified with analytic integrals.
<code>halfMomentNumerator / momentNumerator</code>	Division-free natural recurrences used for parity and denominator proofs.
<code>binaryWeight</code>	Number of one-bits of a natural number.
<code>thueMorseSign</code>	Sign $(-1)^{s_2(n)}$.
<code>halfQBinomial</code>	Exact q-binomial coefficient specialized to $q = 1/2$.
<code>field_simp</code>	Tactic for clearing field denominators after nonvanishing hypotheses have been supplied.
<code>norm_cast / push_cast</code>	Tactics normalizing canonical casts across algebraic operations.
<code>ring_nf</code>	Polynomial normalizer used after divisions, casts, and finite sums are brought to algebraic normal form.
<code>filter_upwards</code>	Tactic for collecting eventual hypotheses and proving a pointwise eventual conclusion.

Example exploration snippets

The following snippets are intentionally schematic where theorem argument order may change. They show the style of interaction rather than replacing editor inspection of the current declarations.

G.1 Inspect the public objects

```
import FabiusFunction

open Fabius

#check UnitInterval
#check BoundedFabius
#check IsFabius
#check fabius
#check fabius_spec
#check rvachevUp
#check extendedFabius
```

G.2 Evaluate exact dyadic data

```
import FabiusFunction.Arithmetic

open Fabius

#eval fabiusInversePowTwoTable 8
#eval fabiusDyadicValue 1 1
#eval fabiusDyadicValue 5 4
#eval extendedFabiusDyadicValue 13 5
```

The returned objects are exact rationals. To evaluate a rational input through the recognition layer, inspect the declarations near `IsDyadicRational`, `dyadicExponent?`, and the evaluator wrappers in `Arithmetic.lean`.

G.3 Work generically under the specification

```
import FabiusFunction.Differential

open Fabius

variable (F : BoundedFabius) (hF : IsFabius F)

#check hF.symmetry_all
-- Search within Differential.lean for the iterated derivative
-- theorem whose interval hypotheses fit the desired dyadic cell.
```

The generic style is useful for proving a new consequence once and applying it both to the canonical function and to any future equivalent construction.

G.4 Find source-facing arithmetic theorems

```
import FabiusFunction.Paper06487

open Fabius

#check proposition_one
#check theorem_seven
#check theorem_twenty
#check theorem_twenty_one
#check conjecture_sixteen
```

A declaration representing a conjecture is a definition or proposition describing the assertion, not a proof of it.

G.5 Inspect asymptotic interfaces

```
import FabiusFunction.PaperFabiusAsymptotic

open Fabius

-- Use #check on the named main, obstruction, and full-expansion
-- theorems, then follow their exact-phase helper declarations.
#check Fabius.fabiusLogScale
```

Because filter statements can be long, editor hover information is often more useful than printing the entire type in a terminal.

G.6 Search tactics

Within an editor, useful text searches include:

```
IsFabius -> HasDerivAt
```

```
fabiusDyadicValue -> Real  
rvachevUp -> Fourier  
negativeLaplaceLog  
HasAsymptoticExpansion  
Legendre -> HasSum
```

Searching by the transition one wants—rather than by a source theorem number—usually finds the strong internal lemma.

Snapshot and reproducibility audit

H.1 Pinned sources

The current-state portions of this walkthrough were checked against the following repository objects at the pinned commit:

- `root lean-toolchain` and `lakefile.toml`;
- `Analysis/FabiusFunction/README.md` and `AGENTS.md`;
- `Analysis/FabiusFunction/docs/PAPER_COVERAGE.md`;
- `Analysis/FabiusFunction/docs/ASYMPTOTIC_COMPLETION_AUDIT.md`;
- the public facade `Analysis/FabiusFunction/Lean/FabiusFunction.lean`; and
- representative core and new leaves, including `Basic.lean`, `Arithmetic.lean`, `Existence.lean`, `FabiusInverse.lean`, `AutonomousIteratedDeriv.lean`, `LambertWHigherDerivatives.lean`, `LambertWAnalytic.lean`, and `ThueMorseSinhProduct.lean`.

The facade itself is the source of the 740-entry appendix. The family explanations use module headers and the facade documentation comment, which records hypotheses and non-claims as well as positive results.

H.2 Counts that are exact, historical, or intentionally omitted

Item	Status in this walkthrough
Pinned commit	Exact: <code>83428e0939114407e02b329621563f835d3e7c20</code> .
Lean and mathlib versions	Exact from the root configuration: <code>4.32.0</code> .
Direct facade imports	Exact: 740 distinct imports, reproduced in Appendix A .
Total source files	Not asserted as a new exact number; the transitive source tree is larger than the direct facade.
Public declarations	The README's 8,909 count is retained only as a 1 September historical checkpoint and is not claimed current.
Walkthrough page count	A property of the rendered artifact, reported after compilation rather than copied from the older document.

Full corpus build	Not inferred from a facade listing; build status belongs to the repository’s compiled-artifact audit and commit messages.
-------------------	---

H.3 Trust boundary

Lean proofs are checked by the Lean kernel under the imports and axioms visible in the source. Executed scripts, generated tables, decimal experiments, and PDF prose do not enlarge the theorem boundary unless a proved checker imports the data and certifies the relevant predicate. This walkthrough is expository: its evidence is the source, but the PDF itself is not a formal certificate.

H.4 Document validation

The delivered PDF was compiled from the delivered source with repeated `pdflatex` passes, then checked for page count, unresolved references, font embedding, and representative rendered pages. The ZIP contains only the matching `.tex` and `.pdf` files.

Bibliographic and repository notes

Bibliography

- [1] Vladimir Reshetnikov, *ProveIt: Analysis/FabiusFunction*, GitHub repository, pinned snapshot 83428e0939114407e02b329621563f835d3e7c20, inspected 3 September 2026. <https://github.com/VladimirReshetnikov/ProveIt/tree/83428e0939114407e02b329621563f835d3e7c20/Analysis/FabiusFunction>
- [2] Vladimir Reshetnikov, *FabiusFunction public Lean facade*, Analysis/FabiusFunction/Lean/FabiusFunction.lean, pinned snapshot as above. <https://github.com/VladimirReshetnikov/ProveIt/blob/83428e0939114407e02b329621563f835d3e7c20/Analysis/FabiusFunction/Lean/FabiusFunction.lean>
- [3] Vladimir Reshetnikov, *The Fabius Function and Rvachev's Up-Function: Exact Arithmetic, Probability, Fourier Analysis, and Corrected Sharp Asymptotics*, formalization-backed exposition in the ProveIt repository. https://github.com/VladimirReshetnikov/ProveIt/tree/main/Analysis/FabiusFunction/docs/Fabius_Function_and_Rvachev_Up
- [4] Vladimir Reshetnikov, *Paper coverage map for the Fabius formalization*, Analysis/FabiusFunction/docs/PAPER_COVERAGE.md. https://github.com/VladimirReshetnikov/ProveIt/blob/main/Analysis/FabiusFunction/docs/PAPER_COVERAGE.md
- [5] Juan Arias de Reyna, *An infinitely differentiable function with compact support: definition and properties*, arXiv:1702.05442 (2017 English translation of the 1982 article). <https://arxiv.org/abs/1702.05442>
- [6] Juan Arias de Reyna, *Arithmetic of the Fabius function*, arXiv:1702.06487, version 3 (2017). <https://arxiv.org/abs/1702.06487>
- [7] J. Fabius, *A probabilistic example of a nowhere analytic C^∞ -function*, Zeitschrift für Wahrscheinlichkeitstheorie und Verwandte Gebiete **5** (1966), 173–174.
- [8] B. Jessen and A. Wintner, *Distribution functions and the Riemann zeta function*, Transactions of the American Mathematical Society **38** (1935), 48–88.
- [9] Leonardo de Moura and Sebastian Ullrich, *The Lean 4 theorem prover and programming language*, Automated Deduction—CADE 28, 2021.
- [10] The mathlib community, *The Lean mathematical library*, CPP 2020 and the continuously maintained Lean 4 library. <https://github.com/leanprover-community/mathlib4>
- [11] Jean-Paul Allouche and Jeffrey Shallit, *Automatic Sequences: Theory, Applications, Generalizations*, Cambridge University Press, 2003.
- [12] R. M. Corless, G. H. Gonnet, D. E. G. Hare, D. J. Jeffrey, and D. E. Knuth, *On the Lambert W function*, Advances in Computational Mathematics **5** (1996), 329–359.

Document metadata

Title	The Fabius Function Ecosystem in Lean: A Complete, Current Walkthrough of the ProveIt Development
Source snapshot	main at 83428e0939114407e02b329621563f835d3e7c20, inspected 3 September 2026
Toolchain	Lean 4.32.0; mathlib v4.32.0
Primary source path	Analysis/FabiusFunction/Lean/FabiusFunction/
Umbrella facade	Analysis/FabiusFunction/Lean/FabiusFunction.lean
Exact facade census	740 distinct direct FabiusFunction.* imports
Document purpose	Human-readable architecture, theorem-family map, proof-engineering guide, and exact public-import ledger
Status boundary	Exposition only; exact theorem statements and proofs remain in Lean